ProctorAI

AI-assisted complaint, identity & access platform

COMPLETE HANDBOOK

ProctorAI Documentation Handbook

The complete reference set: requirements, architecture, API, data, security, deployment, operations and user guidance.

01 Software Requirements Specification
02 Architecture Document
03 API Handbook
04 Database & Migrations
05 Security & Hardening Guide
06 Production Deployment Runbook
07 Configuration Reference
08 Backup, Restore & Disaster Recovery Plan
09 Monitoring & Incident Runbook
10 Release, Versioning & Rollback Guide
11 CI/CD Pipeline Design
12 Contributor & Testing Guide
13 User & Administrator Manual

DOCUMENT ID	PRODUCT VERSION	DOCUMENT VERSION	ISSUED	CLASSIFICATION
PAI-HB-000	1.1.0	1.0	November 2026	Internal

CREATOR & DEVELOPER

Varshit Tyagi & Mohini Teotia

Document control

Title	ProctorAI Documentation Handbook
Document ID	PAI-HB-000
Audience	Everyone who builds, deploys, operates or uses ProctorAI
Product	ProctorAI 1.1.0
Status	Approved for use
Classification	Internal

Creator & Developer

Name	Role
Varshit Tyagi	Creator & Developer
Mohini Teotia	Creator & Developer

Revision history

Version	Date	Author	Change
1.0	November 2026	Varshit Tyagi, Mohini Teotia	Initial release for ProctorAI 1.1.0

Related documents

Every document in this handbook is also published as a standalone PDF in docs/pdf/.

© 2026 Varshit Tyagi & Mohini Teotia. ProctorAI is released under the MIT License. Product names mentioned are trademarks of their respective owners.

Documents in this handbook

01	Software Requirements Specification	PAI-SRS-001
02	Architecture Document	PAI-ARC-002
03	API Handbook	PAI-API-003
04	Database & Migrations	PAI-DB-004
05	Security & Hardening Guide	PAI-SEC-005
06	Production Deployment Runbook	PAI-DEP-006
07	Configuration Reference	PAI-CFG-007
08	Backup, Restore & Disaster Recovery Plan	PAI-BDR-008
09	Monitoring & Incident Runbook	PAI-OPS-009
10	Release, Versioning & Rollback Guide	PAI-REL-010
11	CI/CD Pipeline Design	PAI-CICD-011
12	Contributor & Testing Guide	PAI-DEV-012
13	User & Administrator Manual	PAI-USR-013

DOCUMENT 01

REQUIREMENTS · PAI-SRS-001

Software Requirements Specification

Functional, non-functional and interface requirements for the ProctorAI platform.

1. Introduction
2. Overall description
3. Functional requirements
4. External interface requirements
5. Non-functional requirements
6. Constraints, assumptions and risks
7. Verification and traceability

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) defines what ProctorAI must do and the qualities it must have. It is the reference for design, implementation, testing and acceptance of ProctorAI version 1.1.0. It follows the structure recommended by IEEE 830 / ISO/IEC/IEEE 29148.

1.2 Product scope

ProctorAI is an AI-assisted complaint, identity and access-management platform for educational institutions and similar organisations. It lets an institution:

- manage users, departments and roles with a fine-grained permission model enforced inside the database;
- receive, assign, process and resolve complaints with a complete audit history;
- identify people on-premises with face recognition (detection, enrollment and live recognition);
- notify people in real time on the web, desktop and mobile;
- extend core entities with administrator-defined custom fields;
- observe the health of every service, the database, the cache and the host.

The product ships as a set of containerised microservices with a web application. The same web application is also packaged as native desktop (Windows, macOS, Linux) and mobile (Android, iOS) apps.

1.3 Definitions, acronyms and abbreviations

Term	Meaning
RBAC	Role-based access control
RLS	PostgreSQL Row-Level Security: policies that filter rows per database session
Scope	The breadth of a permission: `own`, `department` or `all`
Entity	A protected resource type, for example `complaints` or `face_enrollments`
JWT	JSON Web Token, used as the short-lived access token
Refresh token	Long-lived opaque token used to obtain new access tokens; rotated on every use
SSE	Server-Sent Events, a one-way streaming channel used for live updates
FCM	Firebase Cloud Messaging, used for optional mobile push notifications
SCRFD / ArcFace	Face detection and face embedding neural network models
pgvector	PostgreSQL extension for vector similarity search
Subject	A person a complaint is about (`student` plus optional additional subjects)
Reporter	The user who raised a complaint (`raised_by`)

1.4 References

- ProctorAI `README.md`, `GUIDE.md`, `backend/README.md`, `backend/face-service/README.md`
- Flyway migrations in `backend/db-schema/src/main/resources/db/migration/`
- Postman collection in `backend/postman/`
- IEEE Std 830-1998 and ISO/IEC/IEEE 29148:2018

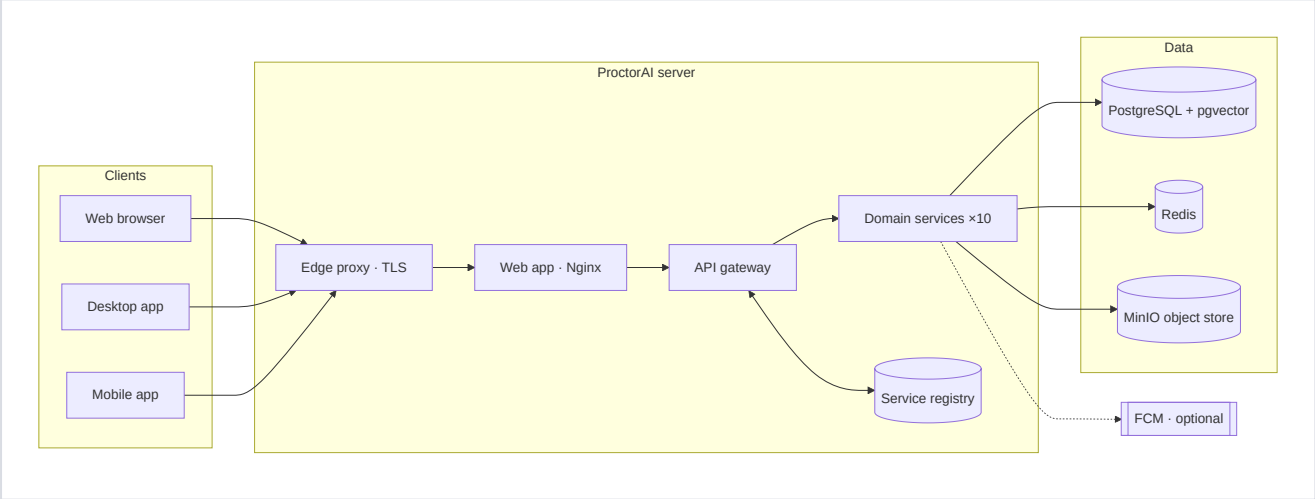
1.5 Document overview

Section 2 describes the product in general terms. Section 3 lists functional requirements by feature area. Section 4 defines external interfaces. Section 5 sets non-functional requirements. Section 6 records constraints, assumptions and dependencies. Section 7 traces requirements to their verification.

2. Overall description

2.1 Product perspective

ProctorAI is a self-contained system. It depends only on open-source infrastructure that ships with it: PostgreSQL 16 with pgvector, Redis 7.4, MinIO (S3-compatible storage) and the Caddy edge proxy. Optionally it connects to Firebase Cloud Messaging for push notifications. Face recognition runs entirely on the server, with no external AI API.



2.2 Product functions

Area	Summary
Authentication	Sign in with email and password, token refresh with rotation, sign out of one or all sessions, brute-force protection
Users	Create, edit, disable and delete users; bulk create; CSV/Excel import with templates
Departments	Create and manage departments and their members
Roles & permissions	Define roles with a level and a permission set over entity × action × scope

Area	Summary
Complaints	File, triage, assign, change status, resolve or reject, comment (public and internal), attach files and evidence, view history
Dashboard	Totals, status and priority breakdowns, 7-day trends, department load, overdue and critical counts
Face recognition	Enroll faces, recognise faces in uploads or from a live camera, bulk enroll, enrollment lock, audit log
Notifications	In-app inbox, real-time stream, preferences, device registration, broadcast sending, FCM integration and delivery log
Custom fields	Administrator-defined typed fields on users, departments, roles and complaints
System monitor	Live host, service, database, cache and storage metrics, with history and issues
Native apps	Desktop and mobile apps with notifications, deep links, camera and native downloads

2.3 User classes and characteristics

User class	Description	Typical default role
System administrator	Installs, configures and operates the platform; unrestricted access	`system_admin`
Administrator	Manages users, roles, departments, complaints and settings; unrestricted access	`admin`
Staff / teacher	Handles complaints of their departments; reads department users	`teacher` (level 20)
Student / member	Files and follows their own complaints; manages own profile and face enrollment	`student` (level 50)
Operator / DevOps	Deploys, backs up, monitors and upgrades the stack	Server access, not an app role
Integrator	Calls the REST API from other systems	Any role with the needed permissions

A lower role **level** means more seniority. Level 0 is reserved for superuser roles.

2.4 Operating environment

Component	Requirement
Server OS	64-bit Linux (x86_64) with Docker 25+ and Docker Compose 2.21+
Server hardware	Minimum 8 GB RAM and 20 GB free disk; 16 GB RAM recommended
Web clients	Chrome/Edge 111+, Firefox 128+, Safari 16.4+
Desktop apps	Windows 10/11 (x64), macOS 13+ (universal), Linux with WebKitGTK 4.1
Mobile apps	Android 8.0 (API 26)+, iOS 16.4+
Network	Public IPv4 address; inbound TCP 80, TCP 443 and UDP 443

2.5 Design and implementation constraints

- Backend: Java 21, Spring Boot 3.5, Spring WebFlux (reactive, non-blocking), R2DBC, Spring Cloud Gateway and Eureka.
- Frontend: React 19, TypeScript, Vite, Tailwind CSS 4, shadcn/ui, TanStack Router and Query; native shells built with Tauri 2.
- Authorization must be enforced in PostgreSQL through RLS policies and security functions, not only in service code.
- Database changes must be delivered as forward-only Flyway migrations. A released migration must never be edited.
- All runtime components must run as Docker containers orchestrated by Docker Compose.
- Face models originate from InsightFace's buffalo_l pack, which is licensed for non-commercial research use. Commercial distribution requires a license review.

2.6 Assumptions and dependencies

- The institution controls a domain name and can create DNS A records.
- Let's Encrypt (or ZeroSSL as fallback) is reachable for TLS certificates.
- Docker Hub is reachable when deploying the published images.
- FCM push requires the institution's own Firebase service account.

3. Functional requirements

Requirements use the keywords **shall** (mandatory) and **should** (recommended). Each requirement has an identifier for traceability.

3.1 Authentication and sessions (AUTH)

ID	Requirement
AUTH-01	The system shall authenticate users with email and password via `POST /auth/login`.
AUTH-02	The system shall issue a signed JWT access token (default lifetime 15 minutes) and an opaque refresh token (default lifetime 30 days).
AUTH-03	For browsers, tokens shall be delivered as `HttpOnly` cookies (`token` on `/`, `refresh_token` on `/auth`), `Secure` behind HTTPS and `SameSite=Lax`.
AUTH-04	Native apps shall be able to receive tokens in the response body by sending `X-Token-Delivery: body`.
AUTH-05	Each refresh shall rotate the refresh token. Reuse of an already rotated token shall revoke the whole token family.
AUTH-06	Users shall be able to sign out of the current session (`/auth/logout`) and of all sessions (`/auth/logout-all`).
AUTH-07	The gateway shall rate-limit sign-in attempts per client IP (default burst of 10 attempts, then 20 per minute).

ID	Requirement
AUTH-08	The system shall lock an account for a configurable period after a configurable number of consecutive failed sign-ins (defaults: 10 failures, 15 minutes).
AUTH-09	Disabled users shall not be able to sign in.
AUTH-10	Revoking or disabling a user shall revoke their refresh tokens.

3.2 Users (USR)

ID	Requirement
USR-01	Authorised users shall list users with paging (max 100 per page), sorting and filtering, including custom-field filters.
USR-02	Authorised users shall create a user with name, unique email, password (8–200 characters), roles, departments, custom fields, `enabled` and `verified`.
USR-03	Authorised users shall update (`PUT`) or partially update (`PATCH`) users, and delete one or many users.
USR-04	The system shall support bulk creation from JSON and import from CSV or Excel with a downloadable template, reporting progress and per-row results.
USR-05	Every user shall be able to read their own profile, roles, permissions and departments (`/users/me/*`).
USR-06	A user shall manage only themselves and more junior users (higher role level), unless they are a superuser. Peers on the same or a parallel role require the explicit `same_role_peers` or `parallel_role_peers` permissions.
USR-07	The system shall prevent deleting, disabling or removing the role of the last system administrator, and users shall not change their own roles or enable/disable their own account.
USR-08	A user shall never grant a permission they do not hold themselves (`PERMISSION_ESCALATION`).

3.3 Departments (DEP)

ID	Requirement
DEP-01	Authorised users shall create, read, update, patch and delete departments with a unique `code`.
DEP-02	Authorised users shall add and remove department members in bulk.
DEP-03	Department membership shall drive `department` -scoped permissions.

3.4 Roles and permissions (ROL)

ID	Requirement
ROL-01	The system shall define permissions as a combination of entity, action (`read` , `write` , `update` , `delete`) and scope (`own` , `department` , `all`).

ID	Requirement
ROL-02	Authorised users shall create roles with a name, description, level (1–1000), `revealIdentity` flag, custom fields and permissions.
ROL-03	Authorised users shall replace, add or remove a role's permissions.
ROL-04	System roles (`system_admin` , `admin`) shall not be deletable or downgradable.
ROL-05	The permission catalogue shall be readable to build the permission matrix in the UI.

3.5 Complaints (CMP)

ID	Requirement
CMP-01	A user shall file a complaint with title (≤200), description (≤5000), optional category (default `general`), priority (`low` , `medium` default, `high` , `urgent`), department, student and up to 50 additional subjects and custom fields.
CMP-02	Complaints filed by a user without `department` or `all` write scope shall always start as `pending` .
CMP-03	Staff shall change status among `pending` , `reviewed` , `resolved` and `rejected` , with an optional note and resolution. Resolving or rejecting shall set `resolved_at` .
CMP-04	Staff shall assign a complaint to a user.
CMP-05	A reporter without staff scope shall only edit their complaint while it is `pending` ; after that it is locked (`COMPLAINT_LOCKED`).
CMP-06	Every status change shall be recorded in an immutable history with actor, from/to status, note and time.
CMP-07	Users shall add comments (≤4000 characters). Staff may mark comments `internal` , which hides them from the reporter and subjects.
CMP-08	Users shall attach files via presigned upload URLs, confirm the upload, list files and obtain time-limited download URLs.
CMP-09	The reporter's identity shall be hidden from people without `revealIdentity` unless the complaint sets `revealReporter` .
CMP-10	Users shall list, filter, sort and page complaints within their permission scope.
CMP-11	The system shall provide dashboard statistics (`/complaints/stats`) covering totals, status, priority, open, assigned to me, overdue, critical open, 7-day trends, average resolution time and department load.
CMP-12	Parties to a complaint shall be notified of creation, assignment, status changes and comments.

3.6 Face recognition (FACE)

ID	Requirement
FACE-01	The system shall detect faces with SCRFD-10G and compute 512-dimensional ArcFace embeddings locally with ONNX Runtime.
FACE-02	Authorised users shall enroll a user's face from a base64 image (largest face is used).
FACE-03	Authorised users shall recognise all faces in an uploaded image or a live camera frame and receive an annotated image with bounding boxes and matches.
FACE-04	A face shall be considered a match only when cosine similarity is at or above `FACE_MATCH_THRESHOLD`.
FACE-05	Matches outside the caller's `users` read scope shall be reported as not matched, so identities never leak.
FACE-06	Self-enrollment shall be capped (`FACE_SELF_ENROLL_LIMIT`, default 1). A privileged user may unlock or re-enroll, which resets the counter.
FACE-07	Authorised users shall bulk-enroll faces from an uploaded file using a template.
FACE-08	Every recognition, enrollment, replacement, removal, unlock and bulk import shall be written to the face audit log.
FACE-09	Images larger than `FACE_MAX_IMAGE_BYTES` (default 8 MB) shall be rejected.

3.7 Notifications (NOT)

ID	Requirement
NOT-01	Users shall have a paged notification inbox with unread count, mark-read, mark-all-read and delete.
NOT-02	The system shall stream new notifications in real time over SSE (`/notifications/stream`).
NOT-03	Users shall set preferences: push on/off, real-time on/off, muted notification types.
NOT-04	Native apps shall register and remove push devices (`android` , `ios` , `web`).
NOT-05	Authorised users shall send notifications to users, roles and departments (up to 1000 users, 100 roles, 100 departments).
NOT-06	Administrators shall configure FCM with an encrypted service-account secret, test it, and view delivery logs and statistics.

3.8 Custom fields (CF)

ID	Requirement
CF-01	Administrators shall define fields for `users`, `departments`, `roles` or `complaints` with a key (lower snake_case, ≤50), label, help text and type (`text`, `number`, `bool`, `date`, `select`, `multi_select`).
CF-02	Fields shall support required, options, default, min/max, max length, regex pattern, sort order and active flag.
CF-03	Fields may apply only to a role or a department; `/custom-fields/applicable` shall return the fields that apply.
CF-04	Values shall be validated on every write of the owning entity.
CF-05	List endpoints shall filter on custom fields with the `cf.<key>` syntax.

3.9 System monitor (MON)

ID	Requirement
MON-01	Users with `system_monitor:read:all` shall view a live overview of host CPU, memory, disks, network, per-service health and resource use, database, Redis and storage statistics.
MON-02	The overview shall be streamed over SSE and include short-term history and detected issues.

3.10 Native applications (APP)

ID	Requirement
APP-01	Desktop and mobile apps shall offer the same features as the web app against any ProctorAI server.
APP-02	The server address shall be configurable at build time (`VITE_SERVER_URL`) or entered at sign-in.
APP-03	Apps shall show system notifications, support `proctorai://` deep links, use the device camera for face recognition and use native save dialogs.

4. External interface requirements

4.1 User interfaces

- Responsive web UI with light and dark themes, a command palette, and layouts that respect safe areas on phones.
- Primary screens: Sign in, Dashboard, Complaints (list, new, detail), Users, Departments, Roles & access, Custom fields, Faces, Notifications, Notification admin, System monitor, Profile.
- Navigation and actions shall be shown only when the user's permissions allow them.

4.2 Software interfaces

Interface	Protocol	Notes
REST API	HTTPS/JSON via the gateway, under `/api` behind the web domain	See API Handbook PAI-API-003
Live streams	SSE (`text/event-stream`)	Notifications, user events, system monitor, job progress
Object storage	S3 API via presigned URLs on the files domain	Upload with `PUT` , download with `GET`
Database	PostgreSQL wire protocol (R2DBC/JDBC), internal only	Not exposed outside the Docker network
Push	Firebase Cloud Messaging HTTP v1	Optional
Metrics	Prometheus text format on `/actuator/prometheus` , internal only	

4.3 Communication interfaces

All public traffic uses HTTPS (HTTP/1.1, HTTP/2 and HTTP/3) once DNS is configured, with HSTS. Plain HTTP is used only during the IP-address bootstrap stage before DNS is ready.

5. Non-functional requirements

5.1 Performance

ID	Requirement
PERF-01	Services shall be non-blocking (WebFlux and R2DBC) to sustain concurrent requests with small thread pools.
PERF-02	List endpoints shall be paginated with a maximum page size of 100.
PERF-03	Interactive API calls should complete in under 300 ms at the 95th percentile on recommended hardware, excluding face recognition.
PERF-04	Face recognition of a single 640-pixel image should complete in under 2 seconds on a 4-core server.
PERF-05	Uploads up to 250 MB shall be supported through presigned URLs; web requests are capped at 300 MB.

5.2 Security

ID	Requirement
SEC-01	Every data access shall pass PostgreSQL RLS policies bound to the caller's identity (`auth.uid`).
SEC-02	Services shall connect with least-privilege database roles (`authenticator` , `proctor`), never the owner.

ID	Requirement
SEC-03	Passwords shall be stored as bcrypt hashes; refresh tokens shall be stored only as hashes.
SEC-04	JWT secrets shall be at least 32 bytes; the registry password shall be at least 32 characters.
SEC-05	Production containers shall run read-only, drop all Linux capabilities, use `no-new-privileges` and have memory limits.
SEC-06	CORS shall allow only managed origins; storage and registry origins shall never be allowed.
SEC-07	The registry dashboard and MinIO console shall never be exposed over plain HTTP.
SEC-08	FCM credentials shall be encrypted at rest with `NOTIFICATION_SECRET_KEY`.

5.3 Reliability and availability

ID	Requirement
REL-01	Every service shall expose a health endpoint, and Compose shall start services in dependency order using health checks.
REL-02	Containers shall restart automatically unless stopped (`restart: unless-stopped`).
REL-03	Database migrations shall complete before any service starts; a failed migration shall stop startup.
REL-04	Data shall survive container recreation through named volumes.
REL-05	The target availability for a single-server deployment is 99.5 % per month, excluding planned maintenance.

5.4 Maintainability and portability

- Shared concerns live in `common-*` libraries; each service owns one bounded context.
- A single multi-stage Dockerfile builds every backend image.
- The stack runs on any Linux x86_64 host with Docker; developers can run it on macOS and Windows with Docker Desktop.

5.5 Usability and accessibility

- The UI should meet WCAG 2.1 AA contrast in both themes.
- All destructive actions shall require confirmation.
- Errors shall be shown with the human-readable `message` returned by the API.

5.6 Observability

- Logs shall be JSON in production with rotation (10 MB × 5 files per container).

- Logs shall carry a trace ID that follows a request across services.
- Metrics shall be available in Prometheus format inside the Docker network.

6. Constraints, assumptions and risks

Type	Item	Mitigation
Licence	InsightFace buffalo_l models are for non-commercial research use	Review licensing or replace models before commercial use
Single host	The reference deployment runs on one server	Scale vertically; see Architecture Document for scale-out options
Certificates	Let's Encrypt rate limits	Avoid repeated purges of production
Secrets	Losing the env file prevents reusing existing data	Back up the env file securely (see PAI-BDR-008)
Biometrics	Face data is sensitive personal data	Obtain consent, restrict permissions, audit every access

7. Verification and traceability

Requirement group	Verified by
AUTH-01…10	`AuthFlowIntegrationTest`, `LoginRateLimitIntegrationTest`, `JwtTokenServiceTest`, Postman collection
SEC-01, SEC-02, ROL-*	`RowLevelSecurityTest`, `MigrationTest`
REL-03	`MigrationTest`, `DbMigratorApplicationTest`
Gateway client address	`ClientAddressFilterTest`, `ClientAddressResolverTest`
USR, DEP, CMP, FACE, NOT, CF, MON	Postman collection and manual acceptance test (User & Admin Manual, PAI-USR-013)

7.1 Acceptance criteria

1. A fresh server reaches a healthy state with `docker/setup.sh` and serves HTTPS on the configured domains.
2. The administrator can sign in with the one-time password and change it.
3. A student can file a complaint, a teacher of the same department can process it, and every status change appears in history and as a notification.
4. A user enrolled with face recognition is matched in a new photo, and the event appears in the face audit log.
5. `./gradlew build` passes in full.

DOCUMENT 02

ARCHITECTURE · PAI-ARC-002

Architecture Document

System context, containers, components, data flows and key design decisions.

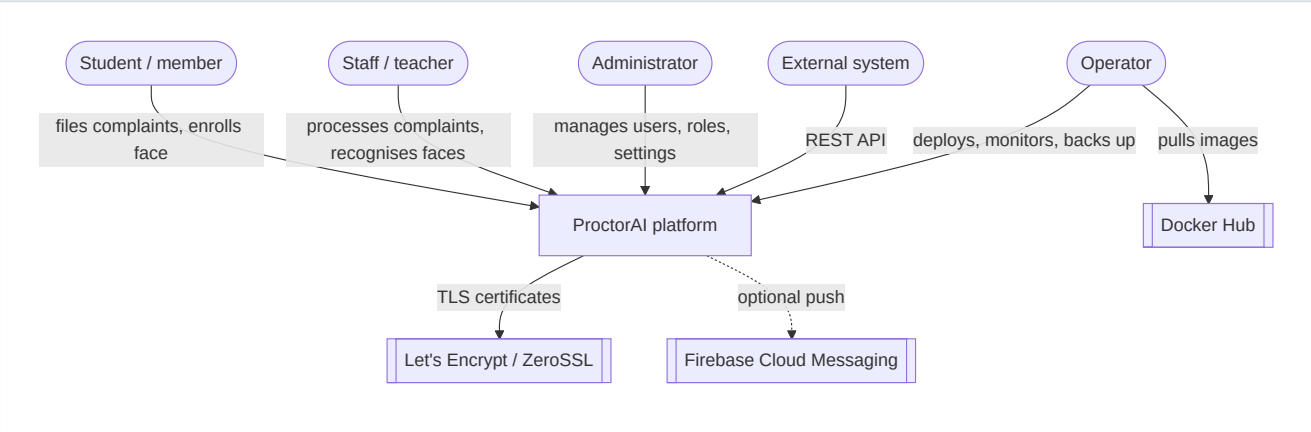
1. Overview
2. System context
3. Container view
4. Component view
5. Key flows
6. Deployment view
7. Cross-cutting concerns
8. Scalability
9. Architecture decision records

1. Overview

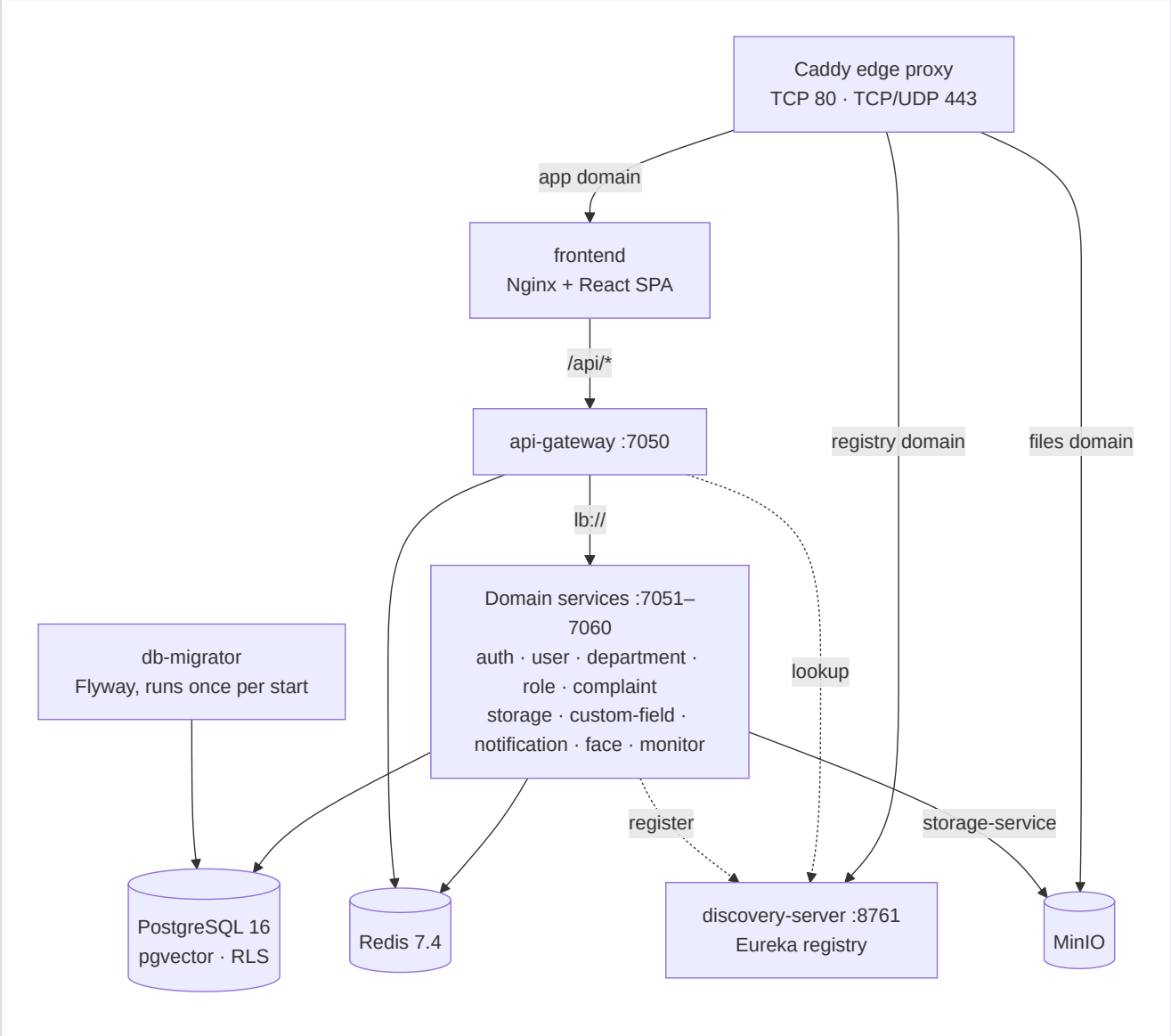
ProctorAI is a reactive microservice system. A Spring Cloud Gateway routes every API call to one of ten domain services that are registered in a Eureka service registry. All services share one PostgreSQL 16 database (with pgvector), where authorization is enforced by Row-Level Security (RLS). Redis carries rate-limit buckets, notification streams, real-time fan-out, job progress and monitoring samples. MinIO stores uploaded files. A Caddy edge proxy terminates TLS in production.

This document follows the C4 model: system context, then containers, then components. It ends with cross-cutting concerns and recorded architecture decisions.

2. System context



3. Container view



3.1 Containers and responsibilities

Container	Technology	Responsibility	Gateway path
`edge`	Caddy 2.11	TLS (Let's Encrypt, ZeroSSL fallback), HTTP/3, HSTS, security headers, routing by domain	—
`frontend`	Nginx + React 19 SPA	Serves the web app, proxies `/api/` to the gateway, blocks `/api/actuator/`	—
`api-gateway`	Spring Cloud Gateway	Routing via `lb://`, CORS, login rate limit, client IP resolution	—
`discovery-server`	Netflix Eureka	Service registry protected by HTTP Basic auth	—
`auth-service`	WebFlux + R2DBC	Login, refresh rotation, logout, lockout	`/auth/**`

Container	Technology	Responsibility	Gateway path
`user-service`	WebFlux + R2DBC	Users, profile, bulk import, user event stream	`/users/**`
`department-service`	WebFlux + R2DBC	Departments and members	`/departments/**`
`role-service`	WebFlux + R2DBC	Roles, permission catalogue and matrix	`/roles/**`
`complaint-service`	WebFlux + R2DBC	Complaints, comments, assignment, status, history, dashboard stats	`/complaints/**`
`storage-service`	WebFlux + AWS SDK	Presigned uploads/downloads, file confirmation, stale upload purge	`/complaints/*/files/**`
`custom-field-service`	WebFlux + R2DBC	Custom field definitions	`/custom-fields/**`
`notification-service`	WebFlux + Redis	Inbox, SSE stream, preferences, devices, FCM, deliveries	`/notifications/**`
`face-service`	WebFlux + ONNX Runtime + Spring AI	Detection, embedding, enrollment, recognition, audit	`/faces/**`
`monitor-service`	WebFlux + OSHI	Host, service, DB, Redis and storage metrics	`/system/**`
`db-migrator`	Spring Boot + Flyway	Applies migrations, sets role passwords, then exits	—
`postgres`	PostgreSQL 16 + pgvector	System of record, RLS, vector index	—
`redis`	Redis 7.4 (AOF on)	Rate limits, streams, pub/sub, progress, monitoring samples	—
`minio` / `minio-init`	MinIO	Object storage; init creates bucket, policy and app user	—

4. Component view

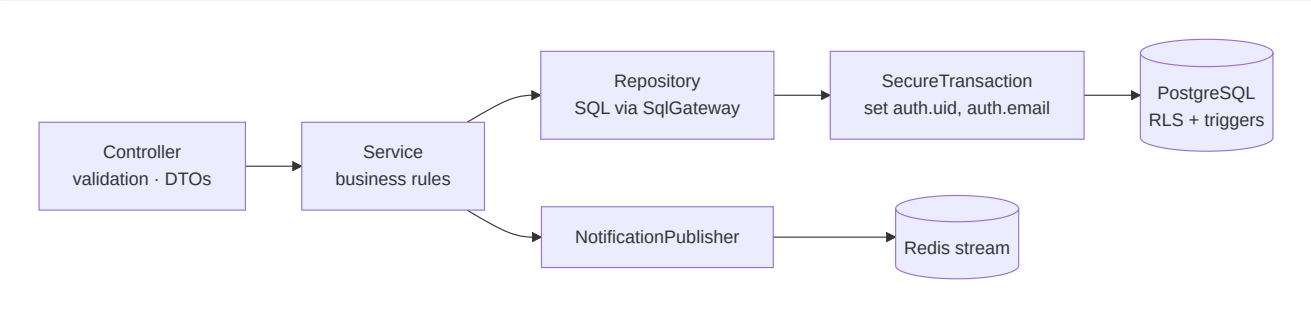
4.1 Shared libraries

Every domain service is assembled from the same building blocks:

Library	Key components
`common-lib`	DTOs (`UserDTO` , `PageResponse` , `ApiError` ...), enums (`EntityType` , `PermissionScope` ...), typed exceptions mapped to HTTP status
`common-security`	`JwtAuthenticationWebFilter` , `JwtTokenService` (HMAC, secret ≥ 32 bytes), `TokenExtractor` (cookie `token` or `Authorization: Bearer`), `CurrentUser` , `BcryptPasswordHasher` , `GlobalExceptionHandler`

Library	Key components
`common-persistence`	`SqlGateway` over R2DBC, `RlsTransaction` / `SecureTransaction`, `SearchQueryParser` (filter DSL), `PagedQueryRunner`, `PersistenceExceptionHandler` (SQLSTATE → HTTP), `DatabaseRoleGuard`
`common-storage`	`FileStorage` for temporary job files and `TemporaryFileSweeper`
`common-customfields`	Definition reader, validator and value store used by services that own extendable entities
`common-notifications`	`NotificationPublisher` (Redis stream), `ProgressBroker` (job progress)
`common-monitoring`	`InstanceSampler` / `InstanceReporter` publishing JVM and request metrics to Redis every 2 s

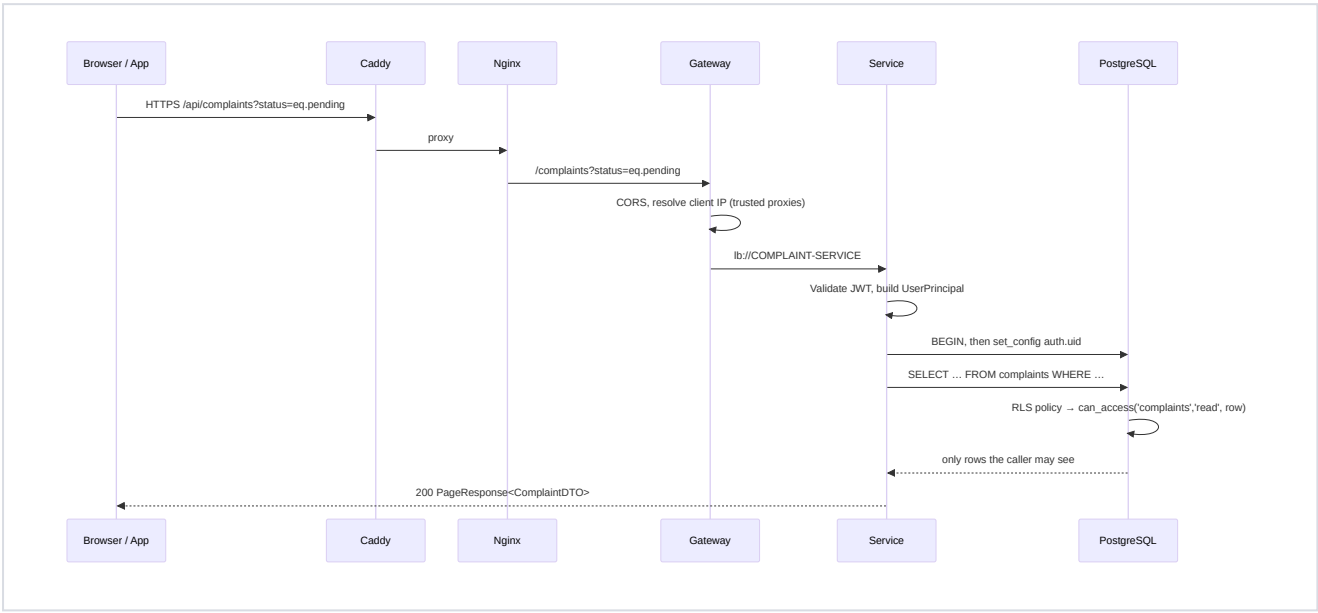
4.2 Layering inside a service



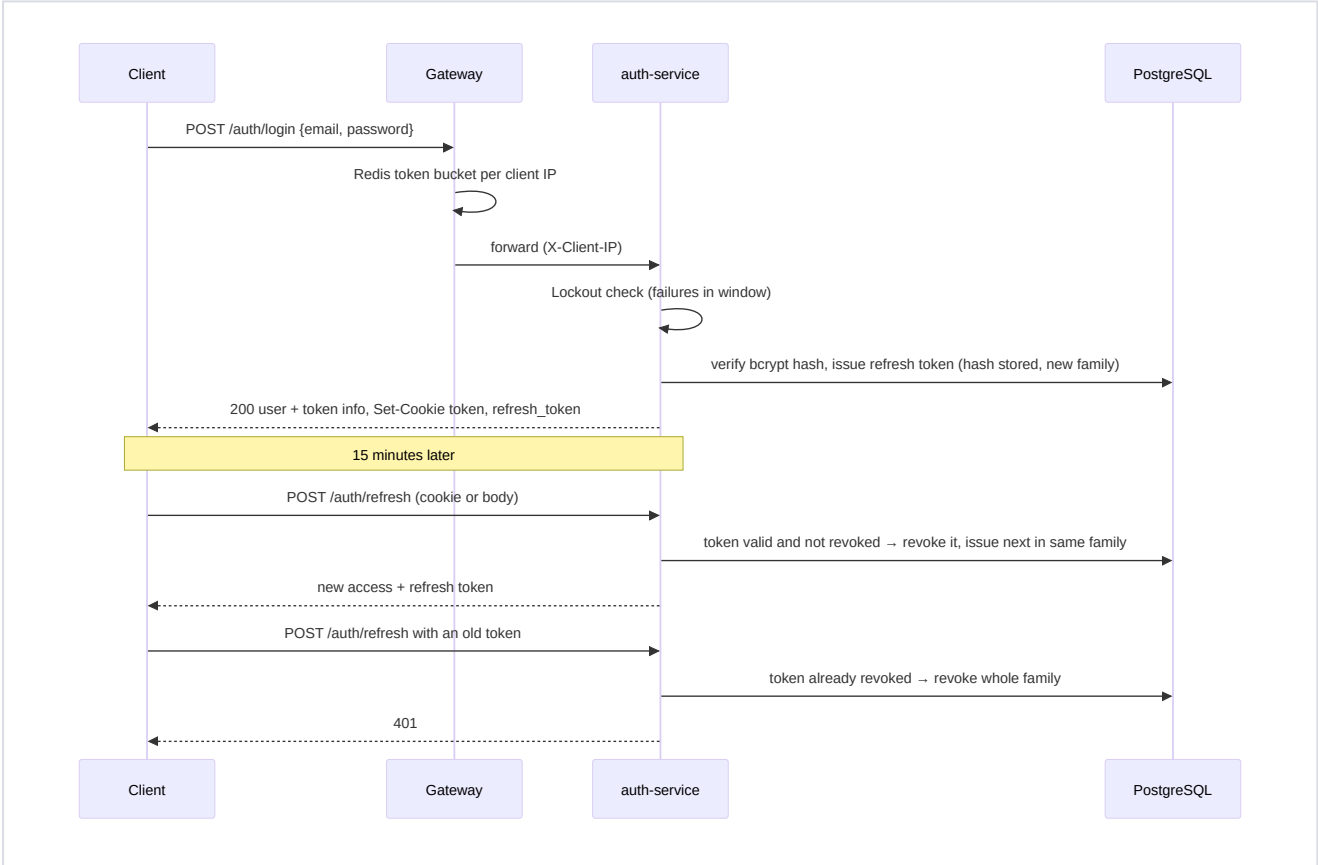
Controllers are thin. Services hold the workflow. Repositories issue explicit SQL through `SqlGateway`. Every request-scoped query runs inside a transaction that first sets `auth.uid` and `auth.email`, so RLS policies and triggers see the caller.

5. Key flows

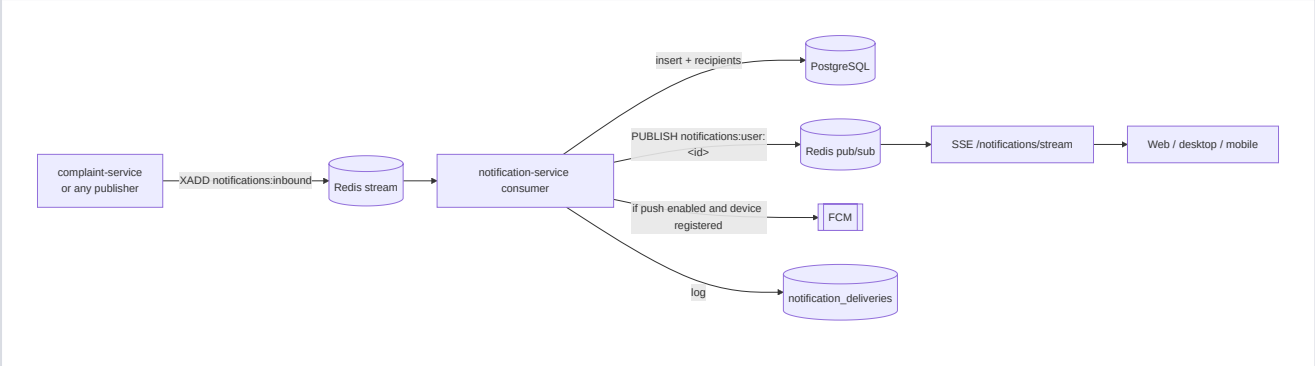
5.1 Request and authorization flow



5.2 Sign-in and token rotation

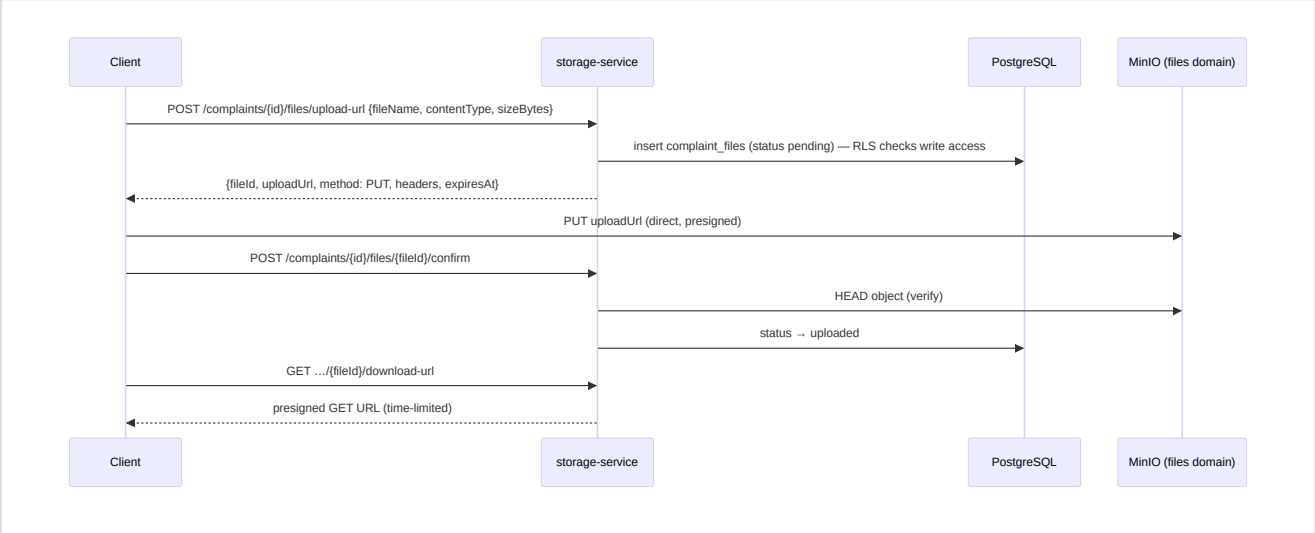


5.3 Notification flow



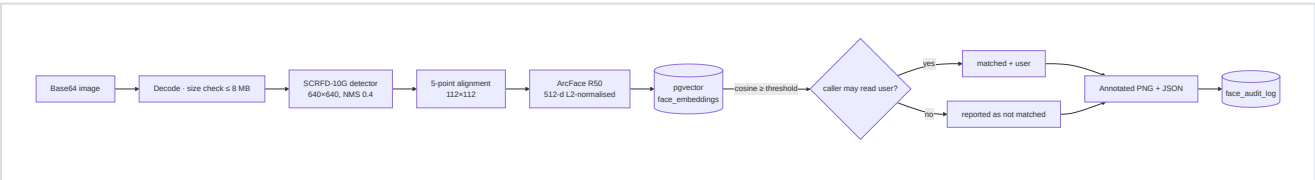
Recipients are resolved in the database (`resolve_notification_recipients`) from user, role and department targets. Preferences (`push_enabled` , `realtime_enabled` , `muted_types`) are applied per user before delivery. Every channel attempt is logged as `sent` , `failed` or `skipped` .

5.4 File upload flow



Unconfirmed uploads are purged by `purge_stale_uploads` . File bytes never pass through the API services.

5.5 Face recognition flow

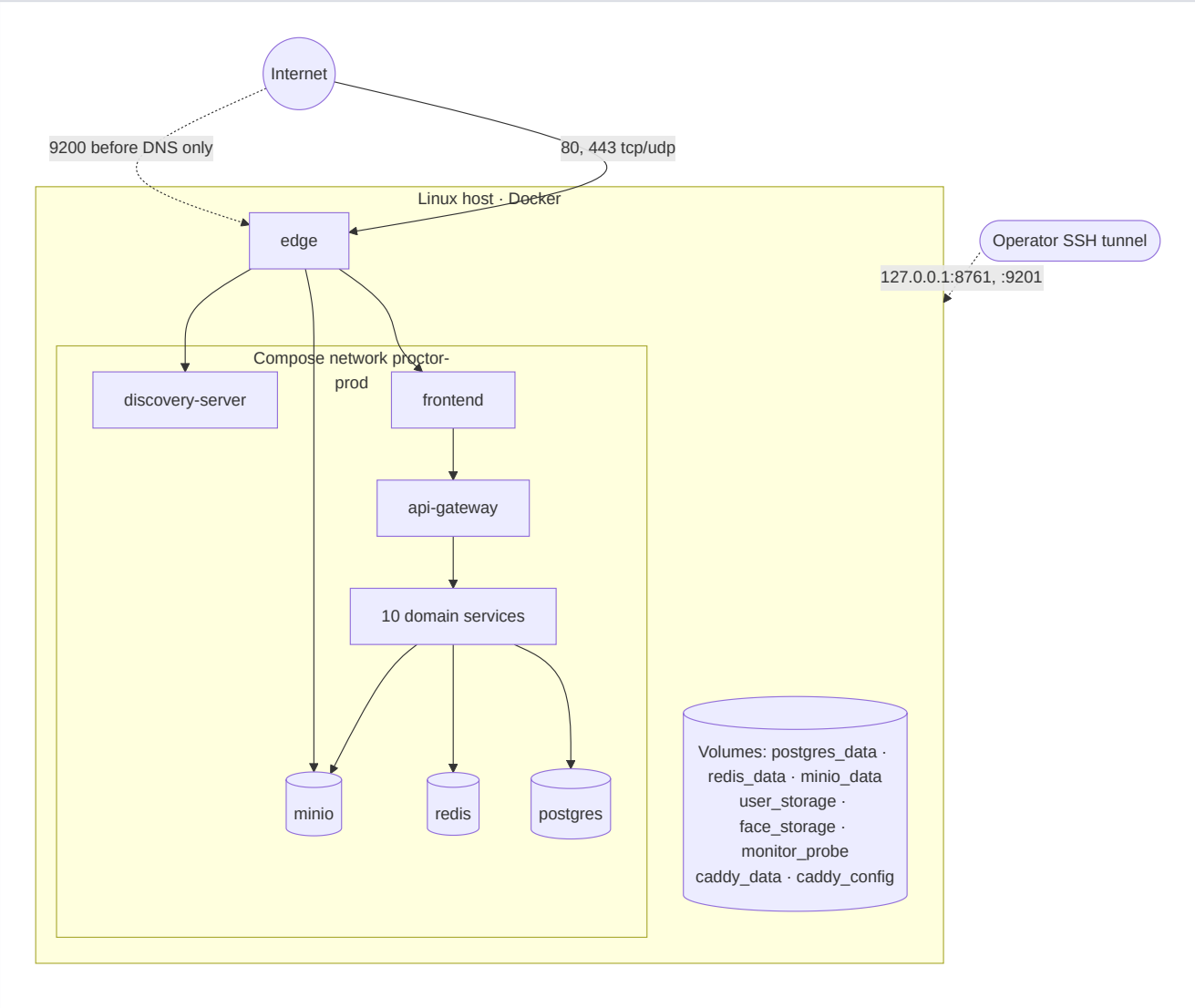


5.6 System monitor flow

Each service runs `InstanceReporter` , which writes a JVM and request sample to Redis key `monitor:instance:<id>` every 2 seconds with a 10-second TTL. `monitor-service` combines those samples with the Eureka registry, host metrics (CPU, memory, disks, network), PostgreSQL statistics functions, Redis

`INFO` and storage usage into a `SystemSnapshot`, keeps a short history, derives issues, and streams it over SSE at `/system/stream`.

6. Deployment view



Domain	Served by	Purpose
`app.<domain>`	Caddy → frontend → gateway	Web app and API (`/api`)
`files.<domain>`	Caddy → MinIO	Presigned uploads and downloads
`registry.<domain>`	Caddy → discovery-server (dashboard only)	Eureka dashboard, behind Basic auth

6.1 Resource budget (production defaults)

Container	Memory limit
Each domain service and gateway	768 MB

Container	Memory limit
`face-service`	2560 MB
`discovery-server`, `monitor-service`	512 MB
`db-migrator`	384 MB
`edge`	256 MB
`frontend`	128 MB

The JVM uses G1 with `MaxRAMPercentage=60` and exits on out-of-memory so Docker restarts it cleanly.

7. Cross-cutting concerns

Concern	Approach
Authentication	Stateless JWT in an `HttpOnly` cookie or bearer header, validated in every service
Authorization	RLS policies and `can_access` / `has_scope` functions in PostgreSQL; triggers protect invariants
Validation	Bean Validation on DTOs; database CHECK constraints and triggers as the last line
Errors	Uniform `ApiError {message, statusCode, status, timestamp}`; SQLSTATE mapped to HTTP
Configuration	Environment variables from the env file; production compose fails fast on missing values
Observability	Actuator health and Prometheus metrics; JSON logs with trace IDs; live system monitor
Resilience	Health-checked startup order, `restart: unless-stopped`, load-balancer cache TTL 5 s, Eureka lease 10 s
Real time	SSE streams fed by Redis pub/sub
Extensibility	Custom fields stored as typed JSON values, filterable via `cf.<key>`
Images	One multi-stage Dockerfile, jlink-trimmed JRE, CDS archive, non-root user, distroless face image

8. Scalability

The reference deployment runs on one host. The architecture supports growth in stages:

1. **Vertical:** increase host RAM/CPU and raise memory limits; face-service benefits most from CPU.
2. **Horizontal services:** run more replicas of a stateless service. The gateway load-balances through Eureka (`lb://`). SSE fan-out works across replicas through Redis pub/sub.
3. **Data tier:** move PostgreSQL to a managed or replicated instance with pgvector; move Redis to a managed instance; replace MinIO with any S3-compatible service by changing `S3_*` variables.
4. **Edge:** put a load balancer in front of several hosts and keep sticky-free routing (tokens are stateless).

9. Architecture decision records

ADR	Decision	Rationale	Consequence
ADR-01	Reactive stack (WebFlux, R2DBC)	High concurrency with few threads, natural fit for SSE	Requires reactive-aware code and libraries
ADR-02	Authorization in the database via RLS	A bug in service code cannot leak rows; one source of truth	Logic lives in SQL; needs dedicated RLS tests
ADR-03	Single shared database	Simple operations, transactional integrity across domains	Services are coupled at the schema level; migrations are central
ADR-04	Flyway migrations in a one-shot migrator	Safe automatic upgrades before services start	Released migrations are immutable
ADR-05	On-premises face recognition (ONNX)	Privacy, no external AI cost, offline operation	Larger image and memory; model licence review
ADR-06	Presigned S3 URLs	File bytes bypass services; scalable	Files domain must be reachable by clients; CORS on MinIO
ADR-07	Redis for streams, pub/sub and rate limits	One lightweight dependency for several needs	Redis must be persistent (AOF on) and backed up
ADR-08	Caddy edge with automatic TLS	Zero-touch certificates, HTTP/3	Ports 80/443 must be reachable for ACME
ADR-09	Tauri for native apps	One codebase, small native binaries	Platform toolchains needed per OS

DOCUMENT 03

API REFERENCE · PAI-API-003

API Handbook

The complete REST and streaming API reference, with authentication, conventions, every endpoint and its payloads.

1. Introduction
2. Base URLs
3. Authentication
4. Conventions
5. Authorization model
6. Users API
7. Departments API
8. Roles and permissions API
9. Complaints API
10. Comments API
11. Files API
12. Custom fields API
13. Notifications API
14. Face recognition API
15. System monitor API
16. Health and metrics
17. Examples
18. Versioning and compatibility

1. Introduction

This handbook documents every public endpoint of the ProctorAI API in version 1.1.0. All endpoints are served by the API gateway and routed to the owning service. The API is JSON over HTTPS. Long-running and live data use Server-Sent Events (SSE).

A ready-to-run Postman collection with development and production environments is available in `backend/postman/`.

2. Base URLs

Environment	Base URL	Notes
Production (web origin)	`https://app.example.com/api`	Nginx strips `/api` and forwards to the gateway
Production before DNS	`http://<server-ip>/api`	Plain HTTP; bootstrap only
Development (gateway)	`http://localhost:7050`	Direct access to the gateway
Development (web origin)	`http://localhost:5173/api`	Vite proxy to the gateway

In this handbook, paths are written relative to the base URL, for example `POST /auth/login`.

3. Authentication

3.1 Token model

Token	Format	Default lifetime	Where it lives
Access token	JWT signed with HMAC (`JWT_SECRET`), claims: `sub` (email), user id, `iss`, `iat`, `exp`	15 minutes (`JWT_ACCESS_EXPIRE_IN_SEC`)	Cookie `token` (path `/`) or `Authorization: Bearer` header
Refresh token	Opaque random value, stored server-side only as a hash	30 days (`JWT_REFRESH_EXPIRE_IN_SEC`)	Cookie `refresh_token` (path `/auth`) or the request body

Both cookies are `HttpOnly`, `SameSite=Lax`, and `Secure` when `APP_COOKIE_SECURE=true`.

Services accept the access token from the `token` cookie first, then from the `Authorization: Bearer <jwt>` header.

3.2 Browser clients

Browsers rely on cookies. Send requests with credentials (`fetch(url, { credentials: "include" })`). The origin must be listed in `APP_ALLOWED_ORIGINS`.

3.3 Native apps and server-to-server clients

Send `X-Token-Delivery: body` on `/auth/login` and `/auth/refresh`. The response then also contains `accessToken` and `refreshToken` in `tokenInfo`. Store the refresh token securely and send the access token as a bearer header.

3.4 Sign in

`POST /auth/login` (public, rate-limited)

JSON

```
{ "email": "admin@college.com", "password": "••••••••" }
```

Response `200 OK`:

JSON

```
{
  "user": {
    "id": "6f1c…", "email": "admin@college.com", "name": "System Admin",
    "enabled": true, "verified": true, "customFields": {},
    "createdAt": "2026-11-01T09:00:00Z", "updatedAt": null
  },
  "tokenInfo": {
    "expiresIn": 900,
    "issuedAt": "2026-11-05T10:00:00Z",
    "expiresAt": "2026-11-05T10:15:00Z",
    "refreshExpiresIn": 2592000,
    "refreshExpiresAt": "2026-12-05T10:00:00Z",
    "accessToken": null,
    "refreshToken": null
  }
}
```

`accessToken` and `refreshToken` are only filled when `X-Token-Delivery: body` is sent. Two `Set-Cookie` headers are always returned.

Status	Meaning
400	Validation failed (`email` must be a valid address ≤ 320 chars; `password` ≤ 200 chars)
401	Wrong credentials or disabled account
429	Gateway rate limit hit, or account temporarily locked after repeated failures

3.5 Refresh

`POST /auth/refresh` (public)

Body is optional: `{ "refreshToken": "…" }`. When omitted, the `refresh_token` cookie is used. The response has the same shape as sign-in and rotates the refresh token.

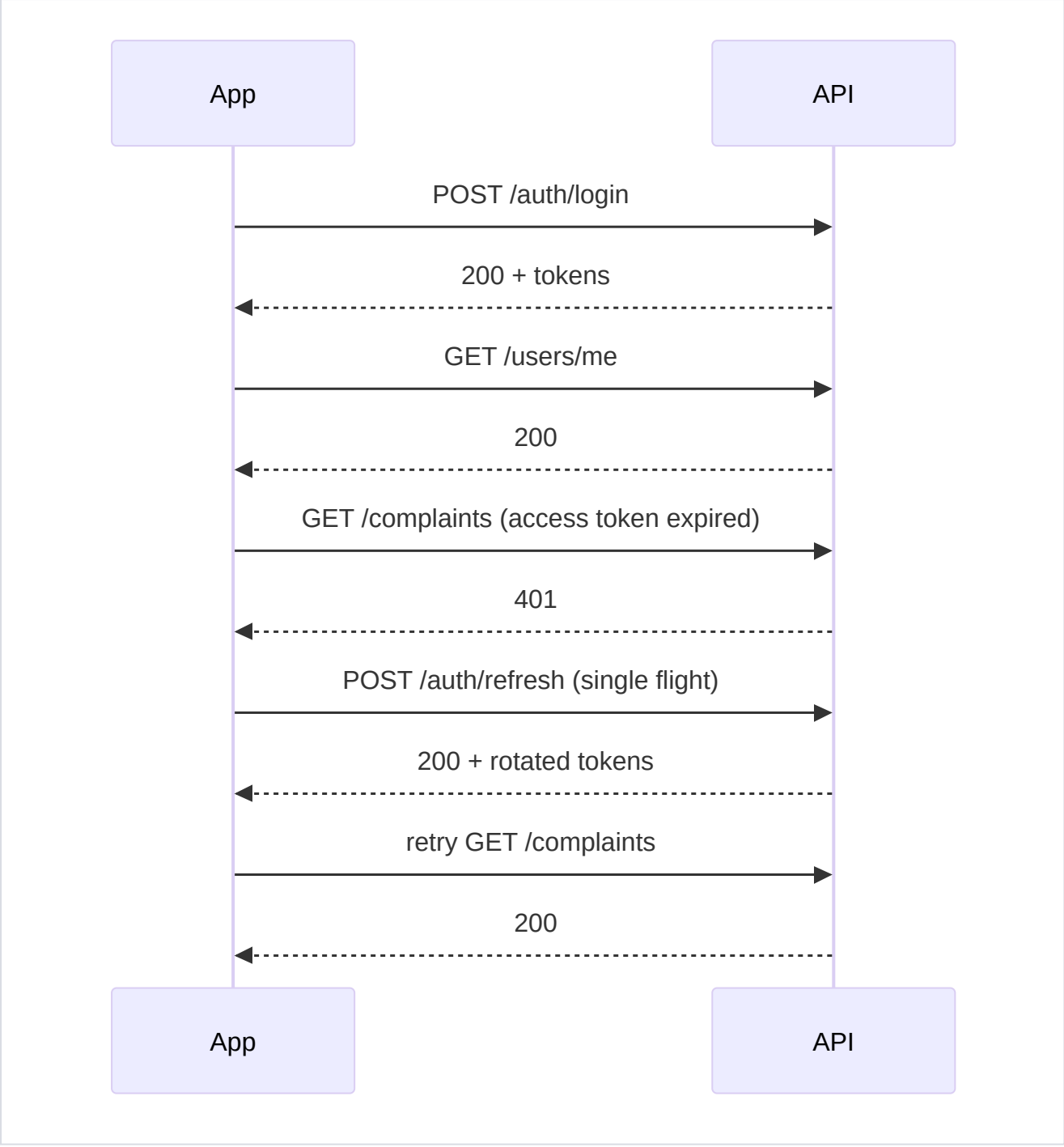
> **IMPORTANT**
> Each refresh token can be used exactly once. Presenting a token that was already rotated is treated as theft: the whole token family is revoked and the client must sign in again. Clients must serialise refresh calls, for example with a single in-flight refresh promise.

3.6 Sign out

Endpoint	Auth	Effect
`POST /auth/logout`	Public	Revokes the given refresh token (body or cookie) and clears both cookies
`POST /auth/logout-all`	Required	Revokes every refresh token of the caller on every device

Both return `200 {"message": "…"}`.

3.7 Recommended client flow



4. Conventions

4.1 Content types and identifiers

- Requests and responses use `application/json; charset=utf-8`, except file imports (`multipart/form-data`) and streams (`text/event-stream`).
- All identifiers are UUIDs. Timestamps are ISO-8601 with offset (`OffsetDateTime`).
- Enumerations are lower case, for example `pending`, `multi_select`.

4.2 HTTP methods

Method	Use
`GET`	Read; never changes state
`POST`	Create, or trigger an action
`PUT`	Full replacement of a resource, or set a sub-resource (status, assignee, members, permissions)
`PATCH`	Partial update; omitted fields are unchanged
`DELETE`	Remove; returns `204 No Content`

4.3 Pagination and sorting

List endpoints that return `PageResponse` accept:

Parameter	Default	Notes
`page`	`0`	Zero-based
`size`	`10` (20 for notifications and face audit, 50 for custom fields and the permission catalogue)	Clamped to 1–100
`sortBy`	`createdAt` (varies)	Any sortable field of the resource
`direction`	`desc`	`asc` or `desc`

JSON

```
{
  "content": [ … ],
  "totalElements": 248,
  "totalPages": 25,
  "number": 0,
  "size": 10,
  "first": true,
  "last": false
}
```

4.4 Filtering

Every other query parameter is a filter on a resource field. The syntax is `field=operator.value`. Without an operator, `eq` is used.

Operator	Meaning	Example
`eq`, `neq`	Equal / not equal	`status=eq.pending`
`gt`, `gte`, `lt`, `lte`	Comparisons	`createdAt=gte.2026-10-01T00:00:00Z`

Operator	Meaning	Example
`like`, `ilike`	Pattern match (`*` or `%` wildcards), `ilike` is case-insensitive	`title=ilike.*hostel*`
`in`	Membership, up to 100 values	`priority=in.(high,urgent)`
`between`	Inclusive range	`createdAt=between.2026-10-01,2026-10-31`
`is`	Null checks	`assignedTo=is.null`
`or`	Disjunction of conditions	`or=(title.ilike.*fee*,description.ilike.*fee*)`

Custom fields are filtered with the `cf.` prefix: `cf.hostel_block=eq.B`, `cf.year=gte.2`. Numeric comparisons on custom fields require a number. Unknown fields return `400 Unknown filter field`.

> **NOTE**
> Results are always limited by row-level security. A filter can narrow what you see, never widen it.

4.5 Errors

Every error uses one body shape:

```json
{
  "message": "'title' must not be blank",
  "statusCode": 400,
  "status": "BAD_REQUEST",
  "timestamp": "2026-11-05T10:02:11.481Z"
}
```

Status	When
400 Bad Request	Validation failed, malformed JSON, invalid filter, database check or business rule (`P0001`) failed
401 Unauthorized	Missing, invalid or expired access token; missing refresh token
403 Forbidden	RLS or a security rule denied the action
404 Not Found	Resource does not exist or is not visible to the caller
409 Conflict	Unique value already exists (for example email or department code)
429 Too Many Requests	Login rate limit or account lockout
500 Internal Server Error	Unexpected failure; details are only in server logs

Business rules enforced in the database return readable messages, for example:

Rule code (stripped from message)	Example message
`COMPLAINT_LOCKED`	complaint is already being processed
`INSUFFICIENT_LEVEL`	you cannot change workflow fields
`PERMISSION_ESCALATION`	you cannot grant a permission you do not hold
`SELF_ROLE_CHANGE`	you cannot change your own roles
`SELF_PROTECTED`	you cannot enable or disable your own account
`LAST_SYSTEM_ADMIN`	the last system admin cannot be removed
`SYSTEM_ROLE_PROTECTED`	system roles cannot be deleted

4.6 Streaming (SSE)

Streaming endpoints return `text/event-stream`. Each stream sends `event: ready` on connect, periodic `event: heartbeat` messages, then domain events. Use `EventSource` with credentials in browsers, or any SSE client with a bearer header in native apps. Reconnect with backoff when the stream closes; a stream also closes when the caller's session is revoked.

4.7 Rate limits

Scope	Default	Variables
`POST /auth/login` per client IP	Bucket of 30 tokens, refilled 1 token/s, 3 tokens per attempt: 10 immediate attempts, then 20 per minute	`LOGIN_RATE_LIMIT_BURST`, `_REPLENISH`, `_COST`
Failed sign-ins per account	10 failures lock the account for 15 minutes	`LOGIN_MAX_FAILURES`, `LOGIN_LOCKOUT`

4.8 CORS

Allowed methods: `GET, POST, PUT, PATCH, DELETE, OPTIONS`. Allowed headers: `Content-Type, Authorization, Accept`. Credentials are allowed. Pre-flight results are cached for one hour.

5. Authorization model

Each endpoint is protected by permissions of the form **entity × action × scope**. The database evaluates them per row.

Action	Typical HTTP use
`read`	`GET`
`write`	`POST` (create)
`update`	`PUT`, `PATCH`, workflow changes

Action	Typical HTTP use
`delete`	`DELETE`

Scope	Rows covered
`own`	Rows that belong to the caller (own profile, own complaints, own notifications)
`department`	Rows linked to any department the caller is a member of
`all`	Every row

Superuser roles (`system_admin` , `admin`) implicitly hold every `all` permission. The protected entities are:

`dashboard` , `users` , `user_roles` , `roles` , `role_permission` , `permissions` , `departments` , `department_users` , `complaints` , `complaint_comments` , `complaint_files` , `complaint_history` , `complaint_reporter` , `custom_fields` , `notifications` , `notification_devices` , `integrations` , `face_recognition` , `face_live_recognition` , `face_match_details` , `face_enrollments` , `face_enroll_lock` , `face_import` , `face_audit` , `same_role_peers` , `parallel_role_peers` , `senior_users` , `system_monitor` .

The caller's effective permissions are available at `GET /users/me/permissions` . Clients should use it to show or hide actions.

The **Permission** column in the endpoint tables names the primary entity and action checked. Row-level security may also consult related entities, for example `complaint_reporter` to reveal the reporter or `senior_users` to view more senior users.

6. Users API

Service: `user-service` · Prefix: `/users`

Method	Path	Description	Permission
GET	`/users`	List users (paged, filterable)	`users:read`
GET	`/users/me`	Current user profile	Authenticated
GET	`/users/me/events`	SSE stream of events for the current user (job progress, session changes)	Authenticated
GET	`/users/me/roles`	Current user's roles with permissions	Authenticated
GET	`/users/me/permissions`	Current user's effective permissions	Authenticated
GET	`/users/me/departments`	Current user's departments	Authenticated
POST	`/users`	Create a user → `201`	`users:write`
POST	`/users/bulk`	Create many users from a JSON array → `201`	`users:write`

Method	Path	Description	Permission
GET	`/users/import/template?format=csv|xlsx`	Download the import template	`users:write`
POST	`/users/import`	Import users from a CSV/Excel file (`multipart` , part `file`) → `202`	`users:write`
DELETE	`/users/bulk`	Delete many users (body: array of ids) → `204`	`users:delete`
GET	`/users/{id}`	Get one user	`users:read`
PUT	`/users/{id}`	Replace a user	`users:update`
PATCH	`/users/{id}`	Partially update a user	`users:update`
DELETE	`/users/{id}`	Delete a user → `204`	`users:delete`
GET	`/users/{id}/roles`	Roles of a user	`user_roles:read`
GET	`/users/{id}/permissions`	Effective permissions of a user	`role_permission:read`
GET	`/users/{id}/departments`	Departments of a user	`department_users:read`

Filterable fields: `id` , `email` , `name` , `enabled` , `verified` , `createdAt` , `updatedAt` , `departmentId` , `roleId` , `cf.<key>` .

6.1 Create user request

JSON

```
{
  "name": "Aarav Mehta",
  "email": "aarav@college.com",
  "password": "Str0ngPass!",
  "roles": ["<role-uuid>"],
  "departments": ["<department-uuid>"],
  "customFields": { "roll_no": "CS-2026-041" },
  "enabled": true,
  "verified": true
}
```

Field	Rules
`name`	Required, ≤ 200
`email`	Required, valid, ≤ 320, unique
`password`	Required, 8–200
`roles` , `departments`	Optional sets of UUIDs
`customFields`	Validated against applicable definitions

Field	Rules
`enabled`, `verified`	Default `true`

`PUT` requires `name` and `email`; `password` is optional. `PATCH` accepts any subset of `name`, `email`, `password`, `enabled`, `verified`, `roles`, `departments`, `customFields`.

6.2 User response (`UserInfoDTO`)

JSON

```
{
  "id": "…", "email": "aarav@college.com", "name": "Aarav Mehta",
  "enabled": true, "verified": true, "customFields": { "roll_no": "CS-2026-041" },
  "createdAt": "…", "updatedAt": "…",
  "roles": [ { "id": "…", "name": "student", "level": 50, "system": false, "superuser": false,
"revealIdentity": false, "customFields": {}, "createdAt": "…" } ],
  "departments": [ { "id": "…", "name": "Computer Science", "code": "CS", "active": true,
"customFields": {}, "createdAt": "…", "updatedAt": null } ]
}
```

6.3 Bulk import

1. Download the template: `GET /users/import/template?format=csv`.
2. Fill columns `name, email, password, roles, departments, enabled, verified` (roles and departments by name or code; see `backend/seed-data/users-bulk-import.csv`).
3. Upload: `POST /users/import` with `file`. The response starts a background job; progress is pushed on `/users/me/events`.

7. Departments API

Service: `department-service` · Prefix: `/departments`

Method	Path	Description	Permission
GET	`/departments`	List departments (paged, filterable)	`departments:read`
POST	`/departments`	Create → `200`	`departments:write`
POST	`/departments/bulk`	Create many → `200`	`departments:write`
GET	`/departments/{id}`	Get one	`departments:read`
PUT	`/departments/{id}`	Replace	`departments:update`
PATCH	`/departments/{id}`	Partial update	`departments:update`
DELETE	`/departments/{id}`	Delete → `204`	`departments:delete`
GET	`/departments/{id}/members`	List members (`id`, `email`, `name`)	`department_users:read`

Method	Path	Description	Permission
PUT	`/departments/{id}/members`	Add members `{ "userIds": [ … ] }`	`department_users:write`
DELETE	`/departments/{id}/members`	Remove members `{ "userIds": [ … ] }`	`department_users:delete`

```JSON
{ "name": "Computer Science", "code": "CS", "description": "B.Tech and M.Tech CS", "active": true,
"customFields": {} }
```

`name` ≤ 200 and `code` ≤ 50 are required on create and replace; `code` is unique. `description` ≤ 2000. `active` defaults to `true`.

8. Roles and permissions API

Service: `role-service` · Prefix: `/roles`

Method	Path	Description	Permission
GET	`/roles`	List roles (paged)	`roles:read`
POST	`/roles`	Create a role with permissions → `200`	`roles:write`
GET	`/roles/permissions`	Permission catalogue (paged, default size 50 sorted by `entity`, filterable by `entity`, `action`, `scope`)	`permissions:read`
GET	`/roles/{id}`	Role with its permissions	`roles:read`
PUT	`/roles/{id}`	Replace role attributes	`roles:update`
PATCH	`/roles/{id}`	Partial update	`roles:update`
DELETE	`/roles/{id}`	Delete → `204` (not for system roles)	`roles:delete`
GET	`/roles/{id}/permissions`	Permissions of a role	`role_permission:read`
PUT	`/roles/{id}/permissions`	Replace the permission set (body: array of permission ids)	`role_permission:update`
PATCH	`/roles/{id}/permissions`	Add permissions (body: array of ids)	`role_permission:write`
DELETE	`/roles/{id}/permissions`	Remove permissions (body: array of ids)	`role_permission:delete`
DELETE	`/roles/{id}/permissions/all`	Remove every permission → `204`	`role_permission:delete`

```json
{
  "name": "warden",
  "description": "Hostel wardens",
  "level": 30,
  "permissions": ["<permission-uuid>", "…"],
  "revealIdentity": false,
  "customFields": {}
}
```

Field	Rules
`name`	Required, ≤ 100, unique
`level`	1–1000; lower is more senior; 0 is reserved for system administrators
`revealIdentity`	When `true`, holders see reporters of complaints even if hidden
`permissions`	You can only grant permissions you hold yourself

9. Complaints API

Service: `complaint-service` · Prefix: `/complaints`

Method	Path	Description	Permission
GET	`/complaints`	List complaints (paged, filterable)	`complaints:read`
GET	`/complaints/stats`	Dashboard statistics	`dashboard:read`
POST	`/complaints`	File a complaint → `201`	`complaints:write`
GET	`/complaints/{id}`	Get one complaint	`complaints:read`
PATCH	`/complaints/{id}`	Partial update (content and, for staff, workflow fields)	`complaints:update`
PUT	`/complaints/{id}/status`	Change status	`complaints:update` (department/all)
PUT	`/complaints/{id}/assignee`	Assign	`complaints:update` (department/all)
DELETE	`/complaints/{id}`	Delete → `204`	`complaints:delete`
GET	`/complaints/{id}/history`	Status history	`complaint_history:read`

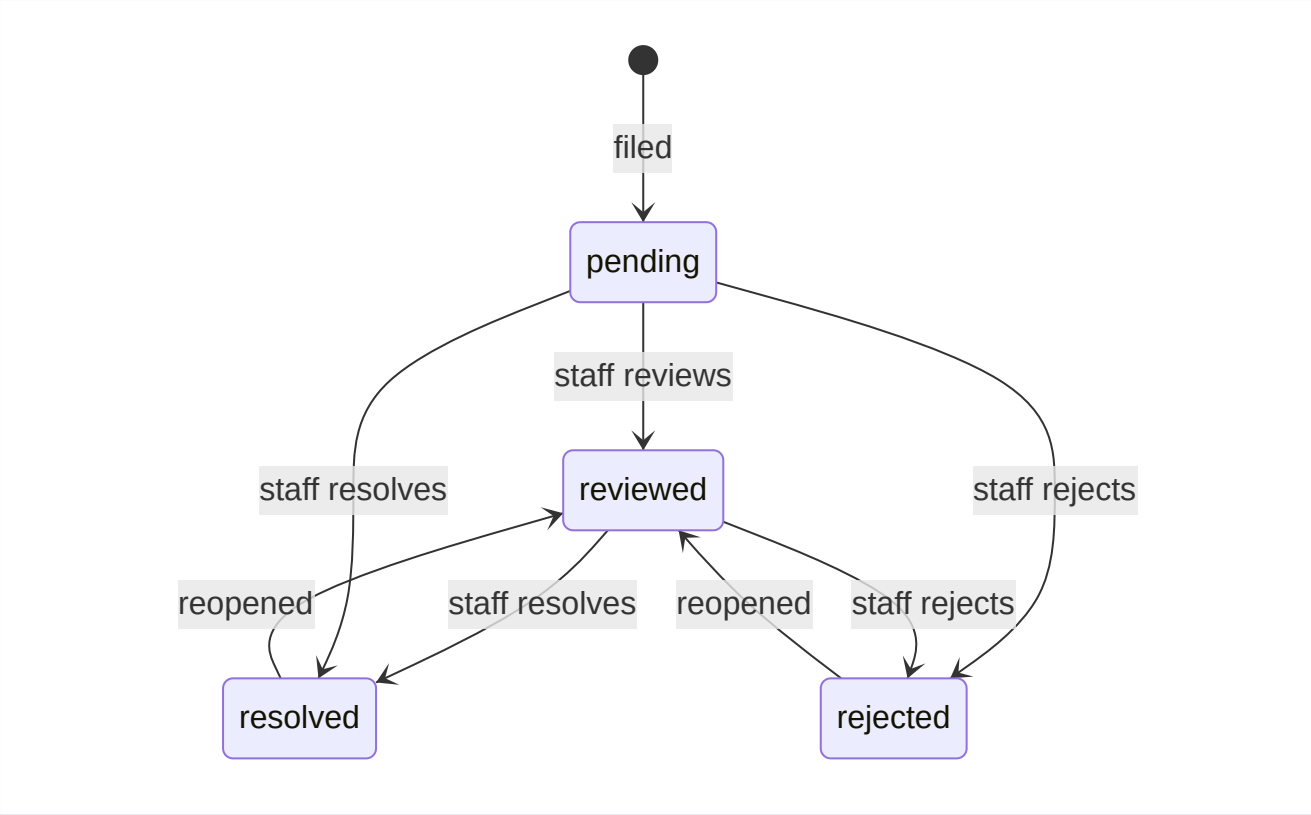
Filterable fields: `id`, `title`, `description`, `category`, `priority`, `status`, `studentId`, `assignedTo`, `departmentId`, `createdAt`, `updatedAt`, `resolvedAt`, `cf.<key>`.

9.1 Create complaint

```json
{
  "title": "Wi-Fi outage in Hostel B",
  "description": "No connectivity on floors 2-4 since Monday evening.",
  "category": "infrastructure",
  "priority": "high",
  "departmentId": "<uuid>",
  "studentId": "<uuid>",
  "subjectIds": ["<uuid>"],
  "customFields": { "hostel_block": "B" }
}
```

Field	Rules
`title`	Required, ≤ 200
`description`	Required, ≤ 5000
`category`	≤ 100, default `general`
`priority`	`low`, `medium` (default), `high`, `urgent`
`departmentId`	Optional; defaults from the reporter's department
`studentId`	Optional; defaults to the reporter
`subjectIds`	Up to 50 additional people the complaint concerns

9.2 Lifecycle



- `PUT /complaints/{id}/status` body: `{ "status": "resolved", "resolution": "Router replaced", "note": "Verified with students" }`. `resolution` ≤ 5000, `note` ≤ 1000.
- `PUT /complaints/{id}/assignee` body: `{ "assignedTo": "<user-uuid>" }`.
- `PATCH` accepts `title`, `description`, `category`, `priority`, `status`, `assignedTo`, `resolution`, `note`, `revealReporter`, `customFields`.
- Reporters without staff scope can edit only while the complaint is `pending`. Afterwards the API returns `403 complaint is already being processed`.
- Every status change writes a history entry with actor, from/to status and note.

9.3 Complaint response (`ComplaintDTO`)

Field	Description
`id`, `title`, `description`, `category`, `priority`, `status`	Core data
`studentId`, `studentName`, `subjects[]`	Who the complaint is about
`assignedTo`, `assignedToName`	Current handler
`departmentId`, `departmentName`	Owning department
`resolution`, `resolvedAt`	Outcome
`raisedBy`, `raisedByName`, `raisedByDepartments[]`	Reporter; `null` when hidden from the caller
`reporterVisible`, `revealReporter`	Whether the reporter is shown to the caller / to everyone

Field	Description
`customFields`, `createdAt`, `updatedAt`	Extension data and timestamps

9.4 Dashboard statistics

`GET /complaints/stats` returns counts within the caller's scope: `total`, per status, `unassigned`, per priority, `open`, `assignedToMe`, `mine`, `overdue`, `criticalOpen`, `createdLast7Days`, `resolvedLast7Days`, `avgResolutionHours`, `trend[] {day, created, resolved}` and `departments[] {name, total, open}`.

10. Comments API

Prefix: `/complaints/{complaintId}/comments`

Method	Path	Description	Permission
GET	`/`	List comments (internal ones only for staff)	`complaint_comments:read`
POST	`/`	Add `{ "body": "…", "internal": false }` → `201`	`complaint_comments:write`
DELETE	`/{commentId}`	Delete → `204`	`complaint_comments:delete`

`body` is required, ≤ 4000 characters. Comments are immutable once posted.

11. Files API

Service: `storage-service` · Prefix: `/complaints/{complaintId}/files`

Method	Path	Description	Permission
GET	`/`	List files of a complaint	`complaint_files:read`
POST	`/upload-url`	Request a presigned upload URL → `201`	`complaint_files:write`
POST	`/{fileId}/confirm`	Confirm the upload finished → `200` with the file	`complaint_files:write`
GET	`/{fileId}/download-url`	Get a presigned download URL (`?inline=true` to view in browser)	`complaint_files:read`
DELETE	`/{fileId}`	Delete the file → `204`	`complaint_files:delete`

11.1 Upload sequence

1. `POST /upload-url` with `{ "fileName": "photo.jpg", "contentType": "image/jpeg", "sizeBytes": 482113 }` (`fileName` ≤ 200, `contentType` ≤ 150, `sizeBytes` > 0).
2. Response: `{ "fileId", "uploadUrl", "method": "PUT", "headers": { … }, "expiresAt" }`.
3. Send the bytes with `PUT uploadUrl` and exactly the returned headers. This request goes to the files domain, not the API.

4. `POST /{fileId}/confirm`. The file status changes from `pending` to `uploaded`.

Unconfirmed uploads are purged automatically. Download URLs expire; request a new one each time.

JSON

```
{
  "id": "…", "complaintId": "…", "uploadedBy": "…", "uploadedByName": "Aarav Mehta",
  "fileName": "photo.jpg", "contentType": "image/jpeg", "sizeBytes": 482113,
  "kind": "attachment", "status": "uploaded", "createdAt": "…"
}
```

12. Custom fields API

Service: `custom-field-service` · Prefix: `/custom-fields`

Method	Path	Description	Permission
GET	`/custom-fields`	List definitions (paged, default size 50 sorted by `sortOrder` ascending, filterable)	`custom_fields:read`
GET	`/custom-fields/applicable?entity=users&roleId=…&departmentId=…`	Active definitions that apply in a context	Authenticated
GET	`/custom-fields/{id}`	Get one	`custom_fields:read`
POST	`/custom-fields`	Create → `201`	`custom_fields:write`
PUT	`/custom-fields/{id}`	Replace (entity, key and type are immutable)	`custom_fields:update`
PATCH	`/custom-fields/{id}`	Update `label`, `helpText`, `required`, `options`, `sortOrder`, `active`	`custom_fields:update`
DELETE	`/custom-fields/{id}`	Delete with its values → `204`	`custom_fields:delete`

JSON

```
{
  "entity": "complaints",
  "key": "hostel_block",
  "label": "Hostel block",
  "helpText": "Block where the issue happened",
  "dataType": "select",
  "required": false,
  "options": ["A", "B", "C"],
  "appliesToDepartmentId": null,
  "appliesToRoleId": null,
  "sortOrder": 10,
  "active": true
}
```

Field	Rules
`entity`	`users`, `departments`, `roles`, `complaints`
`key`	`^[a-z][a-z0-9_]{0,49}$`
`dataType`	`text`, `number`, `bool`, `date`, `select`, `multi_select`
`options`	Up to 100 values (≤ 200 chars each), for `select` and `multi_select`
`minValue`, `maxValue`, `maxLength`, `pattern`	Optional validation (`pattern` ≤ 500 chars, regular expression)

Values are sent inside the owning entity's `customFields` object.

13. Notifications API

Service: `notification-service` · Prefix: `/notifications`

Method	Path	Description	Permission
GET	`/notifications`	Inbox (paged, default size 20, filterable)	`notifications:read`
GET	`/notifications/unread-count`	`{ "unread": n }`	`notifications:read`
GET	`/notifications/stream`	SSE stream (`ready`, `heartbeat`, `notification`)	`notifications:read`
POST	`/notifications/read-all`	Mark all read → `{ "updated": n }`	`notifications:update`
PATCH	`/notifications/{id}/read`	Mark one read → `204`	`notifications:update`
DELETE	`/notifications/{id}`	Delete → `204`	`notifications:delete`
POST	`/notifications/send`	Send to users, roles and departments → `202`	`notifications:write` (department/all)
POST	`/notifications/devices`	Register a push device → `201`	`notification_devices:write`
GET	`/notifications/devices`	List own devices	`notification_devices:read`
DELETE	`/notifications/devices/{id}`	Remove a device → `204`	`notification_devices:delete`
GET	`/notifications/preferences`	Get preferences	Authenticated
PUT	`/notifications/preferences`	Update preferences	Authenticated

13.1 Send request

```json
{
  "userIds": ["…"],
  "roleIds": ["…"],
  "departmentIds": ["…"],
  "type": "announcement",
  "title": "Exam schedule published",
  "body": "Check the portal for your timetable.",
  "data": { "url": "/notifications" },
  "priority": "normal",
  "includeSender": false
}
```

At least one target is required. Limits: 1000 users, 100 roles, 100 departments; `type` ≤ 100, `title` ≤ 200, `body` ≤ 2000. Response: `{ "message": "…", "recipients": 128 }`.

13.2 Notification object

```json
{
  "id": "…", "userId": "…", "type": "complaint.status", "title": "Complaint resolved",
  "body": "Wi-Fi outage in Hostel B was resolved", "data": { "complaintId": "…" },
  "priority": "normal", "source": "complaint-service",
  "readAt": null, "expiresAt": null, "createdAt": "…"
}
```

13.3 Devices and preferences

- Register: `{ "platform": "android", "token": "<fcm token, 20–4096 chars>", "deviceName": "Pixel 8" }`. `platform` is `android`, `ios` or `web`.
- Preferences: `{ "pushEnabled": true, "realtimeEnabled": true, "mutedTypes": ["announcement"] }` (up to 100 muted types).

13.4 Push integration (administrators)

Prefix: `/notifications/admin` · Permission: `integrations` (`read` / `update` / `delete`)

Method	Path	Description
GET	`/integrations`	Integration status (`provider` , `enabled` , `configured` , `config` , `updatedAt`)
PUT	`/integrations/fcm`	`{ "enabled": true, "serviceAccountJson": "{…}" }` ; the secret is encrypted at rest
DELETE	`/integrations/fcm`	Remove FCM credentials → `204`
POST	`/integrations/fcm/test`	Send a test push `{ "deviceToken"?, "title"?, "body"? }` → `{ credentialsValid, devicesTried, sent, failed, detail }`

Method	Path	Description
GET	`/deliveries`	Delivery log (paged): `channel` (`realtime` / `fcm`), `status` (`sent` / `failed` / `skipped`), `detail`
GET	`/stats`	`{ realtimeSent, fcmSent, fcmFailed, skipped, activeDevices, notificationsLast24h }`

14. Face recognition API

Service: `face-service` · Prefix: `/faces`

Method	Path	Description	Permission
POST	`/faces/enroll/{userId}`	Enroll or replace a user's face	`face_enrollments:write` / `update` (scoped to target)
POST	`/faces/recognize`	Recognise faces in an image	`face_recognition:read` ; `face_live_recognition:read` when `source=camera`
GET	`/faces/enrollments`	List enrollments visible to the caller	`face_enrollments:read`
DELETE	`/faces/enrollments/{userId}`	Remove an enrollment → `204`	`face_enrollments:delete`
POST	`/faces/enrollments/{userId}/unlock`	Reset the self-enrollment lock → `204`	`face_enroll_lock:update`
GET	`/faces/audit?event=&subjectId=`	Face audit log (paged, default size 20; optional `event` and `subjectId` filters)	`face_audit:read`
GET	`/faces/enroll/bulk/template?format=csv|xlsx`	Bulk enrollment template	`face_import:read`
POST	`/faces/enroll/bulk`	Bulk enroll from a file (`multipart` , part `file`) → `202`	`face_import:write`

14.1 Enroll

```json
{ "image": "<base64 JPEG or PNG, ≤ 8 MB decoded>" }
```

The largest face is used. Response:

```json
{
  "userId": "…", "name": "Aarav Mehta", "email": "aarav@college.com",
  "enrolledAt": "…", "selfEnrollCount": 1, "selfEnrollLocked": true,
  "canReplace": false, "canRemove": true, "canUnlock": false
}
```

A user enrolling themselves is limited to `FACE_SELF_ENROLL_LIMIT` attempts (default 1). Further self-enrollments return `403` until someone with department or all scope unlocks or re-enrolls them.

14.2 Recognise

```json
{ "image": "<base64>", "source": "upload" }
```

`source` is `upload` (default) or `camera` (live recognition). Response:

```json
{
  "image": "<base64 PNG with boxes and names>",
  "width": 1280, "height": 720,
  "faces": [
    { "box": { "x": 410, "y": 122, "width": 160, "height": 182 },
      "confidence": 0.82, "matched": true,
      "user": { "id": "…", "name": "Aarav Mehta", "email": "aarav@college.com" } },
    { "box": { "x": 820, "y": 140, "width": 150, "height": 170 },
      "confidence": 0.0, "matched": false, "user": null }
  ]
}
```

`confidence` is the cosine similarity. It is only returned in detail to holders of `face_match_details`. A face whose best match the caller may not read is returned as not matched.

14.3 Audit entry

`{ id, event, actorId, actorName, subjectId, subjectName, matched, confidence, facesDetected, detail, createdAt }` with `event` in `recognize`, `live_recognize`, `enroll`, `replace`, `remove`, `unlock`, `bulk_import`.

15. System monitor API

Service: `monitor-service` · Prefix: `/system` · Permission: `system_monitor:read:all`

Method	Path	Description
GET	`/system/overview`	`{ intervalSeconds, latest: SystemSnapshot, history: HistoryPoint[] }`

Method	Path	Description
GET	`/system/stream`	SSE: `overview` first, then `snapshot` events, plus `heartbeat`

A `SystemSnapshot` contains:

Part	Content
`host`	Hostname, OS, CPU model and cores, per-core and total CPU, load average, memory and swap, disks, disk and network rates
`services[]`	Per instance: status, registration, uptime, CPU, heap, RSS, threads, request and error rate, latency, GC rate, DB pool usage
`database`	Version, size, connections by state, longest query, transaction and rollback rates, cache hit ratio, row rates, deadlocks, temp bytes, largest tables
`redis`	Version, role, clients, memory, ops/s, hit ratio, keys, evictions, throughput
`storage`	Files, bytes, pending files, evidence files
`issues[]`	Human-readable problems detected in the latest sample

16. Health and metrics

Endpoint	Exposure	Purpose
`/actuator/health` on each service port	Docker network only	Liveness and readiness for Compose health checks
`/actuator/prometheus` on each service port	Docker network only	Prometheus metrics
`/healthz` on the frontend	Internal	Nginx health check

`/api/actuator/*` is blocked at the web origin on purpose.

17. Examples

17.1 cURL: sign in and list pending complaints (native style)

```bash
BASE=https://app.example.com/api
TOKENS=$(curl -s -X POST "$BASE/auth/login" \
  -H 'Content-Type: application/json' -H 'X-Token-Delivery: body' \
  -d '{"email":"admin@college.com","password":"<password>"}')
ACCESS=$(echo "$TOKENS" | jq -r .tokenInfo.accessToken)

curl -s "$BASE/complaints?status=eq.pending&priority=in.
(high,urgent)&size=20&sortBy=createdAt&direction=desc" \
  -H "Authorization: Bearer $ACCESS" | jq '.content[] | {title, priority, status}'
```

17.2 TypeScript: browser client with single-flight refresh

```ts
let refreshing: Promise<void> | null = null;

async function api<T>(path: string, init: RequestInit = {}): Promise<T> {
  const send = () => fetch(`/api${path}`, {
    ...init,
    credentials: "include",
    headers: { "Content-Type": "application/json", ...init.headers },
  });
  let res = await send();
  if (res.status === 401) {
    refreshing ??= fetch("/api/auth/refresh", { method: "POST", credentials: "include" })
      .then((r) => { if (!r.ok) throw new Error("Session expired"); })
      .finally(() => { refreshing = null; });
    await refreshing;
    res = await send();
  }
  if (!res.ok) throw new Error((await res.json()).message);
  return res.status === 204 ? (undefined as T) : res.json();
}
```

17.3 Upload a file

```bash
R=$(curl -s -X POST "$BASE/complaints/$CID/files/upload-url" -H "Authorization: Bearer $ACCESS" \
  -H 'Content-Type: application/json' \
  -d "{\"fileName\":\"photo.jpg\",\"contentType\":\"image/jpeg\",\"sizeBytes\":$(stat -c%s photo.jpg)}")
curl -s -X PUT "$(echo "$R" | jq -r .uploadUrl)" -H 'Content-Type: image/jpeg' --data-binary @photo.jpg
curl -s -X POST "$BASE/complaints/$CID/files/$(echo "$R" | jq -r .fileId)/confirm" -H "Authorization: Bearer $ACCESS"
```

17.4 Subscribe to notifications

```ts
const es = new EventSource("/api/notifications/stream", { withCredentials: true });
es.addEventListener("notification", (e) => showToast(JSON.parse(e.data)));
es.onerror = () => { es.close(); setTimeout(connect, 3000); };
```

18. Versioning and compatibility

- The API version follows the product version (`1.1.0`). Breaking changes are only introduced in a new major version and announced in the release notes.
- Clients must ignore unknown response fields; new optional fields can appear in minor releases.
- Deprecated endpoints are kept for at least one minor release and marked in the release notes.

DOCUMENT 04

DATA · PAI-DB-004

Database & Migrations

Data model, row-level security, database roles, functions and the Flyway migration process.

1. Overview
2. Entity-relationship model
3. Tables
4. Database roles
5. Row-level security
6. Triggers and invariants
7. Domain functions
8. Migrations
9. Operational queries

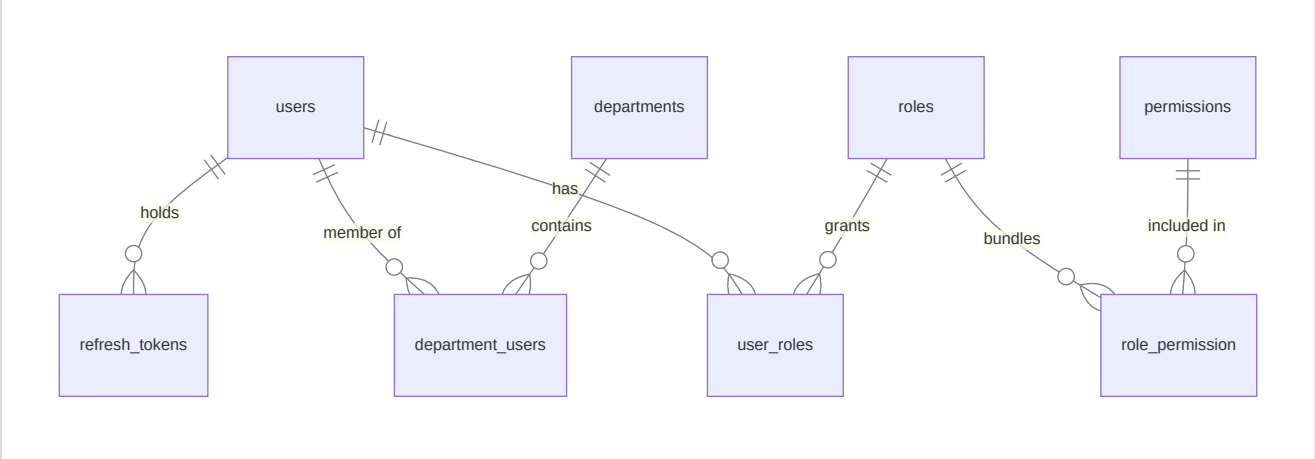
1. Overview

ProctorAI stores all relational data, vector embeddings and authorization logic in a single PostgreSQL 16 database named by `POSTGRES_DB` (default `proctor`). The image is `pgvector/pgvector:pg16`, pinned by digest. The schema is managed by Flyway and applied by the `db-migrator` container before any service starts.

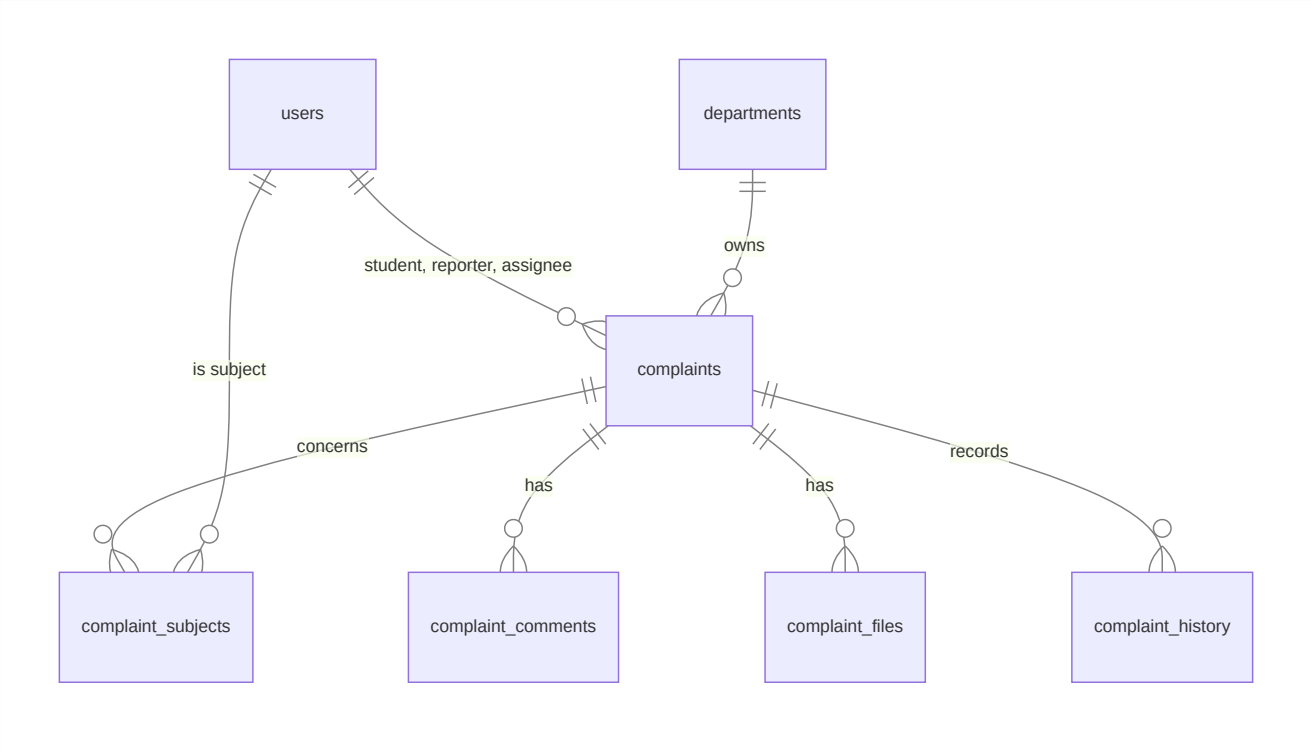
Item	Value
Engine	PostgreSQL 16 with the `vector` extension
Extensions	`pgcrypto`, `vector`, `hstore`, `uuid-ossp`
Primary keys	`uuid` generated by `gen_random_uuid()`
Timestamps	`timestamptz`; `updated_at` maintained by triggers
Authorization	Row-Level Security on every business table
Migration tool	Flyway, location `backend/db-schema/src/main/resources/db/migration/`
Data volume	Docker volume `postgres_data` (production project `proctor-prod`)

2. Entity-relationship model

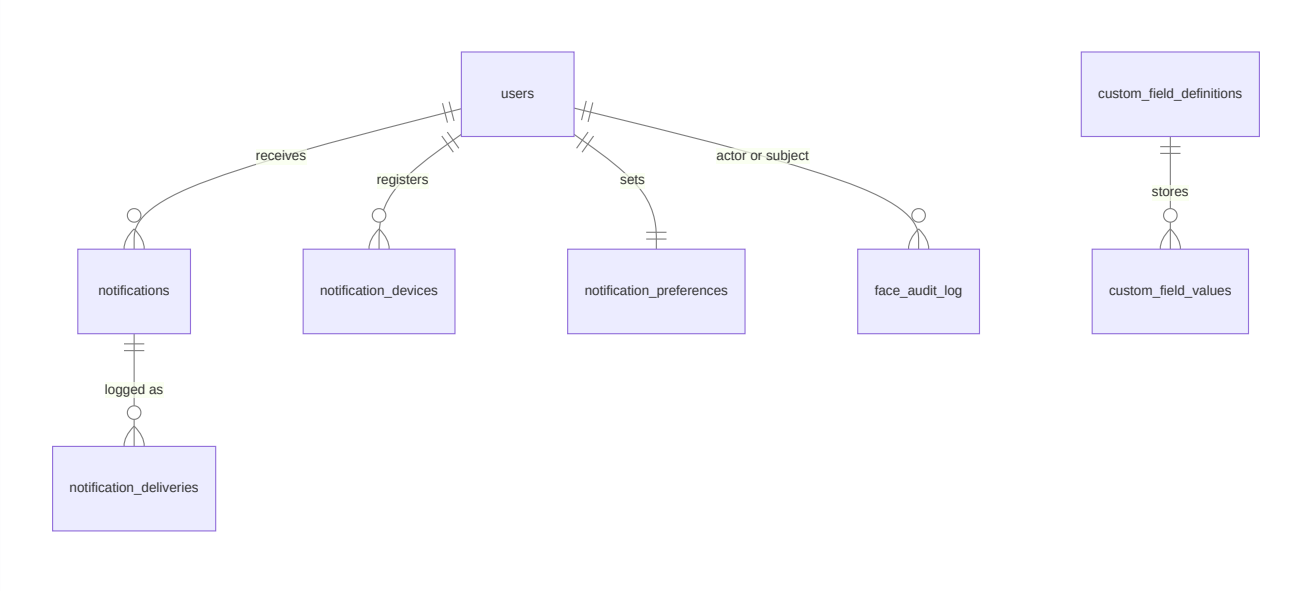
2.1 Identity and access



2.2 Complaints



2.3 Notifications, custom fields and biometrics



3. Tables

3.1 Identity and access

Table	Key columns	Notes
`users`	`id`, `email` (unique), `name`, `password` (bcrypt), `enabled`, `verified`, `created_at`, `updated_at`	Seeded administrator `admin@college.com`
`roles`	`id`, `name` (unique), `description`, `level` (≥ 0, default 100), `is_system`, `is_superuser`, `reveal_identity`	Lower level is more senior
`permissions`	`id`, `entity`, `action`, `scope`, `description`	Catalogue of entity × action × scope
`role_permission`	`role_id`, `permission_id`	Many-to-many
`user_roles`	`user_id`, `role_id`	Many-to-many; rows are immutable (remove and add)
`departments`	`id`, `name`, `code` (unique), `description`, `active`	
`department_users`	`user_id`, `department_id`	Membership drives department scope
`refresh_tokens`	`id`, `user_id`, `family_id`, `token_hash` (unique), `issued_at`, `expires_at`, `revoked_at`, `user_agent`, `ip_address`	Only hashes are stored

3.2 Complaints

Table	Key columns	Notes
`complaints`	`id`, `title`, `description`, `category` (default `general`), `priority`, `status`, `student_id`, `raised_by`, `reveal_reporter`, `assigned_to`, `department_id`, `resolution`, `active`, `resolved_at`	Status and priority are enums
`complaint_subjects`	`complaint_id`, `user_id`	Additional people a complaint concerns
`complaint_comments`	`id`, `complaint_id`, `author_id`, `body`, `internal`	Immutable after insert
`complaint_files`	`id`, `complaint_id`, `uploaded_by`, `file_name`, `content_type`, `size_bytes`, `status` (`pending` / `uploaded`), `kind` (`attachment` / `evidence`), `file_url`	Storage fields immutable; cannot return to `pending`
`complaint_history`	`id`, `complaint_id`, `actor_id`, `from_status`, `to_status`, `note`	Written only by triggers

3.3 Extensions, notifications and biometrics

Table	Key columns	Notes
`custom_field_definitions`	`entity`, `key`, `label`, `data_type`, `required`, `options`, `default_value`, `min_value`, `max_value`, `max_length`, `pattern`, `applies_to_role_id`, `applies_to_department_id`, `sort_order`, `active`	Entity, key and type are immutable
`custom_field_values`	`definition_id`, `entity`, `entity_id`, `value` (jsonb)	Purged when the owner is deleted
`notifications`	`user_id`, `type`, `title`, `body`, `data`, `priority` (`low` / `normal` / `high`), `source`, `read_at`, `expires_at`	
`notification_devices`	`user_id`, `platform` (`android` / `ios` / `web`), `token` (unique), `device_name`, `active`, `last_seen_at`	
`notification_preferences`	`user_id`, `push_enabled`, `realtime_enabled`, `muted_types`	One row per user
`integration_settings`	`provider` (`fcm`), `enabled`, `config`, `secret_ciphertext`, `updated_by`	Secret encrypted with AES-GCM
`notification_deliveries`	`notification_id`, `channel` (`realtime` / `fcm`), `status` (`sent` / `failed` / `skipped`), `detail`, `device_id`	
`face_audit_log`	`event`, `actor_id`, `subject_id`, `matched`, `confidence`, `faces_detected`, `detail`	Append-only audit
`face_embeddings`	Created by Spring AI on first boot of face-service	512-dimension vectors; accessed only by face-service

3.4 Enumerations

Type	Values
`permission_action`	`read`, `write`, `update`, `delete`
`permission_scope`	`own`, `department`, `all`
`complaint_status`	`pending`, `reviewed`, `resolved`, `rejected`
`complaint_priority`	`low`, `medium`, `high`, `urgent`
`custom_field_entity`	`users`, `departments`, `roles`, `complaints`
`custom_field_type`	`text`, `number`, `bool`, `date`, `select`, `multi_select`
`file_status`	`pending`, `uploaded`
`face_event`	`recognize`, `live_recognize`, `enroll`, `replace`, `remove`, `unlock`, `bulk_import`
`entity`	The 28 protected entities listed in the API Handbook

4. Database roles

Role	Login	RLS	Used by	Privileges
Owner (`POSTGRES_USER`)	Yes	Bypasses (owner)	`db-migrator` only	Owns all objects; runs migrations
`authenticator`	Yes	Enforced	`auth-service`	Read `users`; full access to `refresh_tokens`
`proctor`	Yes	Enforced	All other services	DML on business tables under RLS; `CREATE` on schema for Spring AI's vector table
`auth_owner`	No	Bypasses	`security definer` functions	Owner of privileged functions; read all tables

Services refuse to start when connected as a role that bypasses RLS (`DatabaseRoleGuard`). The owner password is never given to a service.

5. Row-level security

5.1 Session context

Each request runs in a transaction that first executes:

```sql
select set_config('auth.uid', :uid, true), set_config('auth.email', :email, true);
```

The third argument `true` makes the setting local to the transaction, so it cannot leak to another request on a pooled connection. Helper `uid()` reads it.

5.2 Policy pattern

For most tables the policies are generated with one template per action:

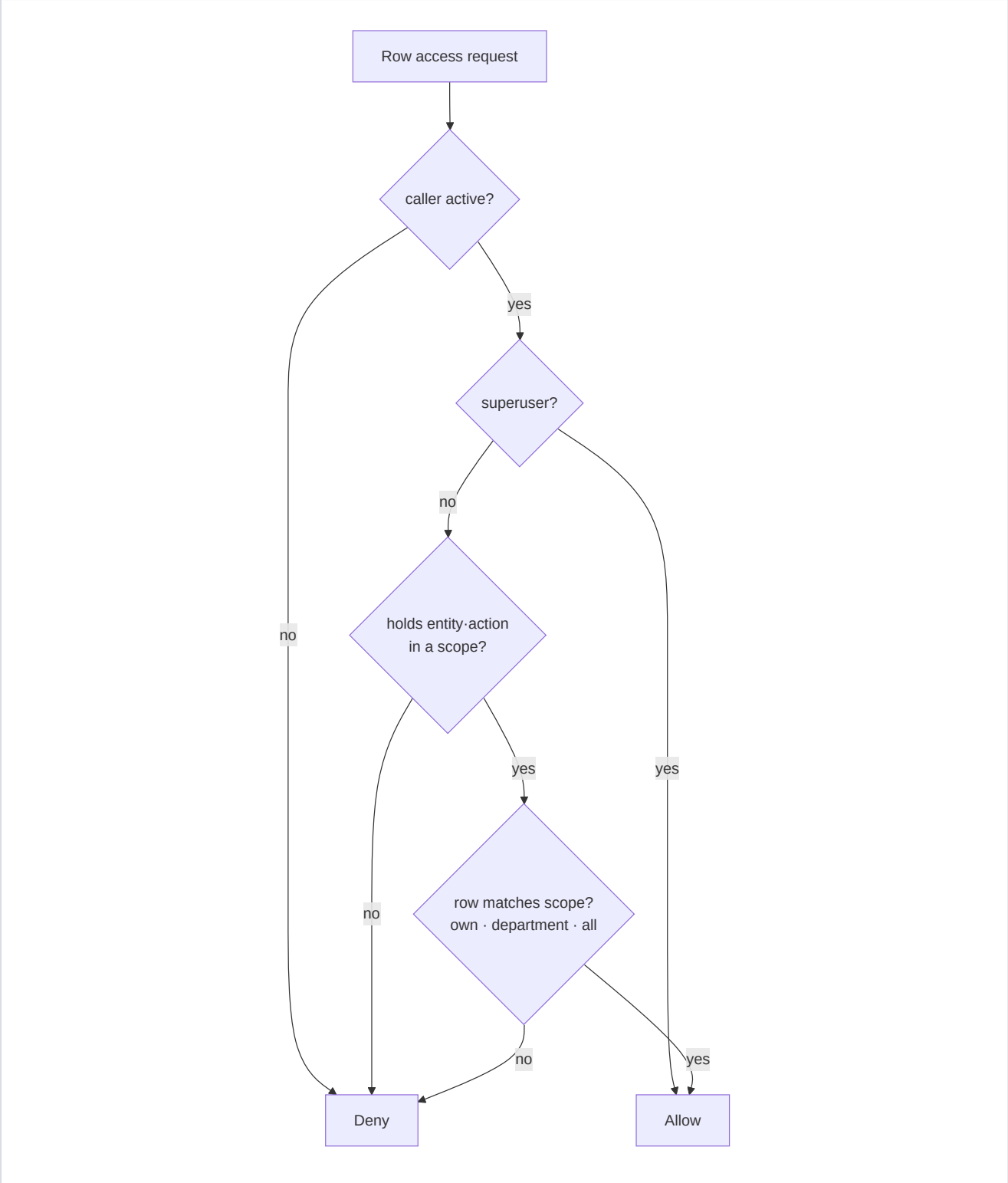
```sql
create policy <table>_select on <table> for select to proctor
  using (can_access('<entity>'::entity, 'read', to_jsonb(<table>)));
create policy <table>_insert on <table> for insert to proctor
  with check (can_access('<entity>'::entity, 'write', to_jsonb(<table>)));
create policy <table>_update on <table> for update to proctor
  using (can_access('<entity>'::entity, 'update', to_jsonb(<table>)))
  with check (can_access('<entity>'::entity, 'update', to_jsonb(<table>)));
create policy <table>_delete on <table> for delete to proctor
  using (can_access('<entity>'::entity, 'delete', to_jsonb(<table>)));
```

Complaints, comments, files, history, subjects, custom fields, refresh tokens, notification preferences, deliveries and the face audit log have hand-written policies with extra rules (for example, internal comments are hidden from reporters and subjects).

5.3 Security functions

Function	Purpose
`uid()`	Current user id from `auth.uid`
`is_active_user(id)`, `is_superuser(id)`	Account and role state
`user_min_level(id)`, `caller_min_level()`	Seniority (lowest level held)
`holds_permission(entity, action, scopes[])`	Whether the caller holds a permission in any of the scopes
`shares_department(id)`, `in_department(dept)`	Department relationships
`user_relation(target)`	`self`, `junior`, `same_role`, `parallel_role` or `senior`
`row_scope_match(entity, row, scope)`	Whether a row is inside `own`, `department` or `all` for the caller
`can_access(entity, action, row)`	Main decision used by policies
`has_scope(entity, action, scopes[], row)`	Whether access is granted through specific scopes (used to separate staff from self-service)
`can_view_user`, `can_manage_user`, `can_manage_role`	User and role management decisions
`complaint_staff(...)`, `is_complaint_subject(...)`	Complaint visibility helpers

5.4 Decision flow



6. Triggers and invariants

Trigger function	Guarantees
`set_updated_at`	Maintains `updated_at`

Trigger function	Guarantees
`guard_roles`	System roles cannot be deleted or renamed; flags immutable; level 0 and superuser roles only by system admins
`guard_user_roles`	No self role changes; only system admins assign superuser roles; the last system admin keeps the role
`guard_role_permission`	No permission escalation; superuser permissions are implicit
`guard_users`	Email, enabled and verified changes need sufficient level; no self enable/disable; the last system admin cannot be deleted or disabled
`guard_department_users`	Department membership changes respect management rights
`complaints_before_insert`	Sets `raised_by`, defaults `student_id` and `department_id`, forces `pending` for self-service, sets `resolved_at`
`complaints_before_update`	`raised_by` immutable; reporter visibility protected; self-service edits only while `pending`
`complaints_record_history`	Writes `complaint_history` on create and every status change, with the note from `app.status_note`
`complaint_children_before_insert`, `complaint_children_immutable`	Comments and files belong to one complaint permanently
`guard_files_update`	Storage fields immutable; uploaded files cannot return to `pending`
`guard_custom_field_definitions`	Entity, key and type cannot change
`purge_custom_values`	Deletes custom values when the owner row is deleted
`revoke_refresh_tokens`	Revokes sessions when a user is disabled or the password changes

Rule violations raise SQLSTATE `42501` with a code prefix (for example `COMPLAINT_LOCKED:`). The API maps them to `403` and strips the prefix from the message.

7. Domain functions

Migration	Functions
`V1_7__storage_functions.sql`	`purge_stale_uploads(interval)`, `complaint_file_usage(...)`
`V1_8__notification_functions.sql`	`system_create_notification`, `system_delivery_profile`, `system_active_devices`, `system_deactivate_device`, `system_log_delivery`, `system_integration`, `register_device`, `purge_old_notifications`, `complaint_parties`, `resolve_notification_recipients`
`V1_9__monitor_functions.sql`	Database statistics used by the system monitor

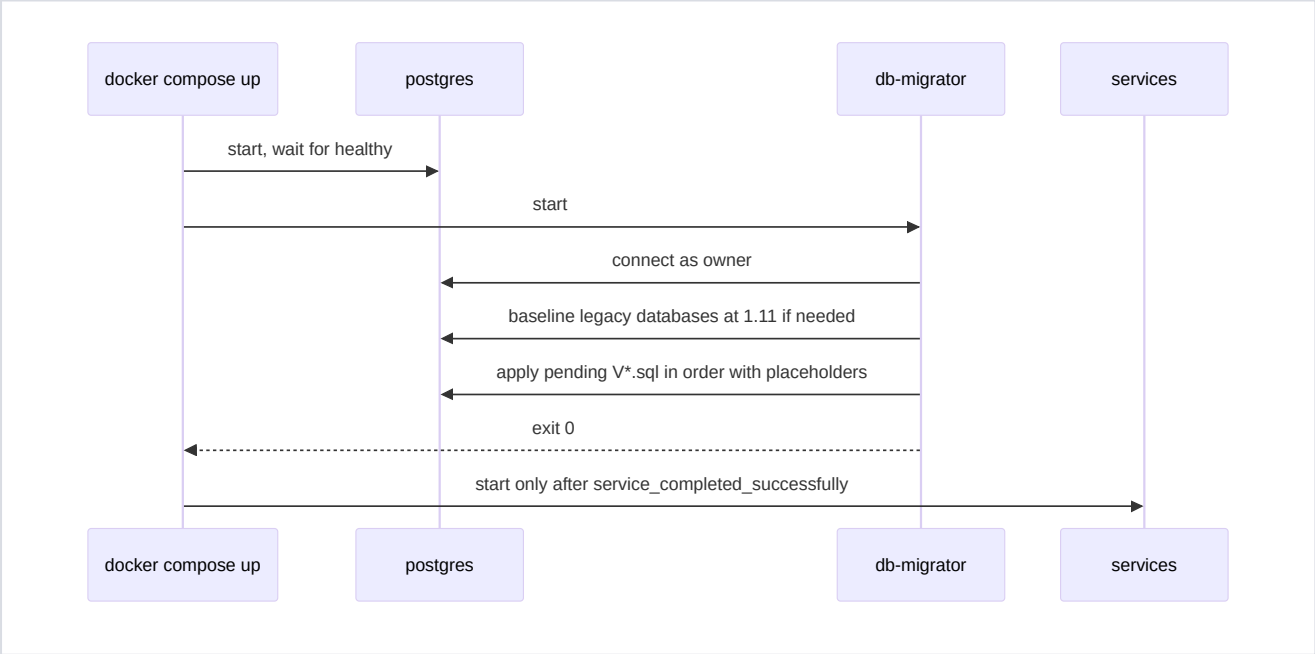
`system_*` functions are `security definer` and run as `auth_owner` for trusted internal work (for example creating notifications for other users) without widening the `proctor` role.

8. Migrations

8.1 Inventory

Version	File	Contents
1.1	`V1_1__roles.sql`	Database roles and their passwords
1.2	`V1_2__extensions.sql`	`pgcrypto`, `vector`, `hstore`, `uuid-ossp`
1.3	`V1_3__schema.sql`	Types, tables, indexes, RLS enabled
1.4	`V1_4__security_functions.sql`	`can_access`, `has_scope` and helpers
1.5	`V1_5__rls_policies.sql`	Row-level security policies
1.6	`V1_6__triggers.sql`	Guards, history and housekeeping triggers
1.7	`V1_7__storage_functions.sql`	Upload purge and usage
1.8	`V1_8__notification_functions.sql`	Notification delivery and recipients
1.9	`V1_9__monitor_functions.sql`	Monitoring statistics
1.10	`V1_10__seed.sql`	Permissions, default roles and grants, administrator
1.11	`V1_11__grants.sql`	Privileges per role

8.2 How migrations run



Placeholders `authenticator_password`, `app_password` and `admin_password` are filled from `AUTHENTICATOR_USER_PASSWORD`, `APP_USER_PASSWORD` and `ADMIN_PASSWORD`. Databases created by the older `init/` scripts are detected and baselined at 1.11 without changes.

8.3 Writing a new migration

1. Create the next version file, for example `V2__add_complaint_tags.sql`. Use `V2_1__…`, `V2_2__…` for a series.
2. Make it forward-only and idempotent where practical (`if not exists`, `on conflict do nothing`).
3. Enable RLS and add policies for every new table. Grant only the privileges the service needs in the same migration.
4. Add or update tests in `backend/db-migrator/src/test` (`MigrationTest`, `RowLevelSecurityTest`).
5. Run `cd backend && ./gradlew build`, then `./setup.sh` to apply locally.

> **CAUTION**
> Never edit or rename a released migration. Flyway validates checksums and refuses to start when a released file changes, which stops every service.

8.4 Migration review checklist

Check	Why
New tables have `enable row level security` and policies	Otherwise rows are invisible (or exposed, if policies are too wide)
Grants added for `proctor` / `authenticator`	Services fail with permission errors otherwise
Long operations avoid full table locks	Prevent downtime during upgrade
Destructive changes split across releases (expand, migrate, contract)	Allows rollback of application images
Backup taken before production upgrade	Migrations are not reversible automatically

9. Operational queries

```sql
select version, description, installed_on, success
from flyway_schema_history order by installed_rank desc limit 5;

select relname, n_live_tup, pg_size_pretty(pg_total_relation_size(relid))
from pg_stat_user_tables order by pg_total_relation_size(relid) desc limit 10;

select count(*) filter (where revoked_at is null and expires_at > now()) as active_sessions
from refresh_tokens;
```

Run them with `docker compose … exec postgres psql -U "$POSTGRES_USER" -d "$POSTGRES_DB"`.

DOCUMENT 05

SECURITY · PAI-SEC-005

Security & Hardening Guide

Threat model, security controls, secret management, hardening checklist and privacy guidance.

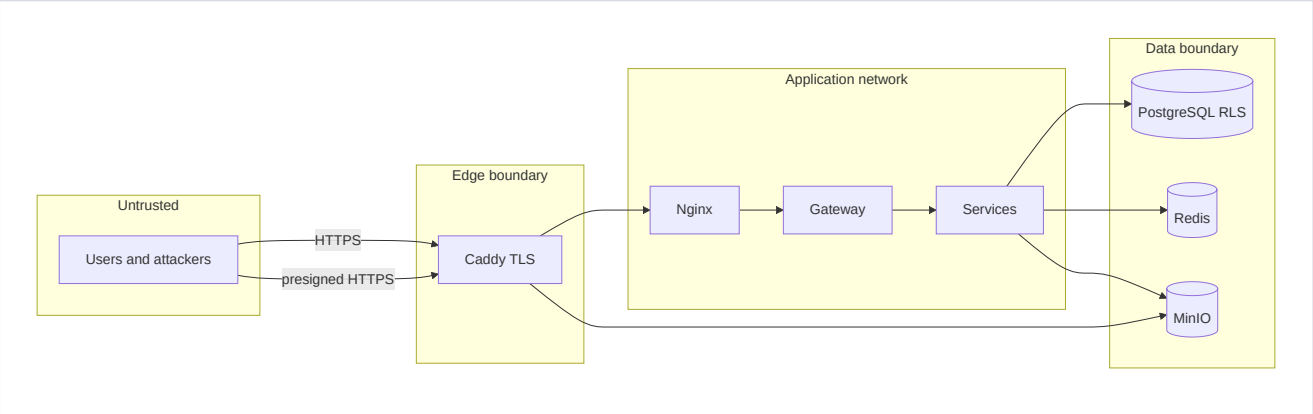
1. Security objectives
2. Threat model
3. Identity and session security
4. Authorization controls
5. Network and transport security
6. Container and host hardening
7. Secret management
8. Data protection and privacy
9. Production hardening checklist
10. Security testing
11. Vulnerability reporting

1. Security objectives

Objective	Statement
Confidentiality	Users only see the rows, files and identities their permissions allow, including reporter identities and biometric matches.
Integrity	Workflow rules (complaint locking, history, role levels, last administrator) cannot be bypassed by any client or service bug.
Availability	Brute force and resource exhaustion are limited; services restart automatically.
Accountability	Status changes and every face recognition operation are audited with actor and time.
Privacy	Biometric data stays on the institution's server and is never sent to external AI services.

2. Threat model

The model uses STRIDE over the main trust boundaries: Internet → edge, edge → application, application → data.



Threat (STRIDE)	Example	Controls
Spoofing	Password guessing, stolen refresh token	bcrypt (cost 12 for the seeded admin), per-IP rate limit, per-account lockout, refresh rotation with reuse detection, `HttpOnly` cookies
Tampering	Changing a complaint after processing, escalating a role	Database triggers (`COMPLAINT_LOCKED` , `PERMISSION_ESCALATION` , `SYSTEM_ROLE_PROTECTED`), RLS `with check` clauses
Repudiation	Denying a status change or a face lookup	Immutable `complaint_history` , `face_audit_log` , trace IDs in logs
Information disclosure	Reading other departments' complaints, identifying a face outside scope, CORS abuse from the files domain	RLS on every table, out-of-scope matches returned as unknown, reporter masking, strict CORS that refuses storage and registry origins

Threat (STRIDE)	Example	Controls
Denial of service	Login floods, huge uploads	Gateway token bucket, body size limits (300 MB web, 250 MB files), 8 MB face image limit, memory limits per container
Elevation of privilege	Service compromise reading all data, container breakout	Least-privilege DB roles without `BYPASSRLS`, role guard at startup, read-only containers, all capabilities dropped, `no-new-privileges`, non-root users

3. Identity and session security

Control	Implementation
Password storage	bcrypt via `BcryptPasswordHasher`; minimum length 8 on create
Access token	JWT, HMAC-SHA with `JWT_SECRET` (startup fails if shorter than 32 bytes), 15-minute default lifetime, issuer checked
Refresh token	Random opaque value; only a hash is stored; rotated on every refresh; reuse revokes the family
Cookies	`HttpOnly`, `Secure` (production), `SameSite=Lax`; refresh cookie scoped to `/auth`
Revocation	Changing the password or disabling a user revokes all refresh tokens (trigger)
Rate limiting	Redis token bucket on `/auth/login` keyed by real client IP
Lockout	10 failed attempts lock the account for 15 minutes (configurable)
Client IP	Resolved from `X-Forwarded-For` honouring only `APP_TRUSTED_PROXIES` hops, so clients cannot spoof their address
Default credentials	Production setups replace `admin123` with a random password shown once

4. Authorization controls

Authorization is enforced in three layers:

1. **Service layer:** the JWT filter authenticates every request except the public auth paths.
2. **Database policies:** every query runs under RLS bound to `auth.uid` and calls `can_access(entity, action, row)`.
3. **Database triggers:** invariants that cannot be expressed as row filters, such as seniority and workflow locks.

Additional protections:

- Users can only manage themselves and more junior users. Managing peers requires explicit `same_role_peers` or `parallel_role_peers` permissions. Viewing more senior users requires

`senior_users`.

- Nobody can grant a permission they do not hold.
- Superuser roles can only be created or assigned by a system administrator; the last system administrator cannot be removed, disabled or deleted.
- Reporter identity is masked unless the viewer's role has `revealIdentity`, the viewer holds `complaint_reporter:read`, or the complaint sets `revealReporter`.
- Internal comments are visible only to staff.

5. Network and transport security

Layer	Control
Edge	Caddy with automatic TLS (Let's Encrypt, ZeroSSL fallback), HTTP→HTTPS redirect, HSTS `max-age=31536000`, HTTP/3, `admin off`
Headers	`X-Content-Type-Options: nosniff`, `Referrer-Policy: strict-origin-when-cross-origin`, `Server` and `Via` removed; Nginx adds `X-Frame-Options: SAMEORIGIN` on the app
Exposure	Only the edge publishes ports 80/443. Registry (8761) and MinIO console (9201) bind to `127.0.0.1` only. Port 9200 is published only until DNS is ready
Registry	HTTP Basic auth on every Eureka call, password ≥ 32 characters, constant-time comparison; public domain serves only the dashboard paths
Metrics	`/actuator/*` reachable only inside the Docker network; `/api/actuator/` returns 404
CORS	Origins managed by setup; wildcards and non-HTTP schemes rejected; storage and registry origins refused
Files	Presigned URLs with expiry; MinIO CORS restricted to the app origin; app user limited to one bucket by policy

> **WARNING**
> The IP bootstrap stage uses plain HTTP. Do not enter real passwords or personal data until DNS is configured and setup has switched to HTTPS.

6. Container and host hardening

Control	Production setting
Read-only root filesystem	`read_only: true`, with a 256 MB `tmpfs` for `/tmp`
Linux capabilities	`cap_drop: ALL` (edge adds only `NET_BIND_SERVICE`)
Privilege escalation	`security_opt: no-new-privileges:true`
User	Non-root `app` user; face-service on distroless without a shell
Resources	Memory limits per container; JVM exits on OOM

Control	Production setting
Images	Third-party images pinned by SHA-256 digest; JRE trimmed with `jlink`
Logs	JSON, rotated 10 MB × 5

Host recommendations:

- Ubuntu 22.04/24.04 LTS with unattended security upgrades enabled.
- `ufw` allowing only SSH, 80/tcp, 443/tcp, 443/udp (and 9200/tcp only before DNS).
- SSH with keys only, root login disabled, `fail2ban` or equivalent.
- Docker daemon kept current; only administrators in the `docker` group (membership is root-equivalent).

7. Secret management

Secret	Purpose	Minimum strength	Rotation impact
`POSTGRES_PASSWORD`	Database owner	32+ random chars	Change in DB and env together
`AUTHENTICATOR_USER_PASSWORD`, `APP_USER_PASSWORD`	Service DB roles	32+ random chars	Rerun migrator (sets passwords), restart services
`JWT_SECRET`	Signs access tokens	≥ 32 bytes	All access tokens become invalid; users refresh automatically
`REDIS_PASSWORD`	Redis auth	32+ random chars	Restart Redis and all services
`MINIO_ROOT_PASSWORD`, `S3_SECRET_KEY`	Object storage	32+ random chars	Update MinIO user and storage-service
`DISCOVERY_PASSWORD`	Registry auth	≥ 32 chars (enforced)	Restart every service
`NOTIFICATION_SECRET_KEY`	Encrypts FCM credentials (AES-256-GCM, key derived with SHA-256)	32+ random chars	Re-enter FCM credentials after rotation
`ADMIN_PASSWORD`	Seeded admin on a new database	20 random chars, shown once	Change in the app afterwards

Rules:

1. Setup generates every secret with a cryptographically secure generator and writes the env file with `600` permissions.
2. Never commit `.env`, `.env.production` or `docker/.env`. They are already in `.gitignore`.

3. Keep an encrypted offline copy of the env file (password manager or vault). Without it, existing volumes cannot be reused.
4. Rotate secrets after staff changes or any suspected exposure, during a maintenance window.

8. Data protection and privacy

ProctorAI processes personal data, including biometric data, which is special-category data under laws such as the GDPR and India's DPDP Act 2023.

Topic	Guidance
Lawful basis	Obtain explicit, informed consent before face enrollment; document the purpose
Minimisation	Store only embeddings and audit records; do not keep raw enrollment images longer than needed
Access	Grant `face_recognition`, `face_live_recognition` and `face_match_details` only to roles that need them
Transparency	Users can view their own face audit records (`face_audit:read:own`)
Erasure	Delete a user's enrollment with `DELETE /faces/enrollments/{userId}` ; deleting a user cascades their data
Retention	Define retention for notifications, audit logs and resolved complaints; purge functions exist for notifications and stale uploads
Breach response	Follow the incident runbook (PAI-OPS-009) and notify authorities within the legal deadline
Model licence	InsightFace buffalo_l models are licensed for non-commercial research use; review before commercial deployment

9. Production hardening checklist

#	Check	Status
1	Domains configured; HTTPS active on app, files and registry	☐
2	Port 9200 closed in firewall after DNS is ready	☐
3	Administrator password changed after first sign-in; one-time password removed from notes	☐
4	`APP_COOKIE_SECURE=true` and only HTTPS origins in `APP_ALLOWED_ORIGINS`	☐
5	`APP_EXTRA_ORIGINS` reviewed; no unnecessary origins	☐
6	Env file permissions `600` ; encrypted offline copy stored	☐
7	SSH keys only; root login disabled; unattended upgrades on	☐
8	Default roles reviewed; face permissions limited to required staff	☐

#	Check	Status
9	Backups scheduled and a restore tested (PAI-BDR-008)	☐
10	Monitoring and alerting active (PAI-OPS-009)	☐
11	CAA DNS record restricting certificate authorities (optional)	☐
12	`FACE_MATCH_THRESHOLD` validated on local data to balance false matches and misses	☐
13	Docker images updated to the latest ProctorAI patch release	☐

10. Security testing

Activity	Frequency	Tooling
Automated tests (RLS, auth flow, rate limit, JWT)	Every build	`./gradlew build` with Testcontainers
Dependency scanning	Every build	OWASP Dependency-Check or `gradle dependencyCheckAnalyze`, `npm audit`
Container image scanning	Every release	Trivy or Grype
TLS configuration check	After deployment	SSL Labs, `testssl.sh`
Penetration test	Yearly or after major changes	OWASP ASVS Level 2 as baseline

11. Vulnerability reporting

Report suspected vulnerabilities privately to the maintainers (Varshit Tyagi & Mohini Teotia) through the GitHub repository's private security advisory feature. Do not open public issues for security problems. Include affected version, reproduction steps and impact. Maintainers acknowledge within 5 working days and coordinate disclosure after a fix is released.

DOCUMENT 06

OPERATIONS · PAI-DEP-006

Production Deployment Runbook

Step-by-step procedure to plan, install, verify and hand over a production ProctorAI server.

1. Purpose and scope
2. Roles and responsibilities
3. Planning
4. Pre-deployment checklist
5. Server preparation
6. DNS records
7. Method A: deploy published images
8. Method B: deploy from source
9. Deployment flow
10. Post-deployment verification
11. Initial application setup
12. Handover
13. Rollback of a failed first installation
14. Everyday commands
15. Troubleshooting during deployment
16. Sign-off

1. Purpose and scope

This runbook takes an operator from an empty Linux server to a verified, HTTPS-secured ProctorAI installation. It covers both supported methods:

Method	Command	Use when
A. Published images (recommended)	`cd docker && ./setup.sh`	Running a released version; fastest (about 5 minutes)
B. Build from source	`./setup.sh prod`	Running your own code changes (about 20 minutes)

Both methods configure the same hardened stack: Caddy edge with automatic TLS, read-only containers, health-checked startup and automatic database migrations.

> **IMPORTANT**
> Use one method per server. Both share the Docker volumes of project `proctor-prod` but keep separate secrets files (`docker/.env` and `backend/.env.production`). Switching methods requires copying the secrets first, otherwise the services cannot open the existing database.

2. Roles and responsibilities

Role	Responsibility
Deployment owner	Executes this runbook, records results in the sign-off table
DNS administrator	Creates A (and optional CAA) records
Security reviewer	Confirms the hardening checklist in PAI-SEC-005
Application administrator	First sign-in, password change, initial roles and departments

3. Planning

3.1 Server sizing

Profile	Users	vCPU	RAM	Disk (SSD)
Evaluation	≤ 50	2	8 GB	40 GB
Small institution	≤ 1,000	4	16 GB	100 GB
Medium institution	≤ 5,000	8	32 GB	250 GB + object storage growth

Face recognition is CPU-bound; more cores improve recognition latency. Plan object storage for complaint attachments (up to 250 MB per file).

3.2 Prerequisites

Item	Requirement
OS	Ubuntu 22.04 or 24.04 LTS (any 64-bit x86_64 Linux works)
Software	Docker 25+ and Docker Compose 2.21+
Network	Public IPv4 address; inbound TCP 80, TCP 443, UDP 443; outbound HTTPS
DNS	Three names, for example `app.example.com`, `files.example.com`, `registry.example.com`
Email	Contact address for certificate expiry notices
Access	SSH user with `sudo`

3.3 Information to collect

Value	Example	Recorded
Server public IP	`203.0.113.10`	
Web app domain	`app.example.com`	
File storage domain	`files.example.com`	
Service registry domain	`registry.example.com`	
Certificate email	`ops@example.com`	
Target version	`1.1.0`	

4. Pre-deployment checklist

#	Check	Done
1	Server provisioned with the sizing above and SSH access confirmed	☐
2	OS fully updated (`sudo apt update && sudo apt upgrade -y`)	☐
3	Cloud firewall / security group allows TCP 80, TCP 443, UDP 443, SSH (and TCP 9200 temporarily)	☐
4	DNS A records created, or a plan to start on the IP address first	☐
5	No AAAA records unless the server really serves IPv6	☐
6	Secure place ready to store the env file and the one-time admin password	☐
7	Maintenance window agreed (for upgrades of existing installations)	☐

5. Server preparation

```bash
sudo apt update && sudo apt install -y git curl
curl -fsSL https://get.docker.com | sudo sh
sudo usermod -aG docker "$USER"
newgrp docker
docker run --rm hello-world

sudo ufw allow OpenSSH
sudo ufw allow 80/tcp
sudo ufw allow 443/tcp
sudo ufw allow 443/udp
sudo ufw allow 9200/tcp
sudo ufw enable
```

Port 9200 serves uploaded files only until DNS is in place. It is closed again in the post-deployment steps.

Recommended extras: enable `unattended-upgrades`, disable SSH password authentication and root login, and set the server time zone and NTP.

6. DNS records

Type	Name	Value
A	`app.example.com`	Server public IPv4
A	`files.example.com`	Server public IPv4
A	`registry.example.com`	Server public IPv4
CAA (optional)	`example.com`	`0 issue "letsencrypt.org"`
CAA (optional)	`example.com`	`0 issue "sectigo.com"` (allows the ZeroSSL fallback)

Setup prints these records with the real IP and the status of each one. DNS can be added before or after installation.

7. Method A: deploy published images

```bash
git clone --depth 1 https://github.com/vishaltyagi807/Proctor-AI.git
cd Proctor-AI/docker
./setup.sh
```

Only the `docker/` folder is needed; it can also be copied with `scp -r docker <user>@<server>:~/proctorai`. If the Docker Hub repository is private, run `docker login` first.

Setup will:

1. Create `.env` with new random secrets (mode `600`).
2. Ask for the three domains and the certificate email (leave domains empty to start on the IP address).
3. Detect the public IP, check DNS and generate the Caddy configuration.
4. Pull the images for the selected version and start every container, waiting until all are healthy.
5. Replace the default administrator password with a random one and **display it once**.
6. Print the URLs and any DNS records still missing.

For an unattended install:

```bash
APP_DOMAIN=app.example.com FILES_DOMAIN=files.example.com \
REGISTRY_DOMAIN=registry.example.com ACME_EMAIL=ops@example.com \
./setup.sh --yes
```

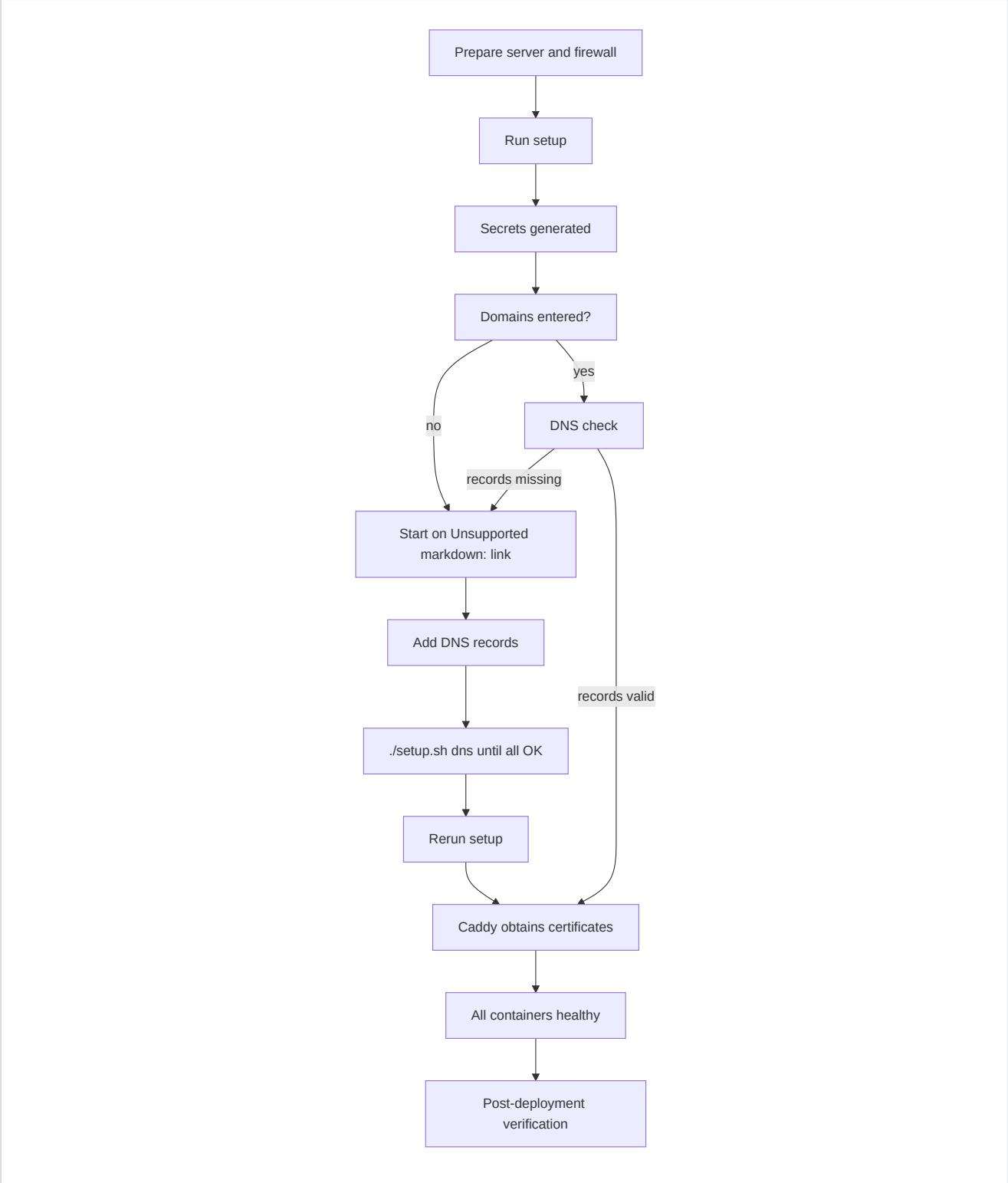
Add `PUBLIC_IP=<address>` if the server cannot detect its own address. Select a version with `./setup.sh update 1.1.0`.

8. Method B: deploy from source

```bash
git clone https://github.com/vishaltyagi807/Proctor-AI.git
cd Proctor-AI
./setup.sh prod
```

The prompts and automatic steps match Method A. In addition, setup downloads the face models (about 190 MB, checksum-verified) and builds every image. Settings are stored in `backend/.env.production`.

9. Deployment flow



9.1 Switching from IP to HTTPS

Part	Before DNS	After DNS
Web app	`http://<server-ip>`	`https://app.example.com`

Part	Before DNS	After DNS
File storage	`http://<server-ip>:9200`	`https://files.example.com`
Service registry	SSH tunnel to `http://localhost:8761`	`https://registry.example.com`
MinIO console	SSH tunnel to `http://localhost:9201`	SSH tunnel only

```bash
./setup.sh dns            # every record must show OK
./setup.sh                # Method A, rerun to switch to HTTPS
./setup.sh prod --yes     # Method B
```

10. Post-deployment verification

#	Check	Command / action	Expected
1	All containers healthy	`./setup.sh status` (Method B: `--prod`)	Every service `healthy`; `db-migrator` and `minio-init` exited `0`
2	Migrations applied	`./setup.sh logs db-migrator`	`Successfully applied` or `Schema is up to date`
3	HTTPS valid	`curl -I https://app.example.com`	`HTTP/2 200`, `strict-transport-security` header present
4	HTTP redirects	`curl -I http://app.example.com`	`308` to HTTPS
5	API reachable	`curl -s https://app.example.com/api/auth/refresh -X POST`	`401` JSON error body
6	Actuator hidden	`curl -I https://app.example.com/api/actuator/health`	`404`
7	Sign-in	Browser: sign in as `admin@college.com` with the one-time password	Dashboard loads
8	Password change	Users → your account → Edit → New password	Success; old password rejected at next sign-in
9	File upload	Create a test complaint and attach a file	Upload completes; download works
10	Real-time	Change the complaint status in a second browser	Notification appears without reload
11	Face service	Faces → enroll a test user, then recognise	Match shown; audit entry recorded
12	System monitor	Admin → System	All services listed as UP
13	Registry protected	Open `https://registry.example.com`	Basic auth prompt
14	Close temporary port	`sudo ufw delete allow 9200/tcp` (and in the cloud firewall)	Port closed

11. Initial application setup

1. Sign in as the administrator and change the password (Users → your account → Edit → New password).
2. Create departments (Departments → New, or bulk create).
3. Review default roles (`teacher`, `student`) and create additional roles as needed (Roles & access).
4. Define custom fields required by the institution (Custom fields).
5. Import users from CSV/Excel using the template (Users → Import).
6. Optionally configure FCM push (Admin → Notifications → Integrations).
7. Assign face recognition permissions only to roles that need them.
8. Create a second superuser account for break-glass access and store its credentials securely.

12. Handover

Deliver to the institution:

Item	Location
URLs (app, files, registry)	Deployment record
Env file backup (encrypted)	Password manager / vault
Administrator credentials	Handed over in person or via password manager
Backup schedule and restore test result	PAI-BDR-008
Monitoring contacts and alert routes	PAI-OPS-009

13. Rollback of a failed first installation

A first installation has no data to protect. To start again:

```bash
./setup.sh purge          # Method A (asks to confirm); Method B: ./setup.sh purge --prod
```

> **WARNING**
>
> Purge deletes the database, uploads, face enrollments and TLS certificates. Let's Encrypt limits repeated issuance for the same names, so avoid purging production more than necessary. For rollback of upgrades, follow PAI-REL-010.

14. Everyday commands

Task	Method A (`docker/`)	Method B (repository root)
Status	`./setup.sh status`	`./setup.sh status --prod`
Logs	`./setup.sh logs [service]`	`./setup.sh logs --prod [service]`
TLS and access logs	`./setup.sh logs edge`	`./setup.sh logs --prod edge`
Stop / remove (keep data)	`./setup.sh stop` / `down`	`./setup.sh stop --prod` / `down --prod`
Update	`./setup.sh update [version]`	Pull code, then `./setup.sh prod --yes`
DNS status	`./setup.sh dns`	`./setup.sh dns`
Registry and MinIO console	`ssh -L 8761:127.0.0.1:8761 -L 9201:127.0.0.1:9201 <user>@<server>`	Same

15. Troubleshooting during deployment

Symptom	Likely cause	Resolution
Certificates never become active	Ports 80/443 blocked or DNS not pointing to the server	Open ports; `./setup.sh dns` ; `./setup.sh logs edge`
Validation fails only sometimes	AAAA record to an address the server does not serve	Remove the AAAA record
A service stays `unhealthy`	Missing variable, DB not ready, memory limit	`./setup.sh logs <service>` ; check free RAM
`db-migrator` exits non-zero	Edited released migration or database error	Read its logs; never edit released migrations
Browser CORS error	Origin not allowed	Add to `APP_EXTRA_ORIGINS` , rerun setup
Uploads fail after HTTPS switch	Old files URL cached in a native app	Rebuild the app with the HTTPS `VITE_SERVER_URL`
Image pull denied	Private repository	`docker login`

16. Sign-off

Step	Name	Date	Signature
Deployment completed			

Step	Name	Date	Signature
Verification completed			
Security checklist completed			
Handover accepted			

DOCUMENT 07

OPERATIONS · PAI-CFG-007

Configuration Reference

Every environment variable, default, owner and effect across the ProctorAI stack.

1. Configuration model
2. Database
3. Authentication and sessions
4. Gateway, web and CORS
5. Domains and TLS (production)
6. Service registry
7. Redis
8. Object storage (MinIO / S3)
9. Temporary files
10. Notifications
11. Face recognition
12. Versions and images
13. Frontend and native apps
14. JVM and container settings (compose)
15. Ports
16. Change procedure

1. Configuration model

ProctorAI is configured entirely through environment variables. Setup scripts generate one env file per stack:

Stack	File	Created by
Development	`backend/.env`	`./setup.sh`
Production from source	`backend/.env.production`	`./setup.sh prod`
Production from published images	`docker/.env`	`docker/setup.sh`

All env files are created with mode `600`. `backend/.env.example` lists every variable with placeholders.

1.1 Ownership legend

Owner	Meaning
Setup (once)	Generated on first run and never changed automatically. Changing it later requires a coordinated rotation.
Setup (every run)	Rewritten on every setup run from what is actually served. Do not edit by hand.
Prompt	Asked by setup; rerun setup to change.
You	Safe to edit; rerun setup (`--yes`) to apply.
Compose override †	Read by the service, but not passed through by any compose file in 1.1.0. Setting it in the env file has no effect. To change it, add it to the service's `environment:` block in the compose file used by setup, then rerun setup.

> **IMPORTANT**
> After editing an env file, apply it with `./setup.sh` (development), `./setup.sh prod --yes` (source) or `./setup.sh` in `docker/` (published images). The production compose file fails fast when a required variable is missing.

2. Database

Variable	Default / generated	Owner	Description
`POSTGRES_DB`	`proctor`	Setup (once)	Database name
`POSTGRES_USER`	`proctor_admin`	Setup (once)	Database owner; used only by `db-migrator`
`POSTGRES_PASSWORD`	32 random chars	Setup (once)	Owner password

Variable	Default / generated	Owner	Description
`AUTHENTICATOR_USER_NAME`	`authenticator`	Setup (once)	Role used by auth-service
`AUTHENTICATOR_USER_PASSWORD`	32 random chars	Setup (once)	Password of `authenticator` (set by the migrator)
`APP_USER_NAME`	`proctor`	Setup (once)	Role used by all other services
`APP_USER_PASSWORD`	32 random chars	Setup (once)	Password of `proctor` (set by the migrator)
`ADMIN_PASSWORD`	`admin123` (dev) / 20 random chars (prod)	Setup (once)	Password of `admin@college.com` on a new database
`R2DBC_URL`	`r2dbc:postgresql://postgres:5432/<db>` in containers	Compose	Reactive connection URL
`SPRING_DATASOURCE_URL`	`jdbc:postgresql://postgres:5432/<db>` (face-service)	Compose	JDBC URL for Spring AI vector store
`MIGRATION_DATABASE_URL`	`jdbc:postgresql://postgres:5432/<db>`	Compose	JDBC URL for Flyway
`POSTGRES_PORT`	`5433`	You (dev)	Host port for PostgreSQL in development

3. Authentication and sessions

Variable	Default	Owner	Description
`JWT_SECRET`	64 random chars	Setup (once)	HMAC key for access tokens; must be ≥ 32 bytes
`JWT_ISSUER`	`proctor.local`	Setup (once)	`iss` claim
`JWT_ACCESS_EXPIRE_IN_SEC`	`900`	You	Access token lifetime (15 minutes)
`JWT_REFRESH_EXPIRE_IN_SEC`	`2592000`	You	Refresh token lifetime (30 days)
`APP_COOKIE_SECURE`	`false` (dev) / `true` (prod)	Setup (every run)	`Secure` flag on cookies
`APP_COOKIE_SAME_SITE`	`Lax`	Compose override †	`SameSite` attribute of cookies
`LOGIN_MAX_FAILURES`	`10`	You	Failed sign-ins before an account is locked
`LOGIN_LOCKOUT`	`15m`	You	Lockout duration (Spring duration format: `30s`, `15m`, `1h`)

4. Gateway, web and CORS

Variable	Default	Owner	Description
`APP_ALLOWED_ORIGINS`	Generated	Setup (every run)	CORS origins: dev `http://localhost:<port>`, `http://127.0.0.1:<port>`; prod `https://<app domain>` (or `http://<ip>` before DNS) plus native app origins
`APP_EXTRA_ORIGINS`	empty	You	Extra allowed origins, comma separated. Validated: no wildcards, HTTP(S) only, storage and registry origins refused
`APP_TRUSTED_PROXIES`	`1` (dev) / `2` (prod)	Compose	Number of proxies that append to `X-Forwarded-For` (Caddy and Nginx in production)
`LOGIN_RATE_LIMIT_REPLENISH`	`1`	You	Tokens added per second to each client IP bucket
`LOGIN_RATE_LIMIT_BURST`	`30`	You	Bucket capacity
`LOGIN_RATE_LIMIT_COST`	`3`	You	Tokens per sign-in attempt (defaults: 10 immediate attempts, then 20/min)
`GATEWAY_SERVER_PORT`	`7050`	You (dev)	Host port of the gateway in development
`FRONTEND_PORT`	`5173`	You (dev)	Host port of the web app in development

5. Domains and TLS (production)

Variable	Example	Owner	Description
`APP_DOMAIN`	`app.example.com`	Prompt	Web app and API
`FILES_DOMAIN`	`files.example.com`	Prompt	Object storage for presigned URLs
`REGISTRY_DOMAIN`	`registry.example.com`	Prompt	Eureka dashboard
`ACME_EMAIL`	`ops@example.com`	Prompt	Let's Encrypt account email
`PUBLIC_IP`	detected	Prompt / auto	Server public IPv4, used before DNS is ready

6. Service registry

Variable	Default	Owner	Description
`DISCOVERY_USERNAME`	`proctor-discovery`	Setup (once)	Basic auth user for Eureka
`DISCOVERY_PASSWORD`	48 random chars	Setup (once)	Basic auth password (≥ 32 chars, enforced)

Variable	Default	Owner	Description
`DISCOVERY_SERVER_PORT`	`8761`	You	Host port (bound to `127.0.0.1` in production)
`DISCOVERY_SERVER_REFRESH_INTERVAL`	`2`	You (dev)	Registry refresh interval in seconds
`DISCOVERY_DASHBOARD_ENABLED`	`true` in prod compose	Compose	Serves the dashboard behind the registry domain
`EUREKA_CLIENT_SERVICEURL_DEFAULTZONE`	Built from the credentials	Compose	Registry URL used by every service

7. Redis

Variable	Default	Owner	Description
`REDIS_HOST`	`redis` in containers	Compose	Host
`REDIS_PORT`	`6379` in containers / `6390` dev host port	Compose / You	Port
`REDIS_PASSWORD`	32 random chars	Setup (once)	`requirepass` password

8. Object storage (MinIO / S3)

Variable	Default	Owner	Description
`MINIO_ROOT_USER`	`proctorminio`	Setup (once)	MinIO root user (console and init only)
`MINIO_ROOT_PASSWORD`	32 random chars	Setup (once)	MinIO root password
`S3_ACCESS_KEY`	`proctor-app`	Setup (once)	Application user, limited to one bucket
`S3_SECRET_KEY`	40 random chars	Setup (once)	Application user secret
`S3_BUCKET`	`proctor-files`	Setup (once)	Bucket for complaint files
`S3_ENDPOINT`	`http://minio:9000` in containers	Compose	Internal S3 endpoint
`S3_PUBLIC_ENDPOINT`	`http://localhost:9200` (dev) / `https://<files domain>`	Setup (every run)	Base URL in presigned links; must be reachable by clients
`MINIO_CORS_ALLOW_ORIGIN`	App origin	Setup (every run)	Origins allowed to upload directly to MinIO
`S3_PORT`	`9200`	You	Host port for the S3 API (dev; prod only before DNS)
`S3_CONSOLE_PORT`	`9201`	You	Host port of the MinIO console (bound to `127.0.0.1`)

9. Temporary files

Variable	Default	Owner	Description
`APP_STORAGE_DIR`	`./storage` (dev) / `/data` (containers)	Compose	Directory for import job files (user-service, face-service)
`APP_STORAGE_TEMP_TTL`	`30m`	Compose override †	Maximum age of temporary job files
`APP_STORAGE_SWEEP_INTERVAL`	`15m`	Compose override †	How often the sweeper runs

10. Notifications

Variable	Default	Owner	Description
`NOTIFICATION_SECRET_KEY`	48 random chars	Setup (once)	Encrypts FCM service-account secrets (AES-256-GCM). Rotating it requires re-entering FCM credentials

FCM credentials themselves are entered in the application (Admin → Notifications), not in env files.

11. Face recognition

Variable	Default	Owner	Description
`FACE_SERVER_PORT`	`7059`	Compose	HTTP port
`FACE_DETECTION_MODEL_PATH`	`./models/face-detector.onnx` (image: `/app/models/…`)	Image	SCRFD model file
`FACE_EMBEDDING_MODEL_PATH`	`./models/face-embedder.onnx`	Image	ArcFace model file
`FACE_DETECTION_INPUT_SIZE`	`640`	You	Detector input square size
`FACE_DETECTION_SCORE_THRESHOLD`	`0.5`	You	Minimum detector confidence
`FACE_DETECTION_NMS_THRESHOLD`	`0.4`	You	Non-maximum suppression IoU
`FACE_MATCH_THRESHOLD`	`0.45` in generated env files	You	Minimum cosine similarity to report a match
`FACE_MAX_IMAGE_BYTES`	`8388608`	You	Maximum decoded image size (8 MB)
`FACE_EMBEDDING_DIMENSIONS`	`512`	Image	Must match the embedder output

Variable	Default	Owner	Description
`FACE_SELF_ENROLL_LIMIT`	`1`	Compose override †	Self-enrollment attempts before lock

> **NOTE**
> Generated env files set `FACE_MATCH_THRESHOLD=0.45`. The production compose file falls back to `0.75` only when the variable is absent. Higher values reduce false matches but increase missed matches. Validate the threshold on a sample of your own users before go-live.

12. Versions and images

Variable	Default	Stack	Description
`APP_VERSION`	`latest`	Source production	Tag for locally built images
`PROCTOR_VERSION`	`1.1.0`	Published images	Image version pulled from Docker Hub (`./setup.sh update <version>` sets it)
`IMAGE_REPOSITORY`	`vishaltyagi807/proctor-ai`	Published images	Docker Hub repository

13. Frontend and native apps

Variable	Where	Description
`VITE_SERVER_URL`	Build time (`npm run build`, `tauri:*:build`)	Server URL baked into native apps; when empty, the app asks at sign-in
`APPLE_DEVELOPMENT_TEAM`	iOS builds	Apple team id for signing
`APPLE_SIGNING_IDENTITY`, `APPLE_ID`, `APPLE_PASSWORD`, `APPLE_TEAM_ID`	macOS builds	Code signing and notarisation
`ANDROID_HOME`, `NDK_HOME`, `JAVA_HOME`	Android builds	Toolchain locations

14. JVM and container settings (compose)

Setting	Development	Production
`JAVA_TOOL_OPTIONS`	C1 only, Serial GC, CDS archive (fast start)	`-XX:+UseG1GC -XX:MaxRAMPercentage=60 -XX:InitialRAMPercentage=25 -XX:+ExitOnOutOfMemoryError`

Setting	Development	Production
Memory limits	None	768 MB per service, 2560 MB face-service, 512 MB registry and monitor, 384 MB migrator, 256 MB edge, 128 MB frontend
Health check	`/actuator/health`	Same, 15 s interval, 120 s start period
Logging	Docker default	`json-file`, 10 MB × 5

15. Ports

Port	Development (host)	Production (host)
80 / 443 tcp, 443 udp	—	Caddy edge (public)
5173	Web app	—
7050	Gateway	—
7051–7060	Each service on `127.0.0.1`	—
8761	Registry	`127.0.0.1` only
5433	PostgreSQL	—
6390	Redis	—
9200	MinIO API	Public only before DNS
9201	MinIO console	`127.0.0.1` only

16. Change procedure

1. Back up the env file: `cp .env .env.bak-$(date +%F)` (keep permissions `600`).
2. Edit only variables owned by **You** or **Prompt**.
3. Apply with the setup command for the stack.
4. Check `./setup.sh status` and the affected service logs.
5. Record the change in the operations log.

DOCUMENT 08

OPERATIONS · PAI-BDR-008

Backup, Restore & Disaster Recovery Plan

What to protect, how to back it up, how to restore it and how to recover from the loss of a server.

1. Objectives
2. What must be protected
3. Backup procedure
4. Automation
5. Restore procedures
6. Disaster recovery: full server loss
7. Point-in-time recovery (optional)
8. Restore testing
9. Backup log

1. Objectives

Metric	Target	Meaning
RPO (Recovery Point Objective)	24 hours (standard) · 1 hour (with WAL archiving)	Maximum data loss acceptable
RTO (Recovery Time Objective)	4 hours	Maximum time to restore service on a new server
Backup retention	7 daily, 4 weekly, 12 monthly	Rolling retention
Restore test	Monthly	A backup is only valid once restored successfully

2. What must be protected

Asset	Location	Contains	Criticality
PostgreSQL	Volume `proctor-prod_postgres_data`	Users, roles, complaints, history, notifications, face embeddings, audit logs, Flyway history	Critical
MinIO	Volume `proctor-prod_minio_data`	Complaint attachments and evidence	Critical
Secrets file	`docker/.env` or `backend/.env.production`	All passwords and keys, domains, version	Critical
Redis	Volume `proctor-prod_redis_data`	Rate-limit buckets, transient streams, progress	Low (rebuilt automatically)
Caddy data	Volumes `caddy_data`, `caddy_config`	TLS certificates and ACME account	Medium (re-issued automatically, but rate-limited)
Temporary files	Volumes `user_storage`, `face_storage`	In-flight import files	Low
Edge configuration	`edge/Caddyfile` next to the setup script	Generated proxy configuration	Low (regenerated by setup)

> **CAUTION**
> The secrets file is as important as the database. The database roles' passwords, the JWT key and the key that encrypts FCM credentials are stored only there. Without it, a restored database cannot be opened by the services until all passwords are reset.

3. Backup procedure

3.1 Variables

Published-image deployments (run inside `docker/`):

```bash
PROD="docker compose -f docker-compose.yaml --env-file .env"
```

Source deployments (run in the repository root):

```bash
PROD="docker compose -f backend/docker-compose.prod.yaml --env-file backend/.env.production"
```

3.2 Database (logical, consistent, online)

```bash
mkdir -p /var/backups/proctorai && cd /var/backups/proctorai
$PROD exec -T postgres sh -c 'pg_dump -U "$POSTGRES_USER" -Fc "$POSTGRES_DB"' > proctor-$(date +%F).dump
$PROD exec -T postgres sh -c 'pg_dumpall -U "$POSTGRES_USER" --roles-only' > roles-$(date +%F).sql
```

`pg_dump -Fc` produces a compressed, consistent snapshot while the system is running. The roles file documents database roles; passwords are re-applied from the env file during restore.

3.3 Object storage

```bash
docker run --rm -v proctor-prod_minio_data:/data:ro -v "$PWD":/backup alpine \
  tar czf /backup/minio-$(date +%F).tgz -C /data .
```

For large stores, prefer incremental mirroring with the MinIO client to an off-site S3 bucket:

```bash
docker run --rm --network proctor-prod_default --env-file <env-file> minio/mc sh -c \
  'mc alias set src http://minio:9000 "$MINIO_ROOT_USER" "$MINIO_ROOT_PASSWORD" && \
   mc alias set dst https://s3.backup-provider.example "$BACKUP_KEY" "$BACKUP_SECRET" && \
   mc mirror --overwrite src/$S3_BUCKET dst/proctorai-files'
```

3.4 Secrets file

```bash
gpg --symmetric --cipher-algo AES256 -o env-$(date +%F).gpg docker/.env
```

Store the encrypted file and its passphrase in separate places (for example the passphrase in the organisation's password manager).

3.5 Certificates (optional)

```bash
docker run --rm -v proctor-prod_caddy_data:/data:ro -v "$PWD":/backup alpine \
  tar czf /backup/caddy-$(date +%F).tgz -C /data .
```

4. Automation

Create `/usr/local/sbin/proctorai-backup.sh` (owned by root, mode `700`):

```bash
#!/usr/bin/env bash
set -euo pipefail
cd /opt/proctorai/docker
PROD="docker compose -f docker-compose.yaml --env-file .env"
DEST=/var/backups/proctorai
STAMP=$(date +%F-%H%M)
mkdir -p "$DEST"
$PROD exec -T postgres sh -c 'pg_dump -U "$POSTGRES_USER" -Fc "$POSTGRES_DB"' >
"$DEST/proctor-$STAMP.dump"
docker run --rm -v proctor-prod_minio_data:/data:ro -v "$DEST":/backup alpine \
  tar czf "/backup/minio-$STAMP.tgz" -C /data .
find "$DEST" -type f -mtime +7 -name '*.dump' -delete
find "$DEST" -type f -mtime +7 -name '*.tgz' -delete
rclone copy "$DEST" offsite:proctorai-backups --max-age 25h
```

Schedule it daily at 02:30:

```bash
echo '30 2 * * * root /usr/local/sbin/proctorai-backup.sh >> /var/log/proctorai-backup.log 2>&1' \
  | sudo tee /etc/cron.d/proctorai-backup
```

Follow the 3-2-1 rule: three copies, on two kinds of media, one off-site. Encrypt off-site copies (for example with `rclone crypt`).

5. Restore procedures

5.1 Restore the database into a running stack

Use this to recover from data corruption or accidental deletion on the same server.

```bash
$PROD stop api-gateway auth-service user-service department-service role-service \
  complaint-service storage-service custom-field-service notification-service face-service monitor-service
$PROD exec -T postgres sh -c 'pg_restore -U "$POSTGRES_USER" -d "$POSTGRES_DB" --clean --if-exists' \
  < proctor-YYYY-MM-DD.dump
$PROD up -d
```

Stopping the services first prevents writes during the restore. `db-migrator` runs again on `up` and applies any newer migrations.

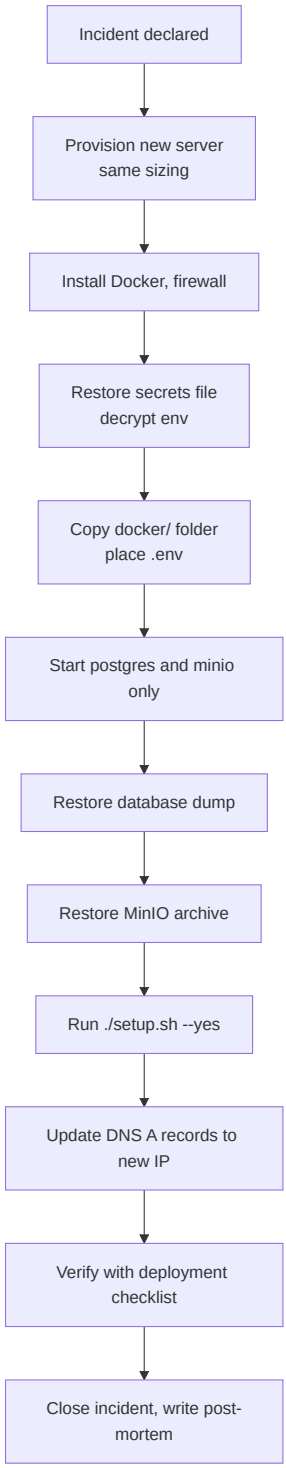
5.2 Restore object storage

```bash
$PROD stop storage-service minio
docker run --rm -v proctor-prod_minio_data:/data -v "$PWD":/backup alpine \
  sh -c 'rm -rf /data/* /data/.minio.sys && tar xzf /backup/minio-YYYY-MM-DD.tgz -C /data'
$PROD up -d
```

5.3 Restore a single complaint's files

Mount the archive in a temporary container, locate the object under `proctor-files/` by the key stored in `complaint_files.file_url`, and copy it back with `mc cp`.

6. Disaster recovery: full server loss



6.1 Step by step

1. **Provision** a server with the same or larger sizing (PAI-DEP-006, Planning).
2. **Prepare** Docker and the firewall (PAI-DEP-006, Server preparation).
3. **Restore secrets:** `gpg -d env-YYYY-MM-DD.gpg > docker/.env && chmod 600 docker/.env`.

4. **Start data services and create the database roles:**

```bash
cd docker
$PROD up -d postgres minio
$PROD run --rm db-migrator
```

   The migrator creates the `authenticator`, `proctor` and `auth_owner` roles with the passwords from the env file, so the restored grants resolve.

5. **Restore the database** over the freshly migrated schema:

```bash
$PROD exec -T postgres sh -c 'pg_restore -U "$POSTGRES_USER" -d "$POSTGRES_DB" --clean --if-exists' < proctor-YYYY-MM-DD.dump
```

6. **Restore files** with the MinIO procedure above.
7. **Start everything:** `./setup.sh --yes`. The migrator re-applies role passwords from the env file and any pending migrations.
8. **Repoint DNS** A records to the new IP if it changed. Certificates are issued again automatically (or restore `caddy_data` to avoid re-issuance).
9. **Verify** with the post-deployment checklist in PAI-DEP-006.
10. **Communicate** recovery and the data-loss window (time of the last backup) to stakeholders.

6.2 Recovery time budget

Step	Estimated time
Provision and prepare server	30 min
Restore secrets and start data services	10 min
Database restore (10 GB)	20–40 min
File restore (100 GB)	60–120 min
Start stack, DNS, verification	30 min
Total	**≈ 2.5–4 hours**

7. Point-in-time recovery (optional)

To reach an RPO of minutes, enable continuous WAL archiving with a tool such as pgBackRest or WAL-G against an off-site S3 bucket. This requires a custom PostgreSQL configuration (`archive_mode=on`, `archive_command`) mounted into the `postgres` service and a base backup schedule. Document the procedure separately when adopted and test PITR quarterly.

8. Restore testing

Test	Frequency	Pass criteria
Database dump restores into a scratch container	Monthly	`pg_restore` exits 0; row counts match production within the backup window
Full DR drill on a staging server	Twice a year	Service usable within RTO; sign-in, complaint view and file download succeed
Secrets file decrypt test	Quarterly	File decrypts with the stored passphrase

Scratch restore check:

```bash
docker run -d --name pg-restore-test -e POSTGRES_PASSWORD=test pgvector/pgvector:pg16
sleep 10
docker exec -i pg-restore-test sh -c 'createdb -U postgres proctor && pg_restore -U postgres -d proctor --no-owner' < proctor-YYYY-MM-DD.dump
docker exec pg-restore-test psql -U postgres -d proctor -c 'select count(*) from complaints;'
docker rm -f pg-restore-test
```

Role-related warnings during a scratch restore with `--no-owner` are expected.

9. Backup log

Date	Type	Size	Location	Verified by	Restore tested

DOCUMENT 09

OPERATIONS · PAI-OPS-009

Monitoring & Incident Runbook

Health signals, metrics, alerting rules, log analysis and step-by-step responses to common failures.

1. Observability overview
2. Built-in system monitor
3. Prometheus and Grafana (recommended)
4. Service level indicators and alert rules
5. Logs
6. Severity levels
7. Incident process
8. Playbooks
9. Routine operations calendar
10. Post-mortem template

1. Observability overview

Signal	Source	Access
Container health	Docker health checks on `/actuator/health`	`./setup.sh status`
Live system view	`monitor-service` (Admin → System)	Web app, requires `system_monitor:read:all`
Metrics	`/actuator/prometheus` on every backend service	Inside the Docker network
Logs	JSON logs per container, 10 MB × 5 rotation	`./setup.sh logs [service]`
Request tracing	Trace ID on every log line, propagated by the gateway	Search logs by trace ID
Edge access logs	Caddy JSON access log	`./setup.sh logs edge`
Service registry	Eureka dashboard	`https://registry.<domain>` (Basic auth) or SSH tunnel

2. Built-in system monitor

The System page shows, refreshed every few seconds over SSE:

Panel	Key indicators
Host	CPU total and per core, load average, memory and swap, disk usage per mount, disk and network throughput
Services	Each instance: UP/DOWN, registered, uptime, CPU, heap, RSS, threads, requests/s, errors/s, latency, GC rate, DB pool usage
Database	Connections (active, idle, idle in transaction, waiting), longest query, transactions/s, rollback rate, cache hit ratio, deadlocks, temp bytes, largest tables
Redis	Clients, memory vs max, ops/s, hit ratio, keys, evictions
Storage	Files, bytes, pending uploads, evidence files
Issues	Automatically detected problems in the latest sample

Services report samples to Redis every 2 seconds with a 10-second TTL. A service that stops reporting disappears from the panel within 10 seconds.

3. Prometheus and Grafana (recommended)

Run Prometheus on the stack's network to collect metrics without exposing them publicly.

```yaml
scrape_configs:
  - job_name: proctorai
    metrics_path: /actuator/prometheus
    static_configs:
      - targets:
          - api-gateway:7050
          - auth-service:7051
          - user-service:7052
          - department-service:7053
          - role-service:7054
          - complaint-service:7055
          - storage-service:7056
          - custom-field-service:7057
          - notification-service:7058
          - face-service:7059
          - monitor-service:7060
```

```bash
docker run -d --name prometheus --network proctor-prod_default \
  -v /opt/monitoring/prometheus.yml:/etc/prometheus/prometheus.yml:ro \
  -p 127.0.0.1:9090:9090 prom/prometheus
```

Add node-exporter and cAdvisor for host and container metrics, and Grafana for dashboards (JVM Micrometer and Spring Boot dashboards work out of the box).

4. Service level indicators and alert rules

SLI / condition	Threshold	Severity	Action
Any container unhealthy	> 2 minutes	Critical	INC-01
Edge returns 5xx	> 2 % of requests for 5 min	Critical	INC-02
Gateway p95 latency	> 1 s for 10 min	Warning	INC-03
Login 429 rate	> 50/min	Warning	INC-07 (possible attack)
JVM heap used	> 90 % of max for 10 min	Warning	INC-04
Container restarts	> 3 in 15 min	Critical	INC-04
PostgreSQL connections	> 80 % of `max_connections`	Warning	INC-05
Disk usage (any mount)	> 80 % warning, > 90 % critical	Warning/Critical	INC-06
Certificate expiry	< 14 days	Warning	INC-08
Backup job	Not successful in 26 h	Critical	PAI-BDR-008

SLI / condition	Threshold	Severity	Action
FCM failures	> 10 % of push deliveries for 1 h	Warning	INC-09

Example Prometheus rules:

```yaml
groups:
  - name: proctorai
    rules:
      - alert: ServiceDown
        expr: up{job="proctorai"} == 0
        for: 2m
        labels: { severity: critical }
      - alert: HighErrorRate
        expr: sum(rate(http_server_requests_seconds_count{status=~"5.."}[5m])) by (instance)
              / sum(rate(http_server_requests_seconds_count[5m])) by (instance) > 0.02
        for: 5m
        labels: { severity: critical }
      - alert: HeapPressure
        expr: sum(jvm_memory_used_bytes{area="heap"}) by (instance)
              / sum(jvm_memory_max_bytes{area="heap"}) by (instance) > 0.9
        for: 10m
        labels: { severity: warning }
```

External uptime checks (for example every minute against `https://app.<domain>/` and `https://app.<domain>/api/auth/refresh` expecting `401`) catch DNS, TLS and edge failures that internal metrics miss.

5. Logs

```bash
./setup.sh logs                       # all services
./setup.sh logs complaint-service     # one service
docker logs --since 30m proctor-prod-auth-service-1 2>&1 | jq -r 'select(.level=="ERROR")'
```

To follow a request across services, take the trace ID from the first log line of the failing call and search every service:

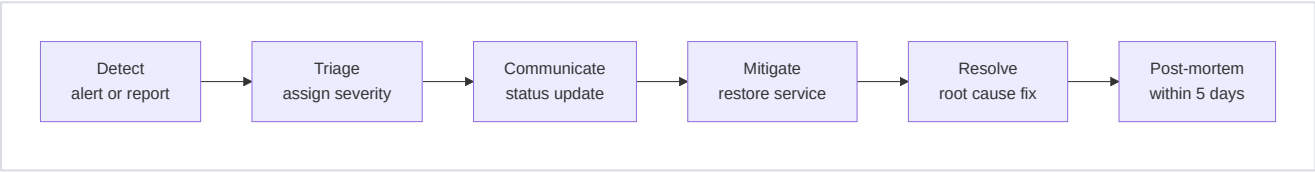
```bash
for c in $(docker ps --format '{{.Names}}' | grep proctor-prod); do
  docker logs --since 1h "$c" 2>&1 | grep -F "<trace-id>" | sed "s/^/[$c] /"
done
```

For long-term retention ship logs to Loki, Elasticsearch or a cloud log service with a Docker logging driver or Promtail.

6. Severity levels

Severity	Definition	Response target	Examples
SEV-1	Platform unavailable or data at risk	15 min, continuous until resolved	Database down, TLS failure, data corruption, breach
SEV-2	Major feature unavailable	1 hour	Complaints cannot be filed, uploads fail, sign-in fails for many
SEV-3	Degraded or partial	Next business day	Face recognition slow, push notifications failing
SEV-4	Cosmetic or single user	Planned	UI glitch, one user's import row error

7. Incident process



1. **Acknowledge** the alert and open an incident record (time, reporter, symptom).
2. **Triage:** check `./setup.sh status`, the System page and recent changes (deploys, config edits).
3. **Communicate** to stakeholders for SEV-1/2 every 30 minutes.
4. **Mitigate** first (restart, roll back, scale), investigate second.
5. **Preserve evidence** before destructive actions: `docker logs` output, database state.
6. **Resolve** and verify with the post-deployment checklist (PAI-DEP-006).
7. **Post-mortem:** blameless write-up with timeline, impact, root cause, action items.

8. Playbooks

8.1 INC-01: a service is unhealthy

```bash
./setup.sh status
./setup.sh logs <service>
docker inspect --format '{{json .State.Health}}' proctor-prod-<service>-1 | jq
```

Finding	Fix
Cannot connect to PostgreSQL / Redis	Check those containers first (INC-05); services recover automatically
`Required variable … is missing`	Fix the env file; rerun setup

Finding	Fix
`DatabaseRoleGuard` violation	Service connected with a privileged role; check `APP_USER_NAME` / `AUTHENTICATOR_USER_NAME`
Not registered in Eureka	Check `discovery-server` health and `DISCOVERY_PASSWORD` consistency
`OutOfMemoryError` / repeated restarts	INC-04

Restart a single service: `$PROD restart <service>` (see PAI-BDR-008 for `$PROD`).

8.2 INC-02: users see errors (5xx) or the site is down

1. `curl -I https://app.<domain>`. No response means edge or DNS: check `./setup.sh logs edge` and `./setup.sh dns`.
2. `502` / `504` from the edge means frontend or gateway: check `frontend` and `api-gateway` health.
3. API errors with `500`: find the trace ID in gateway logs and follow it into the service.
4. If a deployment happened in the last hour, roll back (PAI-REL-010).

8.3 INC-03: slow responses

Check	Command / place
Host CPU or memory saturated	System page → Host; `top`, `free -m`
Long-running queries	System page → Database → longest query; `select pid, now()-query_start, query from pg_stat_activity where state <> 'idle' order by 2 desc;`
DB pool exhausted	System page → Services → DB pool pending
Face recognition load	CPU on `face-service`; limit concurrent live recognition

Cancel a runaway query: `select pg_cancel_backend(<pid>);`

8.4 INC-04: memory pressure or restart loop

1. `docker stats --no-stream` to see usage against limits.
2. Raise the limit for the affected service in the compose file (`deploy.resources.limits.memory`) and rerun setup, or add RAM to the host.
3. For `face-service`, very large images or many concurrent recognitions increase memory; keep `FACE_MAX_IMAGE_BYTES` at 8 MB.

8.5 INC-05: database problems

```bash
$PROD exec postgres pg_isready
$PROD exec postgres psql -U "$POSTGRES_USER" -d "$POSTGRES_DB" -c "select count(*), state from pg_stat_activity group by state;"
```

Symptom	Fix
Too many connections	Find idle-in-transaction sessions and terminate; check for a stuck service
Disk full	INC-06; PostgreSQL stops accepting writes when the disk is full
Corruption or accidental deletion	Stop services and restore (PAI-BDR-008)
Failed migration after upgrade	Read `db-migrator` logs; roll back the image version (PAI-REL-010)

8.6 INC-06: disk space

```bash
df -h
docker system df
sudo du -sh /var/lib/docker/volumes/* | sort -h | tail
```

Free space safely: `docker image prune -a` (removes unused images), clean old backups, archive old resolved complaints' files. Never delete files inside volumes by hand.

8.7 INC-07: brute-force or suspicious sign-ins

1. Inspect the edge access log for many `POST /api/auth/login` from one IP: `./setup.sh logs edge | grep auth/login`.
2. The gateway rate limit and account lockout already slow attacks. For sustained attacks, block the source at the firewall (`sudo ufw deny from <ip>`).
3. If an account may be compromised: disable it (this revokes its sessions), reset the password, and review its recent actions in complaint history and the face audit log.

8.8 INC-08: TLS certificate problems

1. `./setup.sh logs edge | grep -i -E "acme|certificate|error"`.
2. Confirm ports 80 and 443 are open and DNS A records point to the server (`./setup.sh dns`).
3. Remove wrong AAAA records.
4. If Let's Encrypt rate limits are hit, Caddy falls back to ZeroSSL; wait or restore `caddy_data` from backup.

8.9 INC-09: notifications not delivered

Channel	Checks
In-app (real time)	User preference `realtimeEnabled`; Redis healthy; browser shows SSE stream open; corporate proxies buffering SSE
Push (FCM)	Admin → Notifications → Integrations → Test; check delivery log `detail`; device registered and active; user preference `pushEnabled`; muted types

8.10 INC-10: suspected security breach

1. Declare SEV-1. Limit access: block suspicious IPs, disable affected accounts.
2. Preserve evidence: export container logs, take a database dump, snapshot the disk.

3. Rotate secrets (PAI-SEC-005, Secret management). Rotating `JWT_SECRET` and revoking refresh tokens signs out every user:

```sql
update refresh_tokens set revoked_at = now() where revoked_at is null;
```

4. Assess data exposure using the audit tables; notify data-protection authorities and affected users as required by law.
5. Patch the root cause, then restore normal operation and complete a post-mortem.

9. Routine operations calendar

Frequency	Task
Daily	Check alerts and backup log; glance at System page issues
Weekly	Review disk growth, error rates, failed sign-ins; `docker image prune`
Monthly	Restore test (PAI-BDR-008); review roles with face and admin permissions; apply OS updates
Quarterly	Update to the latest ProctorAI patch release; rotate operator credentials; review alert thresholds
Yearly	DR drill; penetration test; review this runbook

10. Post-mortem template

Section	Content
Summary	What happened, impact in one paragraph
Timeline	Detection, key actions, resolution (UTC)
Impact	Users affected, duration, data loss
Root cause	Technical and process causes
What went well / badly	Honest assessment
Action items	Owner and due date for each fix

DOCUMENT 10

RELEASE MANAGEMENT · PAI-REL-010

Release, Versioning & Rollback Guide

How ProctorAI versions are numbered, built, published, upgraded and rolled back safely.

1. Versioning policy
2. Branching model
3. Release checklist
4. Changelog format
5. Upgrade procedure (production)
6. Rollback
7. Hotfix process
8. Native app releases
9. Support policy

1. Versioning policy

ProctorAI follows Semantic Versioning `MAJOR.MINOR.PATCH`.

Part	Increment when	Examples
MAJOR	Breaking API change, removal of a feature, migration that cannot coexist with the previous version	Endpoint removed, response field renamed
MINOR	New backward-compatible feature, new endpoint, additive migration	New custom field type, new report
PATCH	Bug or security fix without new features	Fix in a trigger, dependency update

One version number covers the whole platform: all backend images, the frontend image and the native apps share it. The current release is **1.1.0**.

1.1 Where the version lives

Location	Purpose
Git tag `vX.Y.Z`	Source of truth for a release
Image tags `vishaltyagi807/proctor-ai:<service>-X.Y.Z`	Published images, one per service
Image tags `…:<service>-latest`	Moving tag for the newest release
`PROCTOR_VERSION` default in `docker/docker-compose.yaml`	Version pulled by fresh installs
`frontend/package.json` → `version`	App version shown in the UI and native packages
`frontend/src-tauri/tauri.conf.json` → `version`	Points to `../package.json`, so native apps inherit the frontend version

> **NOTE**
> At the time of writing `frontend/package.json` reports `1.0.0` while the published images are `1.1.0`. Align both in the next release so the UI, native apps and server report the same version.

2. Branching model

Branch	Purpose
`main`	Always releasable; every release is tagged here
`v2` / `development-*`	Integration branches for the next minor or major version
`feature/<name>`	Short-lived feature work, merged by pull request

Branch	Purpose
`hotfix/<version>`	Branched from a release tag for urgent patches

All changes enter `main` through reviewed pull requests with a green build.

3. Release checklist

3.1 Prepare

#	Step	Done
1	All planned pull requests merged; `main` builds green	☐
2	`cd backend && ./gradlew build` passes locally (needs Docker)	☐
3	`cd frontend && npm ci && npm run lint && npm run build` passes	☐
4	New migrations reviewed against the checklist in PAI-DB-004	☐
5	Upgrade tested on a copy of production data (restore latest backup to staging)	☐
6	Version bumped in `frontend/package.json` (native apps follow it); `PROCTOR_VERSION` default updated in `docker/docker-compose.yaml`	☐
7	`CHANGELOG.md` updated (format below)	☐

3.2 Build and publish images

```bash
REPO=vishaltyagi807/proctor-ai
VERSION=1.2.0
for service in db-migrator discovery-server api-gateway auth-service user-service department-service role-service \
               complaint-service storage-service custom-field-service notification-service monitor-service; do
  docker build --target service --build-arg SERVICE=$service --build-arg GRADLE_FLAGS=--no-build-cache \
    -t $REPO:$service-$VERSION -t $REPO:$service-latest backend
done
docker build --target face-service --build-arg SERVICE=face-service --build-arg GRADLE_FLAGS=--no-build-cache \
  -t $REPO:face-service-$VERSION -t $REPO:face-service-latest backend
docker build --target prod -t $REPO:frontend-$VERSION -t $REPO:frontend-latest frontend
docker push --all-tags $REPO
```

The CI/CD design (PAI-CICD-011) automates these steps on a tag push.

3.3 Tag and announce

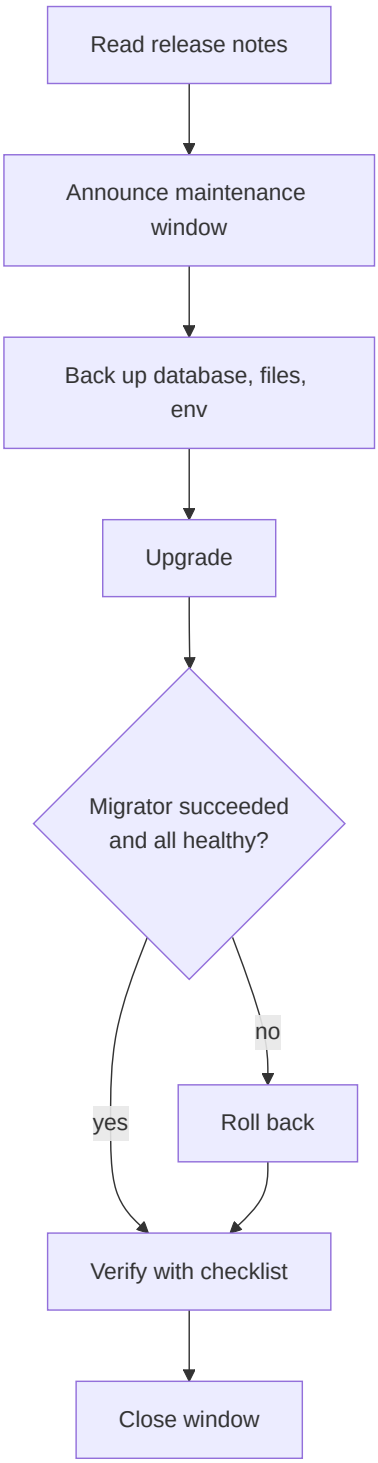
1. Tag: `git tag -a v1.2.0 -m "ProctorAI 1.2.0"` and push the tag.
2. Create a GitHub release with the changelog section and upgrade notes.
3. Attach native app installers (desktop, APK/AAB, IPA via store) built with `VITE_SERVER_URL` where appropriate.

4. Changelog format

`CHANGELOG.md` follows the *Keep a Changelog* convention:

```markdown
## [1.2.0] - 2026-12-01
### Added
- Complaint tags with filtering (`tags=in.(…)`).
### Changed
- Default face match threshold documented per deployment.
### Fixed
- Notification stream reconnect after token refresh.
### Security
- Updated Spring Boot to 3.5.x for CVE-YYYY-NNNN.
### Upgrade notes
- Migration V2__complaint_tags.sql adds a table; no downtime expected.
```

5. Upgrade procedure (production)



1. Read the release notes, especially **Upgrade notes** and **Security**.
2. Take a fresh backup (PAI-BDR-008): database dump, MinIO archive, env file.
3. Upgrade:
   - Published images: `cd docker && ./setup.sh update 1.2.0`

- From source: update the code (`git fetch --tags && git checkout v1.2.0`), then `./setup.sh prod --yes`

4. Watch `./setup.sh logs db-migrator` until it exits successfully, then `./setup.sh status`.
5. Run the post-deployment verification in PAI-DEP-006 (at least items 1–10).
6. Record the upgrade (version, time, operator, result).

Expected downtime is the time to restart the containers plus migrations, typically 1–3 minutes.

6. Rollback

6.1 Decision guide

Situation	Rollback type
New version misbehaves, no migration in the release	**Image rollback**
Release contained only additive migrations (new tables/columns)	**Image rollback** (old code ignores new structures)
Release contained destructive or incompatible migrations, or data was corrupted	**Full rollback** (images + database restore)
Migration failed during upgrade	**Image rollback**; failed migration is rolled back by PostgreSQL's transactional DDL in most cases. Check `flyway_schema_history`

6.2 Image rollback

```bash
cd docker
./setup.sh update 1.1.0
```

From source: `git checkout v1.1.0 && ./setup.sh prod --yes`.

> **WARNING**
> Flyway's default settings ignore *future* migrations, so an older `db-migrator` starts normally against a database that a newer release has already migrated. That does not make the old services compatible with the new schema. If the newer release changed or removed structures the old code uses, perform a full rollback.

6.3 Full rollback

1. Stop the application services (keep `postgres` and `minio` running).
2. Restore the pre-upgrade database dump and, if needed, the MinIO archive (PAI-BDR-008, Restore procedures).
3. Switch to the previous version (`./setup.sh update <previous>`).

4. Verify and communicate the data-loss window (changes made after the backup are lost).

6.4 Expand-migrate-contract

To keep image rollbacks possible, ship schema changes in stages:

Release	Database	Application
N	Add new column/table (expand)	Writes both old and new
N+1	Backfill data	Reads new, still writes both
N+2	Remove old column (contract)	Uses only new

7. Hotfix process

1. Branch from the release tag: `git checkout -b hotfix/1.1.1 v1.1.0`.
2. Apply the minimal fix with a test; open a pull request.
3. After merge, tag `v1.1.1`, build and publish images, update the changelog.
4. Merge the fix forward into active development branches.
5. Notify operators with the CVE or issue reference and urgency.

8. Native app releases

Platform	Artifact	Distribution
Windows	`.msi`, `.exe` (NSIS)	Direct download; sign with a code-signing certificate
macOS	`.dmg`, `.app` (universal)	Direct download; sign and notarise
Linux	`.deb`, `.rpm`, `.AppImage`	Direct download or repository
Android	`.apk`, `.aab`	Direct install or Google Play (signed with the upload keystore)
iOS	`.ipa`	TestFlight and the App Store

Native apps tolerate server upgrades within the same major version. Release native apps before or together with any server major upgrade.

9. Support policy

Version line	Status	Receives
Latest minor (1.1.x)	Active	Features, fixes, security patches
Previous minor (1.0.x)	Maintenance (6 months after next minor)	Security patches

Version line	Status	Receives
Older	End of life	Nothing; upgrade required

DOCUMENT 11

ENGINEERING PROCESS · PAI-CICD-011

CI/CD Pipeline Design

Proposed continuous integration, image publishing and deployment pipeline for ProctorAI.

1. Context
2. Pipeline overview
3. Stages
4. Workflow: continuous integration
5. Workflow: release
6. Workflow: deployment
7. Secrets and environments
8. Quality gates
9. Branch protection
10. Implementation plan

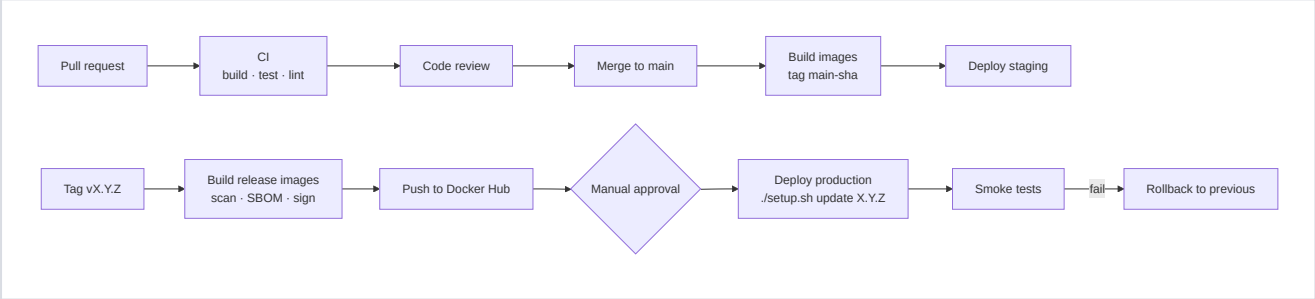
1. Context

Version 1.1.0 is built and published manually (PAI-REL-010). The repository has no automated pipeline yet. This document defines the target pipeline, with ready-to-use GitHub Actions workflows, so that every change is tested and every release is reproducible, scanned and traceable.

1.1 Goals

Goal	Measure
Every pull request is verified	Backend tests, frontend lint and build, migration tests pass before merge
Releases are reproducible	Images built only by CI from a Git tag
Supply chain is visible	SBOM and vulnerability scan for every image
Deployments are safe	Staging first, manual approval for production, automatic health verification
Fast feedback	Pull request pipeline under 15 minutes

2. Pipeline overview



3. Stages

Stage	Trigger	Jobs	Blocking
1. Verify	Pull request, push to any branch	Backend `./gradlew build` (unit + Testcontainers), frontend `npm ci`, `lint`, `build`, shell check of setup scripts	Yes
2. Security checks	Pull request	Dependency review, CodeQL (Java, TypeScript), secret scanning	Yes for high severity
3. Snapshot images	Push to `main`	Build all images, tag `main-<sha>`, push to a private registry or GHCR	No
4. Staging deploy	After stage 3	SSH to staging, set `PROCTOR_VERSION=main-<sha>`, `./setup.sh update`, smoke tests	No
5. Release	Tag `v*.*.*`	Build images with version tags, Trivy scan, SBOM (Syft), sign (Cosign), push to Docker Hub, create GitHub release	Yes

Stage	Trigger	Jobs	Blocking
6. Production deploy	Manual approval on the release	SSH to production, backup, `./setup.sh update X.Y.Z`, smoke tests, automatic rollback on failure	Yes
7. Native apps	Tag `v*.*.*`	Matrix build on Windows, macOS, Linux, Android; attach artifacts to the release	No

4. Workflow: continuous integration

`.github/workflows/ci.yml`

YAML

```yaml
name: CI
on:
  pull_request:
  push:
    branches: [main, v2]
permissions:
  contents: read
concurrency:
  group: ci-${{ github.ref }}
  cancel-in-progress: true
jobs:
  backend:
    runs-on: ubuntu-24.04
    timeout-minutes: 30
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-java@v4
        with:
          distribution: temurin
          java-version: "21"
      - uses: gradle/actions/setup-gradle@v4
      - name: Build and test
        working-directory: backend
        run: ./gradlew build --no-daemon
      - name: Test reports
        if: always()
        uses: actions/upload-artifact@v4
        with:
          name: backend-test-reports
          path: backend/**/build/reports/tests/
  frontend:
    runs-on: ubuntu-24.04
    timeout-minutes: 15
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: "22"
          cache: npm
          cache-dependency-path: frontend/package-lock.json
      - working-directory: frontend
        run: |
          npm ci
          npm run lint
          npm run build
  scripts:
```

```
    runs-on: ubuntu-24.04
    steps:
      - uses: actions/checkout@v4
      - run: shellcheck setup.sh docker/setup.sh clean-up.sh native-doctor.sh
```

Testcontainers uses the Docker engine that GitHub-hosted Ubuntu runners provide, so the integration tests run unchanged.

5. Workflow: release

`.github/workflows/release.yml`

YAML

```yaml
name: Release
on:
  push:
    tags: ["v*.*.*"]
permissions:
  contents: write
  id-token: write
env:
  REPO: vishaltyagi807/proctor-ai
jobs:
  images:
    runs-on: ubuntu-24.04
    timeout-minutes: 90
    strategy:
      fail-fast: true
      matrix:
        include:
          - { service: db-migrator, target: service, context: backend }
          - { service: discovery-server, target: service, context: backend }
          - { service: api-gateway, target: service, context: backend }
          - { service: auth-service, target: service, context: backend }
          - { service: user-service, target: service, context: backend }
          - { service: department-service, target: service, context: backend }
          - { service: role-service, target: service, context: backend }
          - { service: complaint-service, target: service, context: backend }
          - { service: storage-service, target: service, context: backend }
          - { service: custom-field-service, target: service, context: backend }
          - { service: notification-service, target: service, context: backend }
          - { service: monitor-service, target: service, context: backend }
          - { service: face-service, target: face-service, context: backend }
          - { service: frontend, target: prod, context: frontend }
    steps:
      - uses: actions/checkout@v4
      - name: Fetch face models
        if: matrix.service == 'face-service'
        run: ./setup.sh models
      - uses: docker/setup-buildx-action@v3
      - uses: docker/login-action@v3
        with:
          username: ${{ secrets.DOCKERHUB_USERNAME }}
          password: ${{ secrets.DOCKERHUB_TOKEN }}
      - id: version
        run: echo "v=${GITHUB_REF_NAME#v}" >> "$GITHUB_OUTPUT"
      - uses: docker/build-push-action@v6
        with:
          context: ${{ matrix.context }}
```

```
          target: ${{ matrix.target }}
          build-args: |
            SERVICE=${{ matrix.service }}
            GRADLE_FLAGS=--no-build-cache
          tags: |
            ${{ env.REPO }}:${{ matrix.service }}-${{ steps.version.outputs.v }}
            ${{ env.REPO }}:${{ matrix.service }}-latest
          push: true
          sbom: true
          provenance: mode=max
          cache-from: type=gha,scope=${{ matrix.service }}
          cache-to: type=gha,mode=max,scope=${{ matrix.service }}
      - uses: aquasecurity/trivy-action@0.28.0
        with:
          image-ref: ${{ env.REPO }}:${{ matrix.service }}-${{ steps.version.outputs.v }}
          severity: CRITICAL,HIGH
          exit-code: "1"
          ignore-unfixed: true
  github-release:
    needs: images
    runs-on: ubuntu-24.04
    steps:
      - uses: actions/checkout@v4
      - uses: softprops/action-gh-release@v2
        with:
          generate_release_notes: true
```

NOTE

The face-service image needs the ONNX models in `backend/face-service/models/`. `./setup.sh` downloads and verifies them during a normal run; the release job must run the same download step (shown as `./setup.sh models`, which should be added as a small sub-command that only fetches models) or restore them from a cache.

6. Workflow: deployment

`.github/workflows/deploy.yml`

```yaml
name: Deploy
on:
  workflow_dispatch:
    inputs:
      version:
        description: Version to deploy, for example 1.2.0
        required: true
      environment:
        type: choice
        options: [staging, production]
jobs:
  deploy:
    runs-on: ubuntu-24.04
    environment: ${{ inputs.environment }}
    steps:
      - name: Backup and upgrade
        uses: appleboy/ssh-action@v1
        with:
          host: ${{ secrets.DEPLOY_HOST }}
          username: ${{ secrets.DEPLOY_USER }}
          key: ${{ secrets.DEPLOY_SSH_KEY }}
          script: |
            set -e
            sudo /usr/local/sbin/proctorai-backup.sh
            cd /opt/proctorai/docker
            echo "$(grep '^PROCTOR_VERSION=' .env || echo none)" > /tmp/proctorai-previous
            ./setup.sh update ${{ inputs.version }} --yes
      - name: Smoke test
        run: |
          for i in $(seq 1 30); do
            code=$(curl -s -o /dev/null -w '%{http_code}' -X POST "${{ vars.APP_URL
}}/api/auth/refresh")
            [ "$code" = "401" ] && exit 0
            sleep 10
          done
          exit 1
```

The `production` GitHub environment requires approval from a maintainer and restricts the deploy secrets to protected tags. On smoke-test failure, an operator runs the rollback in PAI-REL-010 (or a follow-up job runs `./setup.sh update <previous>`).

7. Secrets and environments

Secret / variable	Scope	Purpose
`DOCKERHUB_USERNAME`, `DOCKERHUB_TOKEN`	Repository	Push images (token with write scope only)
`DEPLOY_HOST`, `DEPLOY_USER`, `DEPLOY_SSH_KEY`	Environment (`staging`, `production`)	SSH deployment; key restricted to the deploy user
`APP_URL`	Environment variable	Smoke test target
`APPLE_*`, `ANDROID_KEYSTORE_*`, `WINDOWS_CERT_*`	Environment `native-release`	Signing native apps

Application secrets (`.env`) never enter CI; they stay on the servers.

8. Quality gates

Gate	Rule
Tests	All backend tests pass, including RLS and migration tests
Lint	No ESLint errors
Build	Frontend production build succeeds
Vulnerabilities	No unfixed CRITICAL or HIGH in images (Trivy) or dependencies (dependency review)
Review	At least one approving review; branch up to date with `main`
Migrations	Changes under `db/migration/` require review from a database owner (CODEOWNERS)

9. Branch protection

Setting	`main`
Require pull request	Yes, 1 approval
Required checks	`backend`, `frontend`, `scripts`, CodeQL
Require linear history	Yes
Restrict force pushes and deletion	Yes
Require signed commits	Recommended

10. Implementation plan

Step	Deliverable	Effort
1	Add `ci.yml`; enable branch protection	0.5 day
2	Add a models-only sub-command to `setup.sh`; add `release.yml`; Docker Hub token	1 day
3	Provision staging server; add `deploy.yml` with environments	1 day
4	Enable CodeQL, Dependabot (Gradle, npm, Docker, Actions) and secret scanning	0.5 day
5	Native app matrix builds with signing	2 days
6	Image signing with Cosign and verification on servers	1 day

DOCUMENT 12

ENGINEERING PROCESS · PAI-DEV-012

Contributor & Testing Guide

Developer onboarding, repository layout, coding conventions, test strategy and pull request workflow.

1. Getting started
2. Repository layout
3. Backend conventions
4. Frontend conventions
5. Test strategy
6. Git workflow
7. Documentation

1. Getting started

1.1 Prerequisites

Tool	Version	Needed for
Git	Any recent	Source control
Docker + Compose	25+ / 2.21+	Running the stack and the backend tests
Java (optional locally)	21	IDE support; builds run inside Docker or via the Gradle wrapper
Node.js	20.19+ or 22.12+	Frontend outside Docker, native apps
Rust	1.90+	Native apps (Tauri)
IDE	IntelliJ IDEA (backend), VS Code (frontend)	Lombok and MapStruct annotation processing must be enabled

Minimum machine: 8 GB RAM (16 GB recommended) and 20 GB free disk.

1.2 First run

```bash
git clone https://github.com/vishaltyagi807/Proctor-AI.git
cd Proctor-AI
./setup.sh
```

Service	URL
Web app	`http://localhost:5173`
API gateway	`http://localhost:7050`
Service registry	`http://localhost:8761` (`proctor-discovery` / `DISCOVERY_PASSWORD` from `backend/.env`)
MinIO console	`http://localhost:9201`

Sign in with `admin@college.com` / `admin123` (development only).

1.3 Daily workflow

Task	Command
Frontend change	Save; Vite reloads instantly
Backend change	`./setup.sh` (rebuilds only changed images)
Logs	`./setup.sh logs <service>`
Status	`./setup.sh status`

Task	Command
Stop	`./setup.sh stop`
Fresh database	`./setup.sh purge` then `./setup.sh`
Frontend outside Docker	`docker compose -f backend/docker-compose.yaml stop frontend && cd frontend && npm run dev`
API testing	Import `backend/postman/ProctorAI.postman_collection.json` and the environment file
Sample users	`backend/seed-data/users-bulk-import.csv`

2. Repository layout

Path	Contents
`backend/build.gradle.kts`, `settings.gradle.kts`	Shared Gradle configuration (Java 21, Spring BOMs, Lombok, MapStruct) and module list
`backend/common-*`	Shared libraries (security, persistence, storage, notifications, custom fields, monitoring, DTOs)
`backend/*-service`, `api-gateway`, `discovery-server`	Deployable services
`backend/db-schema`	Flyway migrations
`backend/db-migrator`	Migration runner and database tests
`backend/test-support`	Testcontainers helpers (`TestDatabase`, `TestRedis`)
`backend/Dockerfile`	Multi-stage build for every backend image
`frontend/src/routes`	TanStack Router route definitions
`frontend/src/pages`, `components/<feature>`	Screens and feature components; `components/ui` holds shadcn/ui primitives
`frontend/src/lib`	API client, session, permissions, queries
`frontend/src/platform`	Web, desktop and mobile integration (Tauri plugins)
`frontend/src-tauri`	Native shell (Rust), icons and capabilities
`docker/`	Published-image deployment
`docs/`	This documentation set and its PDF build

3. Backend conventions

3.1 Structure of a service

```
<service>/src/main/java/dev/varshit/proctor/<service>/
├── <Name>Application.java
├── controller/   REST endpoints, validation, HTTP status
├── dto/          request and response records
├── service/      interfaces and *Impl with business rules
├── repository/   interfaces and Sql* implementations
└── config/       service-specific configuration
```

3.2 Rules

Topic	Convention
Language	Java 21; records for DTOs; `var` where it improves clarity
Reactive	Return `Mono` / `Flux` end to end; never call `block()` in request paths
Data access	Explicit SQL through `SqlGateway`; parameters always bound (`:name`), never concatenated
Authorization	Rely on RLS; run request queries inside `SecureTransaction` so `auth.uid` is set. Do not duplicate permission logic in Java unless the database cannot express it
Validation	Bean Validation annotations on request records; mirror important limits in database constraints
Errors	Throw `common-lib` exceptions (`BadRequestException` , `NotFoundException` , ...); never return stack traces
Configuration	`application.yaml` with `${ENV_VAR:default}` ; add new variables to `.env.example` , the compose files and PAI-CFG-007
Notifications	Publish through `NotificationPublisher` ; never write to `notifications` directly from other services
Shared code	Belongs in a `common-*` module only when used by two or more services
Comments	Code should be self-explanatory; comments only where intent is not obvious and the maintainers ask for them
Formatting	Four-space indentation, IntelliJ default Java style, imports without wildcards

3.3 Adding an endpoint

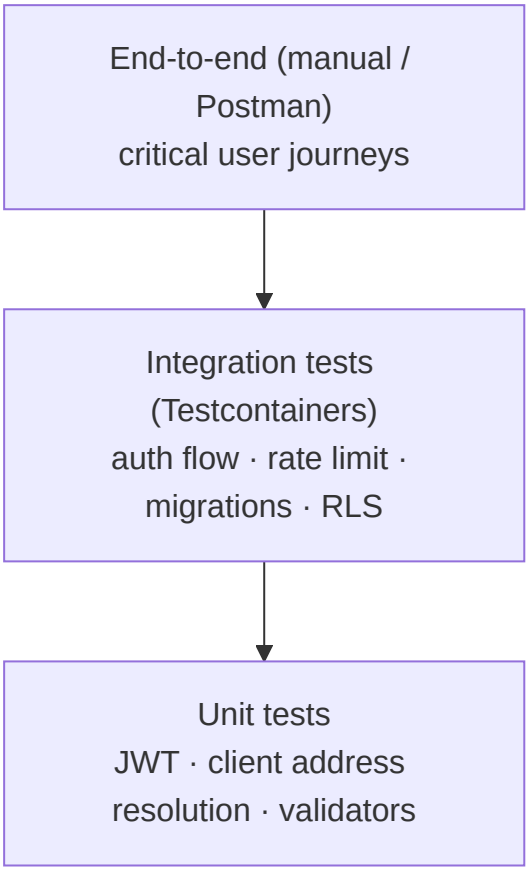
1. Add request/response records in `dto/` with validation.
2. Add the method to the service interface and implementation.
3. Add the repository query; filter and paginate with `PagedQueryRunner` and `FieldSpec` maps.
4. If a new entity or permission is needed, add a migration (enum value, permissions, policies, grants).
5. Add a route to the gateway if the path prefix is new.
6. Update the Postman collection and PAI-API-003.

4. Frontend conventions

Topic	Convention
Language	TypeScript strict mode; no `any` in new code
Routing	TanStack Router route modules in `src/routes`; screens in `src/pages`
Data	TanStack Query for server state; query keys and API calls in `src/lib`
UI	shadcn/ui and Base UI primitives in `components/ui`; Tailwind CSS 4 tokens from `index.css` (`--primary`, `--brand`, ...); no hard-coded colours
Permissions	Use the permission helpers in `src/lib` to show or hide actions; the server remains the authority
Platform	Native features only through `src/platform` so the web build stays free of Tauri calls
Accessibility	Labels on all inputs, keyboard navigation, visible focus, sufficient contrast in light and dark themes
Linting	`npm run lint` must pass

5. Test strategy

5.1 Test pyramid



5.2 Existing automated tests

Test	Module	Covers
`JwtTokenServiceTest`	common-security	Token issue/verify, expiry, secret length
`ClientAddressResolverTest`, `ClientAddressFilterTest`	api-gateway	Trusted proxy handling of `X-Forwarded-For`
`LoginRateLimitIntegrationTest`	api-gateway	Redis token bucket on `/auth/login`
`AuthFlowIntegrationTest`	auth-service	Login, refresh rotation, reuse detection, logout, lockout
`MigrationTest`	db-migrator	All migrations apply on an empty database; legacy baseline
`DbMigratorApplicationTest`	db-migrator	Migrator start-up with placeholders

Test	Module	Covers
`RowLevelSecurityTest`	db-migrator	Policies per role and scope, triggers and invariants

5.3 Running tests

```bash
cd backend
./gradlew build                     # everything; Docker must be running
./gradlew :auth-service:test        # one module
./gradlew :db-migrator:test --tests '*RowLevelSecurityTest'
```

Reports are written to `backend/<module>/build/reports/tests/test/index.html`.

5.4 What to test when you change...

Change	Required tests
A migration, policy, trigger or security function	Extend `RowLevelSecurityTest` with allowed and denied cases for each affected role/scope; `MigrationTest` must pass
Authentication or session logic	Extend `AuthFlowIntegrationTest`
Gateway filters or routes	Unit test plus an integration test where Redis or routing is involved
A service business rule	Unit test of the service with a mocked repository, or an integration test with Testcontainers
A new endpoint	Postman request with example; integration test for permission boundaries
Frontend behaviour	Lint and build; manual check in light/dark themes and at phone width

5.5 Recommended additions

Area	Proposal
Frontend unit tests	Vitest + Testing Library for `src/lib` and complex components
End-to-end	Playwright suite for sign-in, complaint lifecycle, file upload, face enrollment
API contract	Generate an OpenAPI document with springdoc and validate in CI
Load	k6 scripts for sign-in, complaint listing and recognition
Coverage	JaCoCo report with a minimum threshold on `common-*` modules

5.6 Manual acceptance checklist

#	Scenario	Expected
1	Student files a complaint with an attachment	Appears as `pending`; staff of the department notified

#	Scenario	Expected
2	Student edits the complaint after it is reviewed	Rejected with "complaint is already being processed"
3	Teacher of another department opens it	Not visible
4	Teacher resolves it with a note	History shows the change; student notified in real time
5	Internal comment by staff	Not visible to the student
6	Admin creates a role with permissions it holds	Succeeds; granting a permission it does not hold fails
7	Bulk import of the sample CSV	Progress shown; users created with roles and departments
8	Face enroll then recognise	Match shown; audit records both events
9	Self-enroll twice	Second attempt blocked until unlocked
10	Eleven wrong passwords	Account locked; correct password rejected until lockout ends

6. Git workflow

1. Create a branch from the integration branch: `feature/<short-name>` or `fix/<short-name>`.
2. Commit in small, focused steps using Conventional Commits:

Type	Use
`feat:`	New feature
`fix:`	Bug fix
`docs:`	Documentation only
`refactor:`	Code change without behaviour change
`test:`	Tests only
`chore:` / `build:` / `ci:`	Tooling, dependencies, pipelines

3. Keep the branch up to date with `main`; resolve conflicts locally.
4. Open a pull request using the checklist below.

6.1 Pull request checklist

#	Item
1	Description explains *what* and *why*; links the issue
2	`./gradlew build` and `npm run lint && npm run build` pass
3	New or changed behaviour has tests
4	Migrations follow PAI-DB-004 and are never edits of released files

#	Item
5	New configuration added to `.env.example`, compose files and PAI-CFG-007
6	API changes reflected in the Postman collection and PAI-API-003
7	No secrets, `.env` files or generated artefacts committed
8	Screenshots for UI changes (light and dark)

6.2 Review guidelines

Reviewers check correctness, security (especially RLS and permission changes), performance of new queries (indexes, pagination), error handling, naming and test quality. Approve only when you would be comfortable operating the change in production.

7. Documentation

The documentation in `docs/` is written in Markdown (`docs/src/*.md`) and rendered to themed PDFs:

```bash
cd docs
npm install
npm run build          # writes docs/pdf/*.pdf and the combined handbook
```

Update the relevant document in the same pull request as the code change.

DOCUMENT 13

USER GUIDE · PAI-USR-013

User & Administrator Manual

How students, staff and administrators use ProctorAI on the web, desktop and mobile.

1. About ProctorAI
2. Getting started
3. Overview dashboard
4. Complaints
5. Notifications
6. Face recognition
7. Your profile
8. Administration
9. Troubleshooting
10. Glossary

1. About ProctorAI

ProctorAI helps an institution handle complaints, manage people and access, and identify people with on-premises face recognition. Everything you can see and do depends on your **role**. If a menu item or button described here is missing, your role does not include that permission; ask an administrator.

You are...	You typically can
Student / member	File complaints, follow their progress, comment, attach files, manage your notifications and (if allowed) enroll your face once
Staff / teacher	Everything above, plus review, assign and resolve complaints of your departments, see department users, and use face recognition if granted
Administrator	Manage users, departments, roles, custom fields, notification settings and the system monitor; full access to every complaint

2. Getting started

2.1 Supported apps

App	How to get it
Web	Open the address provided by your institution, for example `https://app.example.com`, in Chrome, Edge, Firefox or Safari. On phones you can choose **Add to Home Screen**
Windows, macOS, Linux	Install the package provided by your institution
Android	Install the APK or the app from your institution's store listing
iOS	Install through TestFlight or the App Store

Native apps may ask for the **server address** on first launch. Enter the web address of your institution (it must start with `https://`). The app checks it before saving.

2.2 Signing in

1. Enter your email and password, then select **Sign in**.
2. After repeated wrong passwords, sign-in is paused for your account for a short period (15 minutes by default). Wait and try again, or contact an administrator.
3. Your session renews automatically while you use the app. Select **Sign out** from your profile menu when using a shared device.

2.3 Layout

Area	Purpose
Sidebar	Main navigation: Overview, Complaints, Users, Departments, Roles & access, Custom fields, Face recognition, Notifications, Broadcast & push, System monitor
Bell icon	Unread notification count and recent notifications
Profile menu	My profile, theme (Light, Dark, System), Sign out
Command palette	Press **Ctrl + K** (**⌘ + K** on macOS) to search pages and actions

3. Overview dashboard

The **Overview** page summarises complaints you can see:

Widget	Shows
Totals	All, open, pending, in review, resolved, rejected
My work	Assigned to me, filed by me
Attention	Overdue and critical open complaints, unassigned complaints
Trend	Complaints created and resolved over the last 7 days
Priority	Distribution across low, medium, high and urgent
Departments	Total and open complaints per department
Resolution time	Average hours to resolve

4. Complaints

4.1 Filing a complaint

1. Select **New complaint**.
2. Enter a **title** (up to 200 characters) and a clear **description** (up to 5000).
3. Choose a **category** and **priority** (low, medium, high or urgent). Use *urgent* only for safety or time-critical issues.
4. Choose the **department** responsible, if offered.
5. Optionally add the people the complaint **concerns**. Staff can also identify people from a photo with face recognition.
6. Fill in any additional fields your institution requires.
7. Attach photos or documents by dragging them onto the drop zone.
8. Select **Submit**.

The complaint starts as **Pending**. You receive notifications as it progresses.

4.2 Complaint statuses

Status	Meaning
Pending	Filed and waiting for staff. You can still edit it
In review	Staff are working on it. It can no longer be edited by the reporter
Resolved	Closed with a resolution message
Rejected	Closed without action, with a reason

4.3 Following your complaint

Open **Complaints** and select a complaint to see:

- the details, department, assignee and priority;
- **Activity / history** with every status change and its note;
- **Comments**, where you can add information or answer questions;
- **Files**, with preview and download.

You cannot edit a complaint after staff have started processing it. Add a comment instead.

4.4 Reporter privacy

By default your identity as the reporter is hidden from people who do not need it. Only roles that are allowed to reveal identities, or complaints explicitly marked to reveal the reporter, show your name.

4.5 Processing complaints (staff)

1. Filter **Complaints** by status, priority, department or assignee. Use the search box to find text in titles and descriptions.
2. Open a complaint and select **Assign** to choose the handler.
3. Change the status to **In review** when you start working on it, adding a **note for the history**.
4. Use **internal comments** for discussion among staff; reporters and subjects cannot see them.
5. When finished, set the status to **Resolved** with a **resolution message**, or **Rejected** with the reason.
6. Use **Edit complaint** to correct the category, priority or department if needed.

> TIP
>
> Every status change notifies the reporter and the people involved. A short, clear note in the history saves follow-up questions.

4.6 Attachments

Rule	Value
Maximum size	250 MB per file

Rule	Value
Upload	Shows progress; keep the page open until it completes
Download	Desktop apps open a save dialog; phones open the system viewer
Delete	Allowed for the uploader or staff with permission

5. Notifications

Where	What you get
In the app	Real-time pop-ups and the notification inbox (**Notifications**)
Desktop and mobile apps	System notifications while the app is in the background
Mobile push	When your institution has enabled push and you allow notifications

In **Notifications** you can mark items as read, mark all as read and delete items. Under **Delivery preferences** (*Choose how you get notified*) you can turn real-time or push notifications off, **Mute these types**, and review or remove **Your devices**.

6. Face recognition

Face recognition runs entirely on your institution's server. Photos are not sent to any external service.

6.1 Enrolling your face

1. Open **Face recognition** and choose **Enroll**.
2. Take a photo with the camera or upload one. Use good front lighting, look straight at the camera, remove sunglasses or masks, and make sure only your face is in the frame.
3. Confirm. Your enrollment appears under **Enrolled people**.

> **IMPORTANT**
> You can enroll yourself only once by default. To replace your enrollment, ask a staff member or administrator who manages enrollments to re-enroll or unlock you.

6.2 Recognising people (authorised staff)

Mode	How
Photo	Upload an image; every detected face is boxed and labelled with the matched person or **Unknown**
Live camera	Start **Live camera**; faces are recognised continuously from the device camera

A person is shown only if their face is enrolled, the similarity is above the institution's threshold, and you are allowed to see that person. Otherwise the face is labelled **Unknown**.

6.3 Activity

The activity panel lists recognitions, enrollments, replacements, removals and unlocks with who did them and when. Students can see activity that concerns them.

6.4 Good practice

- Use recognition only for the purposes your institution has approved.
- Treat results as assistance, not proof. Confirm identity by other means before acting.
- Do not save or share annotated images outside ProctorAI.

7. Your profile

Open **My profile** from the profile menu to review your name, email, roles and departments, and to choose the theme. To change your password, open **Users**, select your own account, choose **Edit**, enter a **New password** (at least 8 characters) and save. If you cannot open Users, ask an administrator to set a new password for you.

8. Administration

8.1 Users

Task	Steps
Create a user	**Users** → **New**. Enter name, email, initial password (8+ characters), roles, departments and custom fields
Edit or disable	Open the user → **Edit**. Disabling a user signs them out everywhere
Reset a password	Edit the user → **New password**
Bulk import	**Users** → **Import**. Download the CSV or Excel template, fill in `name, email, password, roles, departments, enabled, verified`, upload, and watch the progress. Fix rows reported with errors and import them again
Bulk delete	Select users in the table → **Delete**
Find users	Search and filter by department, role and verification; choose which columns the table shows, including custom fields

Rules enforced by the system:

- You can manage only yourself and people more junior than you, unless you are a system administrator.
- You cannot change your own roles or disable your own account.

- The last system administrator cannot be removed, disabled or deleted.

8.2 Departments

1. **Departments** → **New**: enter a name and a unique code (for example `CS`).
2. Open a department to add or remove **members** in bulk.
3. Department membership decides what *department-scoped* permissions cover, for example which complaints a teacher sees.

8.3 Roles & access

A role has a **level** and a set of **permissions**. Lower levels are more senior (0 is reserved for system administrators).

Default role	Level	Summary
`system_admin`	0	Unrestricted; cannot be deleted
`admin`	0	Unrestricted; cannot be deleted
`teacher`	20	Department-scoped complaint handling, face recognition, department users
`student`	50	Own complaints, own profile, own notifications, one self-enrollment

The permission matrix lists every area (Users, Complaints, Files, Face recognition, Live camera, Enrollments, Enrollment lock, System monitor…) against the actions **read, write (create), update, delete** and the scopes:

Scope	Covers
Own	Your own records
Department	Records linked to your departments
All	Everything

To create a role: **Roles & access** → **New**, set name, description and level, tick permissions, and save. You can only grant permissions you hold yourself. Tick **Reveal identity** only for roles that must see complaint reporters.

8.4 Custom fields

Add institution-specific information without software changes.

1. **Custom fields** → **New**.
2. Choose where it applies: users, departments, roles or complaints.
3. Enter a key (lower case, for example `hostel_block`), a label and help text.
4. Choose the type: text, number, yes/no, date, single choice or multiple choice.
5. Set validation (required, options, minimum/maximum, maximum length, pattern) and optionally limit it to one role or department.
6. Save. The field appears in the forms and tables immediately.

The key and type cannot be changed later; create a new field instead. Deleting a field deletes its stored values.

8.5 Broadcast & push

Task	Steps
Send an announcement	**Broadcast & push** → **Compose**. Choose individual people, roles and/or departments, enter a title and message, choose priority (low, normal, high), optionally **Include me**, then **Send message**
Enable mobile push	**Push integration** → **Firebase Cloud Messaging**: follow **How to connect**, paste the Firebase service account JSON, enable, then test
Check delivery	**Delivery log** / **Recent deliveries** shows each attempt per channel (Realtime, Push) as sent, failed or skipped with details, plus totals and active devices

8.6 System monitor

System monitor shows the live state of the server: CPU, memory, disks and network; each service with its status and resource use; database and cache health; storage usage; and a list of detected **issues**. Report persistent issues to your technical operator.

9. Troubleshooting

Problem	What to do
"Wrong email or password"	Check caps lock and the email address; after several attempts wait 15 minutes
Signed out unexpectedly	Your session expired or an administrator signed you out; sign in again
A menu item is missing	Your role does not include that permission
"Complaint is already being processed"	Staff have started work; add a comment instead of editing
Upload stops	Check the connection and file size (max 250 MB); keep the page open
No real-time notifications	Reload the page; check notification settings; some networks block live connections
Face not recognised	Re-take the photo with better light and a straight view; ask staff to re-enroll you if needed
App cannot reach the server	Check the server address on the sign-in screen; it must be the institution's `https://` address

10. Glossary

Term	Meaning
Assignee	Staff member responsible for a complaint
Department scope	Permission limited to records linked to your departments
Enrollment	Your stored face template used for recognition
Internal comment	Comment visible only to staff
Level	Seniority of a role; lower is more senior
Reporter	The person who filed a complaint
Subject	A person a complaint is about